

Trivia Night — F

Generated: 2026-07-22

`artifacts/trivia-game/index.html`

```html

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8" />

<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1" />

<title>Trivia Night</title>

<meta name="description" content="Trivia Night — built on Replit. Update this description to reflect the app." />

<meta name="robots" content="index, follow" />

<meta property="og:title" content="Trivia Night" />

<meta property="og:description" content="Trivia Night — built on Replit. Update this description to reflect the app." />

<meta property="og:type" content="website" />

<meta name="twitter:card" content="summary\_large\_image" />

<meta name="twitter:title" content="Trivia Night" />

```
<meta name="twitter:description" content="Trivia Night — built on Replit. Update this description to reflect the app." />
```

```
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
```

```
<link rel="preconnect" href="https://fonts.googleapis.com">
```

```
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

```
<link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:ital,wght@0,300;0,400;0,500;0,600;0,700;1,400&family=Space+Grotesk:wght@400;500;600;700&display=swap"
rel="stylesheet">
```

```
</head>
```

```
<body>
```

```
<div id="root"></div>
```

```
<script type="module" src="/src/main.tsx"></script>
```

```
</body>
```

```
</html>
```

```
...
```

```

```

```
`artifacts/trivia-game/src/index.css`
```

```
```css
```

```
@import "tailwindcss";
```

```
@import "tw-animate-css";
```

```
@plugin "@tailwindcss/typography";
```

```
@custom-variant dark (&:is(.dark *));
```

```
@theme inline {
```

```
  --color-background: hsl(var(--background));
```

```
  --color-foreground: hsl(var(--foreground));
```

```
  --color-border: hsl(var(--border));
```

```
  --color-input: hsl(var(--input));
```

```
  --color-ring: hsl(var(--ring));
```

```
  --color-card: hsl(var(--card));
```

```
  --color-card-foreground: hsl(var(--card-foreground));
```

```
  --color-card-border: hsl(var(--card-border));
```

```
  --color-popover: hsl(var(--popover));
```

```
  --color-popover-foreground: hsl(var(--popover-foreground));
```

```
  --color-popover-border: hsl(var(--popover-border));
```

```
  --color-primary: hsl(var(--primary));
```

```
  --color-primary-foreground: hsl(var(--primary-foreground));
```

```
  --color-secondary: hsl(var(--secondary));
```

```
  --color-secondary-foreground: hsl(var(--secondary-foreground));
```

```
  --color-muted: hsl(var(--muted));
```

```
  --color-muted-foreground: hsl(var(--muted-foreground));
```

```
--color-accent: hsl(var(--accent));  
  
--color-accent-foreground: hsl(var(--accent-foreground));  
  
--color-destructive: hsl(var(--destructive));  
--color-destructive-foreground: hsl(var(--destructive-foreground));  
  
--font-sans: "Plus Jakarta Sans", sans-serif;  
--font-display: "Space Grotesk", sans-serif;  
  
--radius-sm: calc(var(--radius) - 4px);  
--radius-md: calc(var(--radius) - 2px);  
--radius-lg: var(--radius);  
--radius-xl: calc(var(--radius) + 4px);  
}
```

```
:root {  
  --background: 220 60% 5%;  
  --foreground: 0 0% 98%;  
  
  --card: 220 50% 8%;  
  --card-foreground: 0 0% 98%;  
  --card-border: 220 40% 15%;  
  
  --popover: 220 50% 8%;  
  --popover-foreground: 0 0% 98%;  
  --popover-border: 220 40% 15%;
```

--primary: 330 100% 50%; /* Hot Pink */

--primary-foreground: 0 0% 100%;

--secondary: 188 100% 50%; /* Neon Cyan */

--secondary-foreground: 220 60% 5%;

--muted: 220 40% 15%;

--muted-foreground: 220 20% 70%;

--accent: 54 100% 50%; /* Electric Yellow */

--accent-foreground: 220 60% 5%;

--destructive: 350 100% 60%;

--destructive-foreground: 0 0% 100%;

--border: 220 40% 15%;

--input: 220 40% 15%;

--ring: 330 100% 50%;

--radius: 0.5rem;

}

.dark {

--background: 220 60% 5%;

--foreground: 0 0% 98%;

--card: 220 50% 8%;

--card-foreground: 0 0% 98%;

--card-border: 220 40% 15%;

--popover: 220 50% 8%;

--popover-foreground: 0 0% 98%;

--popover-border: 220 40% 15%;

--primary: 330 100% 50%;

--primary-foreground: 0 0% 100%;

--secondary: 188 100% 50%;

--secondary-foreground: 220 60% 5%;

--muted: 220 40% 15%;

--muted-foreground: 220 20% 70%;

--accent: 54 100% 50%;

--accent-foreground: 220 60% 5%;

--destructive: 350 100% 60%;

--destructive-foreground: 0 0% 100%;

--border: 220 40% 15%;

--input: 220 40% 15%;

```
--ring: 330 100% 50%;  
}
```

```
@layer base {  
  * {  
    @apply border-border;  
  }  
  
  body {  
    @apply bg-background text-foreground;  
    font-family: var(--font-sans);  
    background-image:   
      radial-gradient(circle at 15% 50%, rgba(255, 0, 127, 0.05) 0%, transparent 50%),  
      radial-gradient(circle at 85% 30%, rgba(0, 240, 255, 0.05) 0%, transparent 50%);  
    background-attachment: fixed;  
  }  
  
  h1, h2, h3, h4, h5, h6 {  
    font-family: var(--font-display);  
  }  
}
```

```
...
```

```
---
```

```
## `artifacts/trivia-game/src/main.tsx`
```

```
` `` typescript
```

```
import { createRoot } from "react-dom/client";
```

```
import App from "./App";
```

```
import "./index.css";
```

```
createRoot(document.getElementById("root")).render(<App />);
```

```
`` `
```

```
---
```

```
## `artifacts/trivia-game/src/App.tsx`
```

```
` `` typescript
```

```
import { Switch, Route, Router as WouterRouter, Redirect } from "wouter";
```

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
```

```
import { Toaster } from "@components/ui/toaster";
```

```
import { TooltipProvider } from "@components/ui/tooltip";
```

```
import NotFound from "@pages/not-found";
```

```
import { AuthProvider, useAuth } from "./lib/auth";
```

```
import Gate from "./pages/Gate";
```

```
import AdminLogin from "./pages/AdminLogin";
```

```
import Lobby from "./pages/Lobby";
```

```
import GamePlay from "./pages/GamePlay";
```

```
import Results from "./pages/Results";
```

```
import Admin from "./pages/Admin";
```



```
const queryClient = new QueryClient();
```

```
function PlayerRoute({  
  component: Component,  
}): {  
  component: () => React.JSX.Element | null;  
} {  
  const { user } = useAuth();  
  if (!user) return <Redirect to="/" />;  
  return <Component />;  
}
```

```
function Router() {  
  return (  
    <Switch>  
      <Route path="/" component={Gate} />  
      <Route path="/admin-login" component={AdminLogin} />  
      <Route path="/lobby">  
        <PlayerRoute component={Lobby} />  
      </Route>  
      <Route path="/game/:id">  
        <PlayerRoute component={GamePlay} />  
      </Route>  
      <Route path="/results/:gameId">  
        <PlayerRoute component={Results} />  
      </Route>  
    </Switch>  
  );  
}
```

```

    </Route>

    <Route path="/admin" component={Admin} />

    <Route component={NotFound} />

  </Switch>

);
}

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <TooltipProvider>
          <WouterRouter base={import.meta.env.BASE_URL.replace(/\/$/, "")}>
            <Router />
          </WouterRouter>
          <Toaster />
        </TooltipProvider>
      </AuthProvider>
    </QueryClientProvider>
  );
}

export default App;

` ``

```

`artifacts/trivia-game/src/lib/auth.tsx`

```typescript

```
import { useState, createContext, useContext } from "react";
```

```
type AuthState = {
 user: { id: number; name: string } | null;
 isAdmin: boolean;
 loginUser: (user: { id: number; name: string }) => void;
 loginAdmin: () => void;
 logout: () => Promise<void>;
};
```

```
const AuthContext = createContext<AuthState | null>(null);
```

```
export function AuthProvider({ children }: { children: React.ReactNode }) {
 const [user, setUser] = useState<{ id: number; name: string } | null>(() => {
 try {
 const saved = localStorage.getItem("trivia_user");
 return saved ? JSON.parse(saved) : null;
 } catch {
 return null;
 }
 });
```

```
const [isAdmin, setIsAdmin] = useState<boolean>(() => {
 return localStorage.getItem("trivia_admin") === "true";
});
```

```
const loginUser = (u: { id: number; name: string }) => {
 setUser(u);
 localStorage.setItem("trivia_user", JSON.stringify(u));
};
```

```
const loginAdmin = () => {
 setIsAdmin(true);
 localStorage.setItem("trivia_admin", "true");
};
```

```
const logout = async () => {
 try {
 const endpoint = isAdmin ? "/api/admin/logout" : "/api/auth/logout";
 await fetch(endpoint, { method: "POST", credentials: "include" });
 } catch {
 // ignore network errors — clear client state regardless
 }
 setUser(null);
 setIsAdmin(false);
 localStorage.removeItem("trivia_user");
 localStorage.removeItem("trivia_admin");
```

```
};
```

```
return (
```

```
 <AuthContext.Provider value={{ user, isAdmin, loginUser, loginAdmin, logout }}>
```

```
 {children}
```

```
 </AuthContext.Provider>
```

```
);
```

```
}
```

```
export function useAuth() {
```

```
 const context = useContext(AuthContext);
```

```
 if (!context) throw new Error("useAuth must be used within AuthProvider");
```

```
 return context;
```

```
}
```

```
````
```

```
---
```

```
## `artifacts/trivia-game/src/lib/utils.ts`
```

```
````typescript
```

```
import { clsx, type ClassValue } from "clsx"
```

```
import { twMerge } from "tailwind-merge"
```

```
export function cn(...inputs: ClassValue[]) {
```

```
 return twMerge(clsx(inputs))
}
```

```
````
```

```
---
```

```
## `artifacts/trivia-game/src/lib/socket.ts`
```

```
```typescript
```

```
import { io, type Socket } from "socket.io-client";
```

```
// — Typed event maps (mirror server) _____
```

```
export type ServerToClientEvents = {
 "game:started": (payload: { gameId: number; topic: string }) => void;
 "game:ended": (payload: { gameId: number }) => void;
 "answer:submitted": (payload: {
 gameId: number;
 questionId: number;
 playerName: string;
 isCorrect: boolean;
 }) => void;
 "player:joined": (payload: {
 gameId: number;
 playerName: string;
```

```
 participantCount: number;
 }) => void;
};
```

```
export type ClientToServerEvents = {
 "lobby:join": () => void;
 "game:join": (gameId: number) => void;
};
```

```
export type AppSocket = Socket<ServerToClientEvents, ClientToServerEvents>;
```

```
// — Lazy singleton — connect() is called per-hook, disconnect() per-unmount —
```

```
let _socket: AppSocket | null = null;
```

```
export function getSocket(): AppSocket {
 if (!_socket) {
 _socket = io({
 path: "/api/socket.io",
 autoConnect: false,
 reconnection: true,
 reconnectionAttempts: 10,
 reconnectionDelay: 1000,
 reconnectionDelayMax: 5000,
 transports: ["polling", "websocket"],
 });
 }
}
```

```
}
 return _socket;
}
```

```
````
```

```
---
```

```
## `artifacts/trivia-game/src/pages/Gate.tsx`
```

```
```typescript
```

```
import { useState } from "react";
import { useLocation, Link } from "wouter";
import { useAuth } from "../lib/auth";
import { Footer } from "@components/Footer";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Card, CardContent } from "@components/ui/card";
import { useToast } from "@hooks/use-toast";
import { Loader2, AlertCircle } from "lucide-react";
```

```
export default function Gate() {
 const [code, setCode] = useState("");
 const [name, setName] = useState("");
 const [step, setStep] = useState<"code" | "name">("code");
 const [pending, setPending] = useState(false);
```



```
const [codeError, setCodeError] = useState("");
const [nameError, setNameError] = useState("");
const { loginUser } = useAuth();
const [, setLocation] = useLocation();
const { toast } = useToast();

const handleCodeSubmit = async (e: React.FormEvent) => {
 e.preventDefault();
 const trimmed = code.trim().toUpperCase();
 if (!trimmed) {
 setCodeError("Enter your access code");
 return;
 }
 setCodeError("");
 setPending(true);
 try {
 const res = await fetch("/api/auth/verify", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 credentials: "include",
 body: JSON.stringify({ code: trimmed }),
 });
 const data = await res.json();
 if (!data.valid) {
 setCodeError("That code isn't right — try again");
 return;
 }
 }
```

```
}

if (data.role === "admin") {
 // Admin used the player gate — send them to the admin login
 setLocation("/admin-login");
 return;
}

setStep("name");
} catch {
 toast({ variant: "destructive", title: "Connection error — please retry" });
} finally {
 setPending(false);
}
};
```

```
const handleNameSubmit = async (e: React.FormEvent) => {
 e.preventDefault();
 const trimmed = name.trim();
 if (!trimmed) {
 setNameError("Enter your display name");
 return;
 }
 if (trimmed.length > 50) {
 setNameError("Name must be 50 characters or fewer");
 return;
 }
 setNameError("");
```

```
setPending(true);

try {

 const res = await fetch("/api/auth/login", {

 method: "POST",

 headers: { "Content-Type": "application/json" },

 credentials: "include",

 body: JSON.stringify({ code: code.trim().toUpperCase(), name: trimmed }),

 });

 if (res.status === 401) {

 setStep("code");

 setCodeError("Code expired — please re-enter it");

 return;

 }

 if (!res.ok) {

 toast({ variant: "destructive", title: "Something went wrong — please retry" });

 return;

 }

 const user = await res.json();

 loginUser({ id: user.id, name: user.name });

 setLocation("/lobby");

} catch {

 toast({ variant: "destructive", title: "Connection error — please retry" });

} finally {

 setPending(false);

}

};
```

```

return (
 <div className="min-h-[100dvh] flex flex-col items-center justify-center p-4">
 <div className="w-full max-w-md space-y-8 animate-in fade-in slide-in-from-bottom-4 duration-700">
 <div className="text-center space-y-2">
 <h1 className="text-6xl font-bold tracking-tighter text-primary">
 TRIVIA NIGHT
 </h1>
 <p className="text-xl text-muted-foreground">
 The ultimate pub quiz experience
 </p>
 </div>

 <Card className="border-primary/20 bg-card/50 backdrop-blur">
 {step === "code" ? (
 <CardContent className="pt-6">
 <form onSubmit={handleCodeSubmit} className="space-y-4">
 <div className="space-y-2">
 <label className="text-sm font-medium uppercase tracking-widest text-secondary">
 Access Code
 </label>
 <Input
 value={code}
 onChange={(e) => {
 setCode(e.target.value);

```

```

 setCodeError("");
 }}
 placeholder="ENTER CODE"
 autoCapitalize="characters"
 autoComplete="off"
 aria-invalid={!codeError}

 className={`h-14 text-center text-2xl uppercase tracking-widest bg-background
border-primary/30 focus-visible:ring-primary ${
 codeError ? "border-destructive focus-visible:ring-destructive" : ""
 }`}
/>
{codeError && (
 <p className="flex items-center gap-1.5 text-sm text-destructive">
 <AlertCircle className="h-4 w-4 shrink-0" />
 {codeError}
 </p>
)}
</div>
<Button
 type="submit"
 className="w-full h-14 text-lg font-bold tracking-wide"
 disabled={pending}
>
 {pending ? (
 <>
 <Loader2 className="mr-2 h-5 w-5 animate-spin" />

```

```

 CHECKING...
 </>

):(

 "ENTER"

 })

</Button>

</form>

</CardContent>

):(

<CardContent className="pt-6">

 <form onSubmit={handleNameSubmit} className="space-y-4">

 <div className="space-y-2">

 <label className="text-sm font-medium uppercase tracking-widest text-
secondary">

 Display Name

 </label>

 <Input

 value={name}

 onChange={(e) => {

 setName(e.target.value);

 setNameError("");

 }}

 placeholder="YOUR NAME"

 autoFocus

 aria-invalid={!!nameError}

 className={`h-14 text-center text-2xl uppercase tracking-widest bg-background
border-secondary/30 focus-visible:ring-secondary ${

```

```

 nameError ? "border-destructive focus-visible:ring-destructive" : ""
 }` }
 />

 {nameError && (
 <p className="flex items-center gap-1.5 text-sm text-destructive">
 <AlertCircle className="h-4 w-4 shrink-0" />
 {nameError}
 </p>
)}
</div>

<Button
 type="submit"
 className="w-full h-14 text-lg font-bold tracking-wide bg-secondary text-
secondary-foreground hover:bg-secondary/90"
 disabled={pending}
>
 {pending ? (
 <>
 <Loader2 className="mr-2 h-5 w-5 animate-spin" />
 JOINING...
 </>
) : (
 "JOIN LOBBY"
)}
</Button>

<button

```

```

 type="button"

 onClick={() => setStep("code")}

 className="w-full text-sm text-muted-foreground hover:text-foreground
transition-colors"

 >

 Wrong code? Go back

 </button>

</form>

</CardContent>

)}

</Card>

<p className="text-center text-sm text-muted-foreground">
 Hosting tonight?{" "}

 <Link

 href="/admin-login"

 className="text-secondary underline-offset-4 hover:underline font-medium"

 >

 Admin login

 </Link>

</p>

</div>

<Footer />

</div>

);

}

```



```

`artifacts/trivia-game/src/pages/Lobby.tsx`

```typescript

import { useMemo } from "react";

import { useLocation } from "wouter";

import { motion, AnimatePresence } from "framer-motion";

import { Footer } from "@components/Footer";

import {

  useListGames,

  useGetStatsSummary,

  getGetStatsSummaryQueryKey,

  useJoinGame,

  getListGamesQueryKey,

  useListGroupParticipants,

  getListGameParticipantsQueryKey,

} from "@workspace/api-client-react";

import { useQueryClient } from "@tanstack/react-query";

import { useAuth } from "../lib/auth";

import { useLobbySocket } from "../hooks/useGameSocket";

import { Button } from "@components/ui/button";

import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";

```
import { Badge } from "@components/ui/badge";
```

```
import { useToast } from "@hooks/use-toast";
```

```
import {
```

```
 Trophy,
```

```
 Users,
```

```
 Gamepad2,
```

```
 Zap,
```

```
 LogOut,
```

```
 ChevronRight,
```

```
 Radio,
```

```
 Hourglass,
```

```
 Crown,
```

```
 Clock,
```

```
 PlayCircle,
```

```
} from "lucide-react";
```

```
const difficultyColors: Record<string, string> = {
```

```
 easy: "bg-secondary/15 text-secondary border-secondary/40",
```

```
 medium: "bg-accent/15 text-accent border-accent/40",
```

```
 hard: "bg-primary/15 text-primary border-primary/40",
```

```
};
```

```
// Fetches participants for a single game and checks if user is already in it
```

```
function useGameParticipation(gameId: number | null, userId: number) {
```

```
 const { data: participants } = useListGameParticipants(gameId ?? 0, {
```

```
 query: {
```

```

 enabled: !!gameId,
 queryKey: getListGameParticipantsQueryKey(gameId ?? 0),
 refetchInterval: 5000,
 },
});
const joined = useMemo(
 () => (participants ?? []).some((p) => p.userId === userId),
 [participants, userId],
);
return { participants: participants ?? [], joined };
}

```

// Active game card — fetches its own participants

```

function ActiveGameCard({
 game,
 userId,
 onJoin,
 joining,
}: {
 game: { id: number; topic: string; difficulty: string; questionCount: number; status: string };
 userId: number;
 onJoin: (gameId: number, alreadyJoined: boolean) => void;
 joining: boolean;
}) {
 const { participants, joined } = useGameParticipation(game.id, userId);
 const sorted = [...participants].sort((a, b) => b.totalScore - a.totalScore);

```

```

return (
 <motion.div
 initial={{ opacity: 0, y: 16 }}
 animate={{ opacity: 1, y: 0 }}
 className="space-y-4"
 >
 {/* Game banner */}
 <Card className="border-2 border-primary/50 bg-primary/5 backdrop-blur">
 <CardContent className="p-5 space-y-4">
 <div className="flex items-start justify-between gap-3 flex-wrap">
 <div className="space-y-1">
 <div className="flex items-center gap-1.5 text-xs font-bold uppercase tracking-
widest text-primary">

 Live Now
 </div>
 <h2 className="text-2xl font-bold tracking-tight">{game.topic}</h2>
 <div className="flex items-center gap-2 text-sm text-muted-foreground">
 {game.questionCount} {game.questionCount === 1 ? "question" :
"questions"}
 .
 <Badge
 variant="outline"
 className={`uppercase text-[10px] ${difficultyColors[game.difficulty]} ?? ""}` }
 >
 {game.difficulty}

```

```

 </Badge>
 </div>
</div>
<Button
 size="lg"
 className="font-bold h-12 px-6"
 disabled={joining}
 onClick={() => onJoin(game.id, joined)}
>
 {joined ? (
 <>
 <PlayCircle className="mr-2 h-5 w-5" /> Continue Game
 </>
) : (
 <>
 Join Game <ChevronRight className="ml-1 h-5 w-5" />
 </>
)}
</Button>
</div>

```

```

{/* Participants */}
{sorted.length > 0 && (
 <div className="space-y-2">
 <p className="text-xs font-semibold uppercase tracking-widest text-muted-foreground flex items-center gap-1.5">

```

```
 <Users className="h-3.5 w-3.5" /> {sorted.length} player{sorted.length !== 1 ? "s" :
 ""} in the game
```

```
 </p>
```

```
 <div className="grid grid-cols-2 sm:grid-cols-3 gap-1.5">
```

```
 {sorted.map((p, i) => (
```

```
 <div
```

```
 key={p.id}
```

```
 className={`flex items-center gap-2 rounded-md px-2.5 py-1.5 text-sm ${
```

```
 p.userId === userId
```

```
 ? "bg-primary/15 border border-primary/30"
```

```
 : "bg-background/40 border border-transparent"
```

```
 `} }
```

```
 >
```

```
 {i === 0 ? (
```

```
 <Crown className="h-3.5 w-3.5 text-accent shrink-0" />
```

```
) : (
```

```
 <span className="w-3.5 text-center text-xs font-bold text-muted-foreground
shrink-0">
```

```
 {i + 1}
```

```

```

```
)}
```

```

```

```
 {p.userName}
```

```
 {p.userId === userId && (
```

```
 (you)
```

```
)}
```

```

```

```

 <span className="ml-auto font-bold tabular-nums text-accent text-xs shrink-
0">
 {p.totalScore}

 </div>
)}
</div>
</div>
)}
</CardContent>
</Card>
</motion.div>
);
}

```

```

export default function Lobby() {
 const { user, logout } = useAuth();
 const [, setLocation] = useLocation();
 const { toast } = useToast();
 const queryClient = useQueryClient();
 const userId = user?.id ?? 0;

 const { data: games, isLoading } = useListGames(undefined, {
 query: { refetchInterval: 5000, queryKey: getListGamesQueryKey() },
 });

 const { data: stats } = useGetStatsSummary({

```

```
 query: { refetchInterval: 10000, queryKey: getGetStatsSummaryQueryKey() },
});
```

```
const joinGame = useJoinGame();
```

```
useLobbySocket({
 onGameStarted: ({ topic }) => {
 queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });
 toast({
 title: "🎮 Game Starting!",
 description: ` "${topic}" is now live — join now! `,
 });
 },
});
```

```
const activeGames = useMemo(
 () => (games ?? []).filter((g) => g.status === "active"),
 [games],
);
```

```
const waitingGames = useMemo(
 () => (games ?? []).filter((g) => g.status === "waiting"),
 [games],
);
```

```
const handleJoin = (gameId: number, alreadyJoined: boolean) => {
 if (alreadyJoined) {
```



```
 setLocation(`/game/${gameId}`);
 return;
 }
 if (!user) return;
 joinGame.mutate(
 { gameId, data: {} },
 {
 onSuccess: () => {
 queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });
 setLocation(`/game/${gameId}`);
 },
 onError: () =>
 toast({ variant: "destructive", title: "Could not join game" }),
 },
);
};
```

```
const statCards = [
 { label: "Games Hosted", value: stats?.totalGames ?? 0, icon: Gamepad2, color: "text-primary" },
 { label: "Live Games", value: stats?.activeGames ?? 0, icon: Radio, color: "text-secondary" },
 { label: "Players", value: stats?.totalPlayers ?? 0, icon: Users, color: "text-accent" },
 { label: "Answers Fired", value: stats?.totalAnswers ?? 0, icon: Zap, color: "text-primary" },
];
```

```
if (!user) return null;
```

```

return (
 <div className="min-h-[100dvh] p-4 md:p-8">
 <div className="mx-auto max-w-4xl space-y-8">
 {/* Header */}
 <header className="flex items-center justify-between gap-4 animate-in fade-in slide-in-from-top-2 duration-500">
 <div>
 <h1 className="text-3xl md:text-4xl font-bold tracking-tighter text-primary">
 THE LOBBY
 </h1>
 <p className="text-muted-foreground">
 Playing as{" "}
 {user.name}
 </p>
 </div>
 <Button
 variant="ghost"
 onClick={async () => {
 await logout();
 setLocation("/");
 }}
 className="text-muted-foreground hover:text-foreground"
 >
 <LogOut className="mr-2 h-4 w-4" /> Leave
 </Button>
 </header>
 </div>
 </div>
)

```

```
</header>
```

```
{/* Stat strip */}
```

```
<div className="grid grid-cols-2 md:grid-cols-4 gap-3">
```

```
{statCards.map((s, i) => (
```

```
 <motion.div
```

```
 key={s.label}
```

```
 initial={{ opacity: 0, y: 12 }}
```

```
 animate={{ opacity: 1, y: 0 }}
```

```
 transition={{ delay: i * 0.07 }}
```

```
 >
```

```
 <Card className="border-card-border bg-card/60 backdrop-blur">
```

```
 <CardContent className="p-4 flex items-center gap-3">
```

```
 <s.icon className={`h-6 w-6 shrink-0 ${s.color}`} />
```

```
 <div>
```

```
 <div className="text-2xl font-bold tabular-nums">{s.value}</div>
```

```
 <div className="text-xs uppercase tracking-widest text-muted-foreground">
```

```
 {s.label}
```

```
 </div>
```

```
 </div>
```

```
 </CardContent>
```

```
 </Card>
```

```
 </motion.div>
```

```
)})
```

```
</div>
```

```

 { /* Main game area */ }

 <AnimatePresence mode="wait">
 {isLoading ? (
 <motion.div
 key="loading"
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 exit={{ opacity: 0 }}
 className="grid gap-3 md:grid-cols-2"
 >
 {[0, 1].map((i) => (
 <div key={i} className="h-36 rounded-lg bg-card/60 animate-pulse" />
))}
 </motion.div>
) : activeGames.length > 0 ? (
 <motion.div
 key="active"
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 exit={{ opacity: 0 }}
 className="space-y-6"
 >
 <h2 className="text-lg font-semibold uppercase tracking-widest text-muted-foreground">
 Game in Progress
 </h2>

```

```

{activeGames.map((game) => (
 <ActiveGameCard
 key={game.id}
 game={game}
 userId={userId}
 onJoin={handleJoin}
 joining={joinGame.isPending}
 />
))}
</motion.div>
) : waitingGames.length > 0 ? (
 <motion.div
 key="waiting"
 initial={{ opacity: 0, y: 8 }}
 animate={{ opacity: 1, y: 0 }}
 exit={{ opacity: 0 }}
 >
 <Card className="border-secondary/30 bg-secondary/5">
 <CardContent className="py-14 text-center space-y-4">
 <div className="relative mx-auto w-fit">
 <Hourglass className="h-14 w-14 text-secondary/60" />
 <span className="absolute -top-1 -right-1 h-3 w-3 rounded-full bg-secondary
animate-pulse" />
 </div>
 <div>
 <h3 className="text-2xl font-bold tracking-tight text-secondary">

```

```

 Waiting for the next game
 </h3>

 <p className="text-muted-foreground mt-1 max-w-md mx-auto">
 Your host is preparing{" "}

 {waitingGames[0]!.topic}

 . Sit tight — the game will go live any moment.
 </p>
</div>

<div className="flex items-center justify-center gap-2 text-sm text-muted-foreground">
 <Clock className="h-4 w-4" />
 This page refreshes automatically every 5 seconds
</div>

{/* Show upcoming games */}
{waitingGames.length > 0 && (
 <div className="pt-2 space-y-2 max-w-sm mx-auto text-left">
 {waitingGames.map((g) => (
 <div
 key={g.id}
 className="flex items-center justify-between rounded-lg border border-secondary/20 bg-card/60 px-4 py-2.5 text-sm"
 >
 {g.topic}
 <div className="flex items-center gap-2">


```

```

 {g.questionCount} questions

 <Badge
 variant="outline"
 className={`uppercase text-[10px] ${difficultyColors[g.difficulty] ?? ""}` }
 >
 {g.difficulty}
 </Badge>
 </div>
 </div>
)}
</div>
)}
</CardContent>
</Card>
</motion.div>
):(
 <motion.div
 key="empty"
 initial={{ opacity: 0, y: 8 }}
 animate={{ opacity: 1, y: 0 }}
 exit={{ opacity: 0 }}
 >
 <Card className="border-dashed border-primary/30 bg-card/40">
 <CardContent className="py-16 text-center space-y-3">
 <Trophy className="mx-auto h-12 w-12 text-primary/50" />

```

```

 <h3 className="text-2xl font-bold tracking-tight">
 The stage is set
 </h3>

 <p className="text-muted-foreground max-w-md mx-auto">
 Your host is preparing tonight's trivia. Games will appear
 here the moment they're ready — hang tight and warm up those
 brain cells.
 </p>

 <div className="flex items-center justify-center gap-2 text-sm text-muted-
foreground pt-2">
 <Clock className="h-4 w-4" />
 Checking for games every 5 seconds
 </div>

 </CardContent>

</Card>

</motion.div>

)}

</AnimatePresence>

</div>

<Footer />

</div>

);

}

` ``

```



---

## `artifacts/trivia-game/src/pages/GamePlay.tsx`

```typescript

```
import { useMemo, useState, useEffect, useRef } from "react";
import { useLocation, useParams } from "wouter";
import { motion, AnimatePresence } from "framer-motion";
import { Footer } from "@components/Footer";
import { useQuery, useQueryClient } from "@tanstack/react-query";
import {
  useGetGame,
  getGetGameQueryKey,
  useListGameQuestions,
  getListGameQuestionsQueryKey,
  useListGameParticipants,
  getListGameParticipantsQueryKey,
  useListUserAnswers,
  getListUserAnswersQueryKey,
  useSubmitAnswer,
} from "@workspace/api-client-react";
import type { Question } from "@workspace/api-client-react";
import { useAuth } from "../lib/auth";
import { useGameSocket } from "../hooks/useGameSocket";
import { Button } from "@components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
```

```
import { Input } from "@components/ui/input";
import { Badge } from "@components/ui/badge";
import { Progress } from "@components/ui/progress";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@components/ui/select";
import { useToast } from "@hooks/use-toast";
import {
  ArrowLeft,
  Trophy,
  Crown,
  CheckCircle2,
  XCircle,
  Sparkles,
  Loader2,
  Star,
  Clock,
  Check,
  X,
  Minus,
} from "lucide-react";
```

// — Types

```
type Feedback = {  
  isCorrect: boolean;  
  pointsEarned: number;  
  totalScore: number;  
  timeTaken: string;  
  questionId: number;  
  questionType: string;  
};
```

```
type QuestionStats = {  
  totalAnswered: number;  
  correctCount: number;  
};
```

```
const CHOICE_LABELS = ["A", "B", "C", "D", "E", "F"];
```

// — Helpers

```
function formatCorrectAnswer(type: string, answer: string): string {  
  if (type === "matching") {  
    return answer  
      .split("|")  
      .map((pair) => {
```

```

    const idx = pair.indexOf(":");
    return idx === -1 ? pair : `${pair.slice(0, idx)} → ${pair.slice(idx + 1)}`;
  })
  .join(", ");
}

if (type === "true_false") {
  return answer.charAt(0).toUpperCase() + answer.slice(1);
}

return answer;
}

```

// — Full-screen feedback overlay —————

```

function FeedbackOverlay({
  feedback,
  gameId,
  userId,
  isLastQuestion,
  onNext,
  onViewResults,
}): {
  feedback: Feedback;
  gameId: number;
  userId: number;
  isLastQuestion: boolean;
  onNext: () => void;
}

```

```
onViewResults: () => void;
}) {
  const isPartial = !feedback.isCorrect && feedback.pointsEarned > 0;

  const { data: questionStats } = useQuery<QuestionStats>({
    queryKey: ["question-stats", gameId, feedback.questionId],
    queryFn: () =>
      fetch(`/api/games/${gameId}/questions/${feedback.questionId}/answers`).then(
        (r) => r.json(),
      ),
    refetchInterval: 3000,
  });
```

```
const bgClass = feedback.isCorrect
  ? "from-emerald-950 to-emerald-900"
  : isPartial
    ? "from-amber-950 to-amber-900"
    : "from-red-950 to-red-900";
```

```
const accentClass = feedback.isCorrect
  ? "text-emerald-400"
  : isPartial
    ? "text-amber-400"
    : "text-red-400";
```

```
const borderClass = feedback.isCorrect
```

```
? "border-emerald-700"
```

```
: isPartial
```

```
? "border-amber-700"
```

```
: "border-red-700";
```

```
return (
```

```
<motion.div
```

```
  initial={{ y: "100%" }}
```

```
  animate={{ y: 0 }}
```

```
  exit={{ y: "100%" }}
```

```
  transition={{ type: "spring", stiffness: 280, damping: 30 }}
```

```
  className={`fixed inset-0 z-50 bg-gradient-to-b ${bgClass} flex flex-col overflow-y-
  auto` }
```

```
>
```

```
<div className="flex flex-col min-h-full max-w-lg mx-auto w-full px-5 py-8 gap-6">
```

```
{/* — Result hero — */}
```

```
<div className="flex flex-col items-center text-center gap-3 pt-4">
```

```
<motion.div
```

```
  initial={{ scale: 0.3, opacity: 0 }}
```

```
  animate={{ scale: 1, opacity: 1 }}
```

```
  transition={{ type: "spring", stiffness: 400, damping: 18, delay: 0.05 }}
```

```
>
```

```
{feedback.isCorrect ? (
```

```
  <CheckCircle2 className="h-24 w-24 text-emerald-400" />
```

```
): isPartial ? (
```

```

    <Star className="h-24 w-24 text-amber-400" />
  ): (
    <XCircle className="h-24 w-24 text-red-400" />
  )}
</motion.div>

<motion.h2
  initial={{ opacity: 0, y: 10 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ delay: 0.12 }}
  className={`text-5xl font-black tracking-tight ${accentClass}`}
>
  {feedback.isCorrect
    ? "NAILED IT!"
    : isPartial
      ? "PARTIAL!"
      : "NOT QUITE"}
</motion.h2>

<motion.div
  initial={{ opacity: 0, scale: 0.8 }}
  animate={{ opacity: 1, scale: 1 }}
  transition={{ type: "spring", stiffness: 350, damping: 20, delay: 0.2 }}
  className="space-y-1"
>
  {feedback.pointsEarned > 0 ? (

```

```

<p className={`text-4xl font-bold ${accentClass}`}>
  +{feedback.pointsEarned} pts
</p>

): (
  <p className="text-3xl font-bold text-white/40">0 pts</p>
)}
{isPartial && (
  <p className="text-sm text-amber-300/70">Partial credit earned</p>
)}
</motion.div>

<div className="flex items-center gap-1.5 text-white/50 text-sm">
  <Clock className="h-4 w-4" />
  <span>Answered in {feedback.timeTaken}s</span>
  <span className="mx-1.5 text-white/20">·</span>
  <span className="text-white/50">Total: {feedback.totalScore} pts</span>
</div>

</div>

{/* — Per-question aggregate stats — */}
<div className={`rounded-xl border ${borderClass} bg-white/5 overflow-hidden`} >
  <div className="px-4 py-3 border-b border-white/10">
    <p className="text-xs font-bold uppercase tracking-widest text-white/50">
      This question
    </p>

```



```

</div>

{!questionStats ? (
  <p className="text-white/30 text-sm text-center py-6">
    Waiting for answers...
  </p>
): (
  <div className="flex divide-x divide-white/10">
    <div className="flex-1 flex flex-col items-center py-4 gap-1">
      <span className="text-2xl font-black text-white tabular-nums">
        {questionStats.totalAnswered}
      </span>
      <span className="text-[11px] uppercase tracking-widest text-white/40 font-
semibold">
        Answered
      </span>
    </div>
    <div className="flex-1 flex flex-col items-center py-4 gap-1">
      <span className="text-2xl font-black text-emerald-400 tabular-nums">
        {questionStats.correctCount}
      </span>
      <span className="text-[11px] uppercase tracking-widest text-white/40 font-
semibold">
        Correct
      </span>
    </div>
    <div className="flex-1 flex flex-col items-center py-4 gap-1">
      <span className="text-2xl font-black text-red-400 tabular-nums">

```

```

        {questionStats.totalAnswered - questionStats.correctCount}

    </span>

    <span className="text-[11px] uppercase tracking-widest text-white/40 font-
semibold">

        Wrong

    </span>

</div>

</div>

)}

</div>

{ /* — Next button — */}

<div className="mt-auto pt-2">

    <Button

        onClick={isLastQuestion ? onViewResults : onNext}

        size="lg"

        className="w-full h-14 text-lg font-bold bg-white/15 hover:bg-white/25 text-white
border border-white/20"

        >

            {isLastQuestion ? "View Final Results →" : "Next Question →"}

        </Button>

    </div>

</div>

</motion.div>

);

}

```

// — Question sub-components

```
function MultipleChoiceQuestion({
  question,
  onSubmit,
  disabled,
}: {
  question: Question;
  onSubmit: (a: string) => void;
  disabled: boolean;
}) {
  const opts = question.options as { choices?: string[] } | null;
  const choices = opts?.choices ?? [];
  const [selected, setSelected] = useState<string | null>(null);
  useEffect(() => { setSelected(null); }, [question.id]);

  return (
    <div className="space-y-3">
      <div className="grid gap-3 sm:grid-cols-2">
        {choices.map((choice, i) => {
          const isSel = selected === choice;
          return (
            <motion.button
              key={choice}
              initial={{ opacity: 0, x: -8 }}
            >
```

```

    animate={{ opacity: 1, x: 0 }}
    transition={{ delay: i * 0.06 }}
    disabled={disabled}
    onClick={() => !disabled && setSelected(choice)}
    className={`
      flex items-center gap-3 w-full rounded-xl border-2 px-4 py-4 text-left font-medium
      transition-all duration-150 focus:outline-none
      ${isSel
        ? "border-primary bg-primary/15 shadow-sm shadow-primary/20"
        : "border-card-border bg-card/60 hover:border-primary/40 hover:bg-primary/5"}
    }
    ${disabled ? "opacity-70 cursor-not-allowed" : "cursor-pointer"}
  `}
>
<span
  className={` flex h-8 w-8 shrink-0 items-center justify-center rounded-full
    text-sm font-bold border-2 transition-colors
    ${isSel ? "border-primary bg-primary text-primary-foreground" : "border-muted-foreground/30 text-muted-foreground"} ` }
>
  {CHOICE_LABELS[i]}
</span>
<span className="leading-snug">{choice}</span>
</motion.button>
);
}}

```

```

</div>

<AnimatePresence>
  {selected && (
    <motion.div
      initial={{ opacity: 0, y: 6 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0, y: -4 }}
    >
      <Button
        className="w-full h-12 font-bold"
        disabled={disabled}
        onClick={() => onSubmit(selected)}
      >
        {disabled
          ? <><Loader2 className="mr-2 h-4 w-4 animate-spin" /> Submitting...</>
          : <>Confirm: {selected}</>}
        </Button>
      </motion.div>
    )}
  </AnimatePresence>
</div>

);
}

```

```

function TrueFalseQuestion({
  question,

```

```

onSubmit,
disabled,
}: {
  question: Question;
  onSubmit: (a: string) => void;
  disabled: boolean;
} {
  const [selected, setSelected] = useState<"true" | "false" | null>(null);
  useEffect(() => { setSelected(null); }, [question.id]);

  return (
    <div className="space-y-3">
      <div className="grid grid-cols-2 gap-3">
        {[["true", "false"] as const].map((val) => {
          const isSel = selected === val;
          return (
            <motion.button
              key={val}
              initial={{ opacity: 0, scale: 0.95 }}
              animate={{ opacity: 1, scale: 1 }}
              transition={{ delay: val === "true" ? 0 : 0.06 }}
              disabled={disabled}
              onClick={() => !disabled && setSelected(val)}
              className={`
                h-20 rounded-xl border-2 text-xl font-bold transition-all duration-150 focus:outline-
                none

```

```

      ${isSel && val === "true"
        ? "border-secondary bg-secondary/20 text-secondary shadow-sm shadow-
secondary/20"
        : isSel && val === "false"
          ? "border-destructive bg-destructive/15 text-destructive"
          : val === "true"
            ? "border-card-border bg-card/60 hover:border-secondary/50 hover:bg-
secondary/10"
            : "border-card-border bg-card/60 hover:border-destructive/50 hover:bg-
destructive/10"}
      ${disabled ? "opacity-70 cursor-not-allowed" : "cursor-pointer"}
    `}
  >
    {val === "true" ? "✓ TRUE" : "✗ FALSE"}
  </motion.button>
);
}}}
</div>
<AnimatePresence>
  {selected && (
    <motion.div initial={{ opacity: 0, y: 6 }} animate={{ opacity: 1, y: 0 }} exit={{ opacity: 0 }}>
      <Button
        className={`w-full h-12 font-bold ${selected === "true"
          ? "bg-secondary hover:bg-secondary/90 text-secondary-foreground"
          : "bg-destructive hover:bg-destructive/90 text-destructive-foreground"} `}
        disabled={disabled}
        onClick={() => onSubmit(selected)}
      >

```

```

    >
    {disabled
      ? <><Loader2 className="mr-2 h-4 w-4 animate-spin" /> Submitting...</>
      : <>Confirm: {selected === "true" ? "True" : "False"}</>
    </Button>
  </motion.div>
)}
</AnimatePresence>
</div>
);
}

```

```

function WriteInQuestion({
  question,
  onSubmit,
  disabled,
}: {
  question: Question;
  onSubmit: (a: string) => void;
  disabled: boolean;
}) {
  const [val, setVal] = useState("");
  useEffect(() => { setVal(""); }, [question.id]);
  return (
    <form className="space-y-3" onSubmit={(e) => { e.preventDefault(); if (val.trim())
onSubmit(val.trim()); }}>

```



```
<Input value={val} onChange={(e) => setVal(e.target.value)} placeholder="Type your answer..." className="h-14 text-lg" disabled={disabled} autoFocus />
```

```
<Button type="submit" className="w-full h-12 font-bold" disabled={!val.trim() || disabled}>
```

```
{disabled ? <><Loader2 className="mr-2 h-4 w-4 animate-spin" /> Submitting...</> : "Lock It In"}
```

```
</Button>
```

```
</form>
```

```
);
```

```
}
```

```
function ImageQuestion({
```

```
  question,
```

```
  onSubmit,
```

```
  disabled,
```

```
}: {
```

```
  question: Question;
```

```
  onSubmit: (a: string) => void;
```

```
  disabled: boolean;
```

```
}) {
```

```
  const [val, setVal] = useState("");
```

```
  useEffect(() => { setVal(""); }, [question.id]);
```

```
  return (
```

```
    <div className="space-y-4">
```

```
      {question.imageUrl && (
```

```
        <motion.div initial={{ opacity: 0, scale: 0.97 }} animate={{ opacity: 1, scale: 1 }}
```

```

        className="overflow-hidden rounded-xl border border-card-border bg-
background/60">

        <img src={question.imageUrl} alt="Identify this" className="w-full max-h-80 object-
contain" />

        </motion.div>

    )}

    <form className="space-y-3" onSubmit={(e) => { e.preventDefault(); if (val.trim())
onSubmit(val.trim()); }}>

        <Input value={val} onChange={(e) => setVal(e.target.value)} placeholder="Your
answer..." className="h-14 text-lg" disabled={disabled} autoFocus />

        <Button type="submit" className="w-full h-12 font-bold" disabled={!val.trim() ||
disabled}>

            {disabled ? <><Loader2 className="mr-2 h-4 w-4 animate-spin" /> Submitting...</> :
"Lock It In"}

        </Button>

    </form>

</div>

);
}

```

```

function MatchingBoard({
    question,
    onSubmit,
    disabled,
}): {
    question: Question;
    onSubmit: (a: string) => void;
    disabled: boolean;

```

```

}) {

  const pairs = useMemo(() => {

    const opts = question.options as { pairs?: { left: string; right: string }[] } | null;

    return opts?.pairs ?? [];

  }, [question.options]);

  const shuffledRights = useMemo(() => [...pairs.map((p) => p.right)].sort(() =>
Math.random() - 0.5), [pairs]);

  const [choices, setChoices] = useState<Record<string, string>>({});

  const allChosen = pairs.length > 0 && pairs.every((p) => choices[p.left]);

  const submit = () => {

    const answer = pairs

      .map((p) => `${p.left}:${choices[p.left]}`)

      .sort((a, b) => a.localeCompare(b))

      .join("|");

    onSubmit(answer);

  };

  return (

    <div className="space-y-3">

      <p className="text-xs text-muted-foreground">Match each item on the left with its
answer on the right.</p>

      {pairs.map((p) => (

        <div key={p.left} className="flex items-center gap-3">

          <div className="flex-1 rounded-md border border-card-border bg-background/60 px-
4 py-3 font-medium text-sm">{p.left}</div>

```

```

    <span className="text-muted-foreground shrink-0"></span>

    <Select value={choices[p.left] ?? ""} onChange={(v) => setChoices((c) => ({ ...c,
[p.left]: v }))) disabled={disabled}>

      <SelectTrigger className={`flex-1 ${choices[p.left] ? "border-secondary/50 bg-
secondary/5" : ""}`}>

        <SelectValue placeholder="Match with..." />

      </SelectTrigger>

      <SelectContent>

        {shuffledRights.map((r) => (<SelectItem key={r} value={r}>{r}</SelectItem>))}

      </SelectContent>

    </Select>

  </div>

  )))

  <Button className="w-full h-12 font-bold mt-2" disabled={!allChosen || disabled}
onClick={submit}>

    {disabled ? <><Loader2 className="mr-2 h-4 w-4 animate-spin" /> Submitting...</> :
"Lock In Matches"}

  </Button>

</div>

);

}

```

// — Main Gameplay page

```

export default function Gameplay() {

  const params = useParams<{ id: string }>();

```

```
const gameId = Number(params.id);

const { user } = useAuth();

const userId = user?.id ?? 0;

const [, setLocation] = useLocation();

const { toast } = useToast();

const queryClient = useQueryClient();
```

```
const [feedback, setFeedback] = useState<Feedback | null>(null);

const questionStartRef = useRef<number>(Date.now());
```

```
useGameSocket(gameId || null, {

  onAnswerSubmitted: ({ playerName, isCorrect }) => {

    queryClient.invalidateQueries({ queryKey: getListGameParticipantsQueryKey(gameId)

});

    toast({

      title: isCorrect ? `  ${playerName} got it right!` : `  ${playerName} just answered`,

      duration: 2500,

    });

  },

  onGameEnded: () => {

    toast({ title: "  Game over! Redirecting to results..." });

    setTimeout(() => setLocation(`/results/${gameId}`), 1500);

  },

});
```

```
const { data: game } = useGetGame(gameId, {
```

```

    query: { enabled: !!gameId, queryKey: getGetGameQueryKey(gameId) },
  });

  const { data: questions } = useListGameQuestions(gameId, {
    query: { enabled: !!gameId, queryKey: getListGameQuestionsQueryKey(gameId),
    refetchInterval: 10000 },
  });

  const { data: myAnswers } = useListUserAnswers(gameId, userId, {
    query: { enabled: !!gameId && !!userId, queryKey: getListUserAnswersQueryKey(gameId,
    userId) },
  });

  const { data: participants } = useListGameParticipants(gameId, {
    query: { enabled: !!gameId, queryKey: getListGameParticipantsQueryKey(gameId),
    refetchInterval: 5000 },
  });

  const submitAnswer = useSubmitAnswer();

  const sorted = useMemo(
    () => [...(questions ?? [])].sort((a, b) => a.orderIndex - b.orderIndex),
    [questions],
  );

  const answeredIds = useMemo(
    () => new Set((myAnswers ?? []).map((a) => a.questionId)),
    [myAnswers],
  );

  const current = sorted.find((q) => !answeredIds.has(q.id));

```

```

const answeredCount = sorted.filter((q) => answeredIds.has(q.id)).length;
const total = sorted.length;
const isLastQuestion = !!current && sorted.indexOf(current) === total - 1;

const sortedParticipants = useMemo(
  () => [...(participants ?? [])].sort((a, b) => b.totalScore - a.totalScore),
  [participants],
);
const myScore = participants?.find((p) => p.userId === userId)?.totalScore ?? 0;
const myRank = sortedParticipants.findIndex((p) => p.userId === userId) + 1;

// Reset timer when question changes
useEffect(() => {
  if (current?.id) {
    questionStartRef.current = Date.now();
  }
}, [current?.id]);

const handleSubmit = (question: Question, userAnswer: string) => {
  if (!userAnswer.trim() || submitAnswer.isPending) return;
  const timeTaken = ((Date.now() - questionStartRef.current) / 1000).toFixed(1);

  submitAnswer.mutate(
    { gameId, data: { questionId: question.id, userAnswer } },
    {
      onSuccess: (res) => {

```

```

    setFeedback({
      isCorrect: res.isCorrect,
      pointsEarned: res.pointsEarned,
      totalScore: res.totalScore,
      timeTaken,
      questionId: question.id,
      questionType: question.questionType,
    });
    queryClient.invalidateQueries({ queryKey: getListGameParticipantsQueryKey(gameId)
  });
},
  onError: (err: unknown) => {
    const msg = err instanceof Error ? err.message : String(err);
    if (msg.includes("409") || msg.toLowerCase().includes("already answered")) {
      nextQuestion();
      return;
    }
    toast({ variant: "destructive", title: "Could not submit answer", description: "Please try
again." });
  },
},
);
};

```

```

const nextQuestion = () => {
  setFeedback(null);

```



```
    queryClient.invalidateQueries({ queryKey: getListUserAnswersQueryKey(gameId, userId)
});
};
```

```
const renderQuestion = (q: Question) => {
  const sub = { question: q, disabled: submitAnswer.isPending };
  switch (q.questionType) {
    case "multiple_choice":
      return <MultipleChoiceQuestion {...sub} onSubmit={(a) => handleSubmit(q, a)} />;
    case "true_false":
      return <TrueFalseQuestion {...sub} onSubmit={(a) => handleSubmit(q, a)} />;
    case "matching":
      return <MatchingBoard key={q.id} {...sub} onSubmit={(a) => handleSubmit(q, a)} />;
    case "image_recognition":
      return <ImageQuestion {...sub} onSubmit={(a) => handleSubmit(q, a)} />;
    case "write_in":
      return <WriteInQuestion {...sub} onSubmit={(a) => handleSubmit(q, a)} />;
    default:
      return null;
  }
};
```

```
if (!user) return null;
```

```
return (
  <div className="min-h-[100dvh] p-4 md:p-6 relative">
```

```

{/* — Feedback overlay — */}

<AnimatePresence>
  {feedback && (
    <FeedbackOverlay
      feedback={feedback}
      gameId={gameId}
      userId={userId}
      isLastQuestion={isLastQuestion}
      onNext={nextQuestion}
      onViewResults={() => setLocation(`/results/${gameId}`)}
    />
  )}
</AnimatePresence>

```

```

<div className="mx-auto max-w-6xl space-y-5">

```

```

{/* Header */}

<header className="flex items-center gap-3">
  <Button variant="ghost" size="sm" onClick={() => setLocation("/lobby")}
    className="text-muted-foreground hover:text-foreground shrink-0">
    <ArrowLeft className="mr-1 h-4 w-4" /> Lobby
  </Button>

```

```

<div className="flex-1 min-w-0 text-center">
  <h1 className="text-lg md:text-xl font-bold tracking-tight truncate">
    {game?.topic ?? "Loading..."}

```

```

</h1>

{game && (
  <div className="flex items-center justify-center gap-2 mt-0.5">
    <Badge variant="outline" className="uppercase text-
[10px]">{game.difficulty}</Badge>

    {total > 0 && (
      <span className="text-xs text-muted-foreground">
        Q{Math.min(answeredCount + 1, total)} of {total}
      </span>
    )}
  </div>
)}

</div>

<div className="text-right shrink-0">
  <div className="text-2xl font-bold tabular-nums text-accent flex items-center gap-1
justify-end">
    <Star className="h-4 w-4" />{myScore}
  </div>
  <div className="text-[10px] uppercase tracking-widest text-muted-foreground">
    {myRank > 0 ? `Rank #${myRank}` : "Points"}
  </div>
</div>
</header>

{/* Progress bar */}

{total > 0 && (

```

```

<div className="space-y-1">
  <Progress value={{answeredCount / total} * 100} className="h-2.5 rounded-full" />
  <div className="flex justify-between text-[11px] text-muted-foreground">
    <span>{answeredCount} answered</span>
    <span>{total - answeredCount} remaining</span>
  </div>
</div>
)}

```

```

{/* Question + Leaderboard */}

```

```

<div className="grid gap-5 lg:grid-cols-[1fr_300px]">
  <div>
    <AnimatePresence mode="wait">
      {total === 0 ? (
        <motion.div key="empty" initial={{ opacity: 0 }} animate={{ opacity: 1 }}>
          <Card className="border-dashed border-primary/30 bg-card/40">
            <CardContent className="py-16 text-center space-y-3">
              <Sparkles className="mx-auto h-12 w-12 text-primary/50" />
              <h3 className="text-2xl font-bold">Questions loading soon</h3>
              <p className="text-muted-foreground max-w-md mx-auto">
                The host hasn't added questions yet — this page checks automatically.
              </p>
            </CardContent>
          </Card>
        </motion.div>
      ) : current ? (

```

```

<motion.div
  key={current.id}
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  exit={{ opacity: 0, y: -20 }}
  transition={{ type: "spring", stiffness: 280, damping: 26 }}
>

  <Card className="border-card-border bg-card/60 backdrop-blur">
    <CardHeader className="pb-2">
      <div className="flex items-center justify-between gap-2">
        <Badge variant="outline" className="uppercase tracking-widest text-[10px]">
          {current.questionType.replace(/_/g, " ")}
        </Badge>
        <span className="text-sm font-bold text-accent flex items-center gap-1">
          <Star className="h-3.5 w-3.5" /> {current.points} pts
        </span>
      </div>
      <CardTitle className="text-xl md:text-2xl leading-snug pt-2">
        {current.questionText}
      </CardTitle>
    </CardHeader>
    <CardContent className="pt-2">{renderQuestion(current)}</CardContent>
  </Card>
</motion.div>

):(
  /* All done */

```

```

<motion.div
  key="done"
  initial={{ opacity: 0, scale: 0.92 }}
  animate={{ opacity: 1, scale: 1 }}
  transition={{ type: "spring", stiffness: 280, damping: 24 }}
>

<Card className="border-2 border-accent/50 bg-accent/5">
  <CardContent className="py-16 text-center space-y-4">
    <motion.div
      initial={{ scale: 0.5, rotate: -10 }}
      animate={{ scale: 1, rotate: 0 }}
      transition={{ type: "spring", stiffness: 350, damping: 18 }}
    >
      <Trophy className="mx-auto h-20 w-20 text-accent" />
    </motion.div>
    <h2 className="text-4xl font-bold tracking-tight">THAT'S A WRAP!</h2>
    <p className="text-xl">
      You finished with{" "}
      <span className="font-bold text-accent">{myScore}</span> points</span>
      {myRank > 0 && (
        <span className="text-muted-foreground"> · Rank #{myRank}</span>
      )}
    </p>
    <p className="text-muted-foreground max-w-sm mx-auto">
      Watch the leaderboard — other players are still answering.
    </p>

```

```

        <div className="flex gap-3 justify-center flex-wrap">
            <Button className="font-bold h-12 px-8" onClick={() =>
setLocation(`/results/${gameId}` )}>
                View Results
            </Button>

            <Button variant="outline" onClick={() => setLocation("/lobby")}
className="font-bold h-12 px-8">
                Back to Lobby
            </Button>

        </div>

    </CardContent>

</Card>

</motion.div>

)}

</AnimatePresence>

</div>

```

```

{/* Live leaderboard sidebar */}

<aside>

    <Card className="border-card-border bg-card/60 backdrop-blur lg:sticky lg:top-4">

        <CardHeader className="pb-3">

            <CardTitle className="text-sm uppercase tracking-widest text-muted-foreground
flex items-center gap-2">

                <Trophy className="h-4 w-4 text-accent" /> Live Leaderboard

            </CardTitle>

        </CardHeader>

        <CardContent className="space-y-1">

```

```

{sortedParticipants.length === 0 ? (
  <p className="text-sm text-muted-foreground py-4 text-center">No players
yet.</p>
) : (
  sortedParticipants.map((p, i) => {
    const isMe = p.userId === userId;
    return (
      <motion.div
        key={p.id}
        layout
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        className={`flex items-center gap-3 rounded-lg px-3 py-2.5 ${isMe ? "bg-
primary/10 border border-primary/30" : "bg-background/40"}` }
      >
        <span className="w-6 text-center font-bold tabular-nums text-muted-
foreground shrink-0">
          {i === 0 ? <Crown className="h-4 w-4 text-accent mx-auto" /> : <span
className="text-xs">{i + 1}</span>}
        </span>
        <span className="flex-1 truncate text-sm font-medium">
          {p.userName}
          {isMe && <span className="text-primary text-[10px] ml-1">(you)</span>}
        </span>
        <span className="font-bold tabular-nums text-accent text-sm shrink-
0">{p.totalScore}</span>
      </motion.div>
    )
  })
)

```



```
        );  
    })  
    }  
    </CardContent>  
  </Card>  
  </aside>  
  </div>  
</div>  
<Footer />  
</div>  
);  
}
```

```

---

```
`artifacts/trivia-game/src/pages/Results.tsx`
```

```
```typescript
```

```
import { useMemo, useState } from "react";  
import { useParams, useLocation } from "wouter";  
import { Footer } from "@components/Footer";  
import { motion, AnimatePresence } from "framer-motion";  
import { useQuery } from "@tanstack/react-query";  
import {
```

```
useListGameQuestions,  
getListGameQuestionsQueryKey,  
useListUserAnswers,  
getListUserAnswersQueryKey,  
} from "@workspace/api-client-react";  
  
import { useAuth } from "../lib/auth";  
  
import { Button } from "@components/ui/button";  
  
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";  
  
import { Badge } from "@components/ui/badge";  
  
import { Progress } from "@components/ui/progress";  
  
import { useToast } from "@hooks/use-toast";  
  
import {  
  Trophy,  
  Crown,  
  Medal,  
  Star,  
  Target,  
  Zap,  
  ChevronDown,  
  ChevronUp,  
  CheckCircle2,  
  XCircle,  
  Minus,  
  Share2,  
  ArrowLeft,  
  BarChart3,
```

```
} from "lucide-react";
```

```
// — Types
```

```
type GameResultParticipant = {
```

```
  id: number;
```

```
  userId: number;
```

```
  userName: string;
```

```
  totalScore: number;
```

```
  rank: number;
```

```
  correctCount: number;
```

```
  totalAnswered: number;
```

```
  joinedAt: string;
```

```
};
```

```
type GameResults = {
```

```
  game: { id: number; topic: string; difficulty: string; questionCount: number; status: string };
```

```
  participants: GameResultParticipant[];
```

```
  totalQuestions: number;
```

```
};
```

```
type QuestionStat = {
```

```
  id: number;
```

```
  questionText: string;
```

```
  questionType: string;
```

```
points: number;
orderIndex: number;
totalAnswered: number;
correctCount: number;
percentCorrect: number | null;
};
```

```
const QUESTION_TYPE_LABELS: Record<string, string> = {
  multiple_choice: "Multiple Choice",
  true_false: "True / False",
  write_in: "Write-In",
  matching: "Matching",
  image_recognition: "Image",
};
```

```
const DIFFICULTY_COLORS: Record<string, string> = {
  easy: "bg-secondary/15 text-secondary border-secondary/40",
  medium: "bg-accent/15 text-accent border-accent/40",
  hard: "bg-primary/15 text-primary border-primary/40",
};
```

```
function RankIcon({ rank }: { rank: number }) {
  if (rank === 1) return <Crown className="h-5 w-5 text-yellow-400" />;
  if (rank === 2) return <Medal className="h-5 w-5 text-slate-300" />;
  if (rank === 3) return <Medal className="h-5 w-5 text-amber-600" />;
  return <span className="text-sm font-bold text-muted-foreground">{rank}</span>;
}
```

```
}
```

```
// — Results page —————
```

```
export default function Results() {  
  const params = useParams<{ gameId: string }>();  
  const gameId = Number(params.gameId);  
  const { user } = useAuth();  
  const userId = user?.id ?? 0;  
  const [, setLocation] = useLocation();  
  const { toast } = useToast();  
  const [expandedQuestions, setExpandedQuestions] = useState(false);  
  const [expandedQ, setExpandedQ] = useState<Set<number>>(new Set());  
  
  const { data: results, isLoading: resultsLoading } = useQuery<GameResults>({  
    queryKey: ["game-results", gameId],  
    queryFn: async () => {  
      const r = await fetch(`/api/games/${gameId}/results`);  
      if (!r.ok) throw new Error(`Failed to load results: ${r.status}`);  
      return r.json();  
    },  
    enabled: !!gameId,  
    refetchInterval: 10000,  
  });  
  
  const { data: questionStats = [] } = useQuery<QuestionStat[]>({
```

```
queryKey: ["question-stats", gameId],
queryFn: async () => {
  const r = await fetch(`/api/games/${gameId}/questions/stats`);
  if (!r.ok) throw new Error(`Failed to load question stats: ${r.status}`);
  return r.json();
},
enabled: !!gameId,
});
```

```
const { data: questions = [] } = useListGameQuestions(gameId, {
  query: { enabled: !!gameId, queryKey: getListGameQuestionsQueryKey(gameId) },
});
```

```
const { data: myAnswers = [] } = useListUserAnswers(gameId, userId, {
  query: { enabled: !!gameId && !!userId, queryKey: getListUserAnswersQueryKey(gameId,
userId) },
});
```

// — Derived stats —

```
const me = results?.participants.find((p) => p.userId === userId);
const myRank = me?.rank ?? 0;
const myScore = me?.totalScore ?? 0;
const myCorrect = me?.correctCount ?? 0;
const totalQ = results?.totalQuestions ?? 0;
const accuracy = totalQ > 0 ? Math.round((myCorrect / totalQ) * 100) : 0;
```

```
const sortedQuestions = useMemo(  
  () => [...questions].sort((a, b) => a.orderIndex - b.orderIndex),  
  [questions],  
);
```

```
const answerMap = useMemo(  
  () => new Map(myAnswers.map((a) => [a.questionId, a])),  
  [myAnswers],  
);
```

```
const streak = useMemo(() => {  
  let max = 0, cur = 0;  
  for (const q of sortedQuestions) {  
    const ans = answerMap.get(q.id);  
    if (ans?.isCorrect) { cur++; max = Math.max(max, cur); }  
    else cur = 0;  
  }  
  return max;  
}, [sortedQuestions, answerMap]);
```

```
const bestType = useMemo(() => {  
  const map: Record<string, { correct: number; total: number }> = {};  
  for (const q of sortedQuestions) {  
    const ans = answerMap.get(q.id);  
    if (!ans) continue;  
    const t = q.questionType;
```

```

    map[t] ??= { correct: 0, total: 0 };
    map[t]!.total++;
    if (ans.isCorrect) map[t]!.correct++;
  }
  let best: string | null = null, bestAcc = -1;
  for (const [type, s] of Object.entries(map)) {
    if (s.total === 0) continue;
    const acc = s.correct / s.total;
    if (acc > bestAcc) { best = type; bestAcc = acc; }
  }
  return best ? { type: best, accuracy: Math.round(bestAcc * 100) } : null;
}, [sortedQuestions, answerMap]);

```

```

const statsMap = useMemo(
  () => new Map(questionStats.map((s) => [s.id, s])),
  [questionStats],
);

```

```

const handleShare = async () => {
  const text = `I scored ${myScore} points${totalQ > 0 ? ` (${myCorrect}/${totalQ} correct)` : ""} on "${results?.game.topic ?? "Trivia Night"}"! 🎉`;
  try {
    await navigator.clipboard.writeText(text);
    toast({ title: "Copied to clipboard!", description: text });
  } catch {
    toast({ variant: "destructive", title: "Could not copy to clipboard" });
  }
}

```



```
}  
};
```

```
const toggleQ = (id: number) => {  
  setExpandedQ((prev) => {  
    const next = new Set(prev);  
    next.has(id) ? next.delete(id) : next.add(id);  
    return next;  
  });  
};
```

```
if (resultsLoading || !results) {  
  return (  
    <div className="min-h-[100dvh] flex items-center justify-center">  
      <div className="text-center space-y-3">  
        <Trophy className="mx-auto h-12 w-12 text-primary/40 animate-pulse" />  
        <p className="text-muted-foreground">Loading results...</p>  
      </div>  
    </div>  
  );  
}
```

```
const { game, participants } = results;
```

```
return (  
  <div className="min-h-[100dvh] p-4 md:p-8">
```

```
<div className="mx-auto max-w-4xl space-y-8">
```

```
  { /* — Header — */ }
```

```
  <motion.div
```

```
    initial={{ opacity: 0, y: -16 }}
```

```
    animate={{ opacity: 1, y: 0 }}
```

```
    className="text-center space-y-3"
```

```
  >
```

```
    <div className="flex justify-start mb-2">
```

```
      <Button variant="ghost" size="sm" onClick={() => setLocation("/lobby")}>
```

```
        className="text-muted-foreground hover:text-foreground">
```

```
        <ArrowLeft className="mr-1 h-4 w-4" /> Back to Lobby
```

```
      </Button>
```

```
    </div>
```

```
    <motion.div
```

```
      initial={{ scale: 0.5, rotate: -15 }}
```

```
      animate={{ scale: 1, rotate: 0 }}
```

```
      transition={{ type: "spring", stiffness: 300, damping: 18, delay: 0.1 }}
```

```
    >
```

```
      <Trophy className="mx-auto h-20 w-20 text-accent" />
```

```
    </motion.div>
```

```
  </div>
```

```
  <h1 className="text-4xl md:text-5xl font-black tracking-tight text-primary">
```

```
    GAME OVER
```

```
  </h1>
```

```
  <p className="text-xl font-semibold text-muted-foreground mt-1">{game.topic}</p>
```

```

</div>

<div className="flex items-center justify-center gap-2">
  <Badge variant="outline" className={` uppercase
${DIFFICULTY_COLORS[game.difficulty] ?? ""}`}>
    {game.difficulty}
  </Badge>
  <span className="text-muted-foreground text-sm">
    {totalQ} {totalQ === 1 ? "question" : "questions"}
  </span>
</div>

</motion.div>

{/* — My stats — */}
{me && (
  <motion.div initial={{ opacity: 0, y: 12 }} animate={{ opacity: 1, y: 0 }} transition={{ delay:
0.15 }}>
    <Card className="border-2 border-accent/40 bg-accent/5">
      <CardContent className="p-5">
        <div className="flex items-center gap-3 mb-4">
          <div className="flex items-center justify-center h-12 w-12 rounded-full border-2
border-accent/50 bg-accent/10">
            <RankIcon rank={myRank} />
          </div>
          <div>
            <p className="text-lg font-bold">{user?.name}</p>
            <p className="text-sm text-muted-foreground">
              Rank #{myRank} of {participants.length}
            </p>
          </div>
        </div>
      </CardContent>
    </Card>
  </motion.div>
)
}

```

```

    </p>

  </div>

  <div className="ml-auto text-right">

    <p className="text-4xl font-black tabular-nums text-accent">{myScore}</p>

    <p className="text-xs uppercase tracking-widest text-muted-foreground">points</p>

  </div>

</div>

<div className="grid grid-cols-2 sm:grid-cols-4 gap-3">

  {[

    { icon: Target, label: "Correct", value: ` ${myCorrect} / ${totalQ} `, color: "text-secondary" },

    { icon: BarChart3, label: "Accuracy", value: ` ${accuracy}% `, color: "text-primary" },

    { icon: Zap, label: "Best Streak", value: ` ${streak} `, color: "text-accent" },

    {

      icon: Star,

      label: "Best Type",

      value: bestType ? ` ${bestType.accuracy}% ` : "—",

      sub: bestType ? QUESTION_TYPE_LABELS[bestType.type] ?? bestType.type : undefined,

      color: "text-secondary",

    },

  ]}.map((s) => (

    <div key={s.label} className="rounded-lg border border-card-border bg-background/40 p-3 text-center">

      <s.icon className={` h-5 w-5 mx-auto mb-1 ${s.color} ` />

```

```

    <p className="text-lg font-bold tabular-nums">{s.value}</p>

    {s.sub && <p className="text-[10px] text-muted-foreground">{s.sub}</p>}

    <p className="text-[11px] uppercase tracking-widest text-muted-
foreground">{s.label}</p>

    </div>

  )}}

</div>

</CardContent>

</Card>

</motion.div>

)}

{ /* — Final leaderboard — */

<motion.div initial={{ opacity: 0, y: 12 }} animate={{ opacity: 1, y: 0 }} transition={{ delay:
0.25 }}>

  <Card className="border-card-border bg-card/60 backdrop-blur">

    <CardHeader className="pb-3">

      <CardTitle className="flex items-center gap-2 text-sm uppercase tracking-widest
text-muted-foreground">

        <Trophy className="h-4 w-4 text-accent" /> Final Standings

      </CardTitle>

    </CardHeader>

    <CardContent className="p-0">

      {participants.map((p, i) => {

        const isMe = p.userId === userId;

        const pAcc = totalQ > 0 ? Math.round((p.correctCount / totalQ) * 100) : 0;

        return (

```

```

<motion.div
  key={p.userId}
  initial={{ opacity: 0, x: -12 }}
  animate={{ opacity: 1, x: 0 }}
  transition={{ delay: 0.3 + i * 0.05 }}
  className={`flex items-center gap-4 px-5 py-3.5 border-b border-border/50
last:border-0 ${isMe ? "bg-accent/5" : ""}`} ` }
>

  { /* Rank */ }

  <div className="w-8 flex justify-center shrink-0">
    <RankIcon rank={p.rank} />
  </div>

  { /* Name + accuracy */ }

  <div className="flex-1 min-w-0">
    <p className={`font-semibold truncate ${isMe ? "text-accent" : ""}`} `>
      {p.userName}
      {isMe && <span className="text-xs text-muted-foreground ml-
1">(you)</span> }
    </p>

    <div className="flex items-center gap-2 mt-0.5">
      <Progress value={pAcc} className="h-1.5 w-20" />
      <span className="text-[11px] text-muted-foreground tabular-nums">
        {p.correctCount}/{totalQ} · {pAcc}%
      </span>
    </div>
  </div>
</div>

```

```

    { /* Score */
    <div className="text-right shrink-0">
      <span className={` text-xl font-black tabular-nums ${isMe ? "text-accent" :
""}` `}>
        {p.totalScore}
      </span>
      <p className="text-[10px] uppercase tracking-widest text-muted-foreground">pts</p>
    </div>
  </motion.div>
);
}}
</CardContent>
</Card>
</motion.div>

{ /* — Question breakdown — */
{sortedQuestions.length > 0 && (
  <motion.div initial={{ opacity: 0, y: 12 }} animate={{ opacity: 1, y: 0 }} transition={{ delay:
0.4 }}>
    <Card className="border-card-border bg-card/60 backdrop-blur">
      <button
        className="w-full flex items-center justify-between p-5 text-left"
        onClick={() => setExpandedQuestions((v) => !v)}
      >
        <span className="font-semibold flex items-center gap-2">

```

```
<BarChart3 className="h-4 w-4 text-muted-foreground" />
```

```
Question-by-Question Breakdown
```

```
</span>
```

```
{expandedQuestions
```

```
  ? <ChevronUp className="h-4 w-4 text-muted-foreground" />
```

```
  : <ChevronDown className="h-4 w-4 text-muted-foreground" />}
```

```
</button>
```

```
<AnimatePresence>
```

```
{expandedQuestions && (
```

```
  <motion.div
```

```
    initial={{ height: 0, opacity: 0 }}
```

```
    animate={{ height: "auto", opacity: 1 }}
```

```
    exit={{ height: 0, opacity: 0 }}
```

```
    transition={{ duration: 0.2 }}
```

```
    className="overflow-hidden"
```

```
>
```

```
<div className="border-t border-border/50 divide-y divide-border/50">
```

```
  {sortedQuestions.map((q, i) => {
```

```
    const myAns = answerMap.get(q.id);
```

```
    const stat = statsMap.get(q.id);
```

```
    const isOpen = expandedQ.has(q.id);
```

```
    const status = !myAns
```

```
      ? "unanswered"
```

```
      : myAns.isCorrect
```

```
      ? "correct"
```



```
: myAns.pointsEarned > 0
```

```
? "partial"
```

```
: "wrong";
```

```
return (
```

```
<div key={q.id}>
```

```
<button
```

```
    className="w-full flex items-start gap-3 px-5 py-3.5 text-left hover:bg-  
muted/30 transition-colors"
```

```
    onClick={() => toggleQ(q.id)}
```

```
>
```

```
<span className="text-xs font-bold text-muted-foreground mt-0.5 w-6  
shrink-0">
```

```
    Q{i + 1}
```

```
</span>
```

```
<div className="flex-1 min-w-0">
```

```
<p className="text-sm font-medium leading-snug line-clamp-2">
```

```
    {q.questionText}
```

```
</p>
```

```
<p className="text-[11px] text-muted-foreground mt-0.5">
```

```
    {QUESTION_TYPE_LABELS[q.questionType] ?? q.questionType} · {q.points}
```

```
pts
```

```
    {stat?.percentCorrect !== null && (
```

```
      <> · {stat.percentCorrect}% got it right</>
```

```
    )}
```

```
</p>
```

```
</div>
```

```

<div className="flex items-center gap-2 shrink-0">
  {status === "correct" && (
    <CheckCircle2 className="h-5 w-5 text-emerald-500" />
  )}
  {status === "partial" && (
    <Minus className="h-5 w-5 text-amber-500" />
  )}
  {status === "wrong" && (
    <XCircle className="h-5 w-5 text-red-500" />
  )}
  {status === "unanswered" && (
    <span className="text-xs text-muted-foreground">—</span>
  )}
  {isOpen ? (
    <ChevronUp className="h-4 w-4 text-muted-foreground" />
  ) : (
    <ChevronDown className="h-4 w-4 text-muted-foreground" />
  )}
</div>
</button>

```

```

<AnimatePresence>
  {isOpen && (
    <motion.div
      initial={{ height: 0, opacity: 0 }}
      animate={{ height: "auto", opacity: 1 }}
    >

```

```

exit={{ height: 0, opacity: 0 }}
transition={{ duration: 0.15 }}
className="overflow-hidden"
>
<div className="px-14 pb-4 space-y-2">
  {myAns ? (
    <>
      <div className="grid grid-cols-2 gap-3">
        <div className={` rounded-lg border p-3 text-sm ${
          status === "correct"
            ? "border-emerald-500/40 bg-emerald-500/5"
            : status === "partial"
            ? "border-amber-500/40 bg-amber-500/5"
            : "border-red-500/40 bg-red-500/5"
          }`} >
          <p className="text-[10px] font-semibold uppercase tracking-widest
text-muted-foreground mb-1">Your answer</p>
          <p className="font-medium leading-
snug">{myAns.userAnswer}</p>
        </div>
        {myAns.correctAnswer !== undefined && (
          <div className="rounded-lg border border-secondary/30 bg-
secondary/5 p-3 text-sm">
            <p className="text-[10px] font-semibold uppercase tracking-widest
text-muted-foreground mb-1">Correct answer</p>
            <p className="font-medium leading-
snug">{myAns.correctAnswer}</p>

```

```

    </div>

  )}

</div>

<div className="flex items-center justify-between text-sm">

  <span className={` font-bold ${
    status === "correct"
      ? "text-emerald-500"
      : status === "partial"
        ? "text-amber-500"
        : "text-muted-foreground"
    }`}>

    {myAns.pointsEarned > 0 ? `+${myAns.pointsEarned}` : "0"} /
{q.points} pts

  </span>

  {stat && stat.totalAnswered > 0 && (
    <div className="flex items-center gap-2 text-muted-foreground">
      <span>{stat.correctCount}/{stat.totalAnswered} players
correct</span>

      <Progress value={stat.percentCorrect ?? 0} className="w-16 h-1.5"
/>

    </div>

  )}

</div>

</>

):(

  <p className="text-sm text-muted-foreground italic">You didn't answer
this question.</p>

```

```

        }}
      </div>
    </motion.div>

  }}
</AnimatePresence>
</div>

);
}}
</div>
</motion.div>

)}
</AnimatePresence>
</Card>
</motion.div>
)}

```

```

{/* — Actions — */}
<motion.div
  initial={{ opacity: 0, y: 8 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ delay: 0.5 }}
  className="flex flex-col sm:flex-row gap-3"
>
  <Button
    size="lg"
    className="flex-1 h-12 font-bold"

```

```

        onClick={() => setLocation("/lobby")}
      >
        Play Again
      </Button>
      <Button
        variant="outline"
        size="lg"
        className="flex-1 h-12 font-bold"
        onClick={handleShare}
      >
        <Share2 className="mr-2 h-4 w-4" />
        Share Result
      </Button>
    </motion.div>
  </div>
  <Footer />
</div>
);
}

...

---

## `artifacts/trivia-game/src/pages/Admin.tsx`

```

```typescript

```
import { useMemo, useState, useEffect } from "react";

import { useLocation } from "wouter";

import { motion, AnimatePresence } from "framer-motion";

import { useQuery, useQueryClient } from "@tanstack/react-query";

import { cn } from "@lib/utls";

import {
 DndContext,
 closestCenter,
 PointerSensor,
 KeyboardSensor,
 useSensor,
 useSensors,
 type DragEndEvent,
} from "@dnd-kit/core";

import {
 SortableContext,
 sortableKeyboardCoordinates,
 verticalListSortingStrategy,
 useSortable,
 arrayMove,
} from "@dnd-kit/sortable";

import { CSS } from "@dnd-kit/utilities";

import {
 useListGames,
 getListGamesQueryKey,
```

```
useCreateGame,
useUpdateGame,
useDeleteGame,
useListGameQuestions,
getListGameQuestionsQueryKey,
useCreateQuestion,
useUpdateQuestion,
useDeleteQuestion,
useGetStatsSummary,
getGetStatsSummaryQueryKey,
useListGameParticipants,
getListGameParticipantsQueryKey,
useImportOpenTdbQuestions,
useGenerateGeminiQuestions,
useRegenerateQuestion,
useEnhanceQuestion,
useFactCheckQuestion,
} from "@workspace/api-client-react";
import type {
 Game,
 Question,
 RegenerateQuestionPreview,
 EnhanceQuestionResult,
 FactCheckSingleResult,
} from "@workspace/api-client-react";
import { useAuth } from "../lib/auth";
```



```
import { Button } from "@components/ui/button";

import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";

import { Input } from "@components/ui/input";

import { Textarea } from "@components/ui/textarea";

import { Badge } from "@components/ui/badge";

import { Label } from "@components/ui/label";

import { Separator } from "@components/ui/separator";

import {
 Select,
 SelectContent,
 SelectItem,
 SelectTrigger,
 SelectValue,
} from "@components/ui/select";

import { Switch } from "@components/ui/switch";

import {
 Dialog,
 DialogContent,
 DialogHeader,
 DialogTitle,
 DialogTrigger,
} from "@components/ui/dialog";

import { useToast } from "@hooks/use-toast";

import {
 Shield,
 Plus,
```

Trash2,  
Pencil,  
Play,  
Flag,  
ListChecks,  
X,  
Gamepad2,  
Radio,  
Users,  
Zap,  
LayoutDashboard,  
PlusCircle,  
Settings,  
HelpCircle,  
LogOut,  
Eye,  
EyeOff,  
Crown,  
AlertTriangle,  
ChevronRight,  
Trophy,  
CheckCircle2,  
GripVertical,  
CheckSquare,  
ToggleLeft,  
PenLine,

ImageIcon,  
ArrowLeftRight,  
ShieldAlert,  
Star,  
BarChart3,  
Download,  
ChevronDown,  
ChevronUp,  
BookOpen,  
Lightbulb,  
ExternalLink,  
FlaskConical,  
Loader2,  
Database,  
RefreshCw,  
ShieldCheck,  
Link,  
Square,  
SlidersHorizontal,  
Wand2,  
Sparkles,  
} from "lucide-react";

// — Types —————

```
type Section = "dashboard" | "create" | "manage" | "questions" | "review" | "settings" |
"results" | "help";
```

```
type QuestionType = Question["questionType"];
```

```
type QuestionFormState = {
 questionText: string;
 questionType: QuestionType;
 correctAnswer: string;
 choices: string[];
 pairs: { left: string; right: string }[];
 imageUrl: string;
 points: string;
 alternateAnswers: string;
 source: string;
 factCheckUrl: string;
};
```

```
// — Question form helpers —————
```

```
const DEFAULT_POINTS: Record<QuestionType, number> = {
 multiple_choice: 10,
 true_false: 5,
 write_in: 15,
 matching: 20,
 image_recognition: 15,
};
```

```

const emptyForm: QuestionFormState = {
 questionText: "",
 questionType: "multiple_choice",
 correctAnswer: "",
 choices: ["", "", "", ""],
 pairs: [
 { left: "", right: "" },
 { left: "", right: "" },
 { left: "", right: "" },
 { left: "", right: "" },
],
 imageUrl: "",
 points: "10",
 alternateAnswers: "",
 source: "",
 factCheckUrl: "",
};

```

```

function emptyFormForType(type: QuestionType): QuestionFormState {
 return { ...emptyForm, questionType: type, points: String(DEFAULT_POINTS[type]) };
}

```

```

function formFromQuestion(q: Question): QuestionFormState {
 const opts = q.options as
 | {

```

```

 choices?: string[];

 pairs?: { left: string; right: string }[];

 alternateAnswers?: string[];

}

| null;

return {

 questionText: q.questionText,

 questionType: q.questionType,

 correctAnswer: q.questionType === "matching" ? "" : (q.correctAnswer ?? ""),

 choices: opts?.choices?.length

 ? [...opts.choices]

 : ["", "", "", ""],

 pairs: opts?.pairs?.length

 ? opts.pairs.map((p) => ({ ...p }))

 : [

 { left: "", right: "" },

 { left: "", right: "" },

 { left: "", right: "" },

 { left: "", right: "" },

],

 imageUrl: q.imageUrl ?? "",

 points: String(q.points),

 alternateAnswers: opts?.alternateAnswers?.join(", ") ?? "",

 source: q.source ?? "",

 factCheckUrl: q.factCheckUrl ?? "",

};

```

```
}
```

```
function buildPayload(form: QuestionFormState, orderIndex: number) {
 const points = Math.max(1, Number(form.points) ||
 DEFAULT_POINTS[form.questionType]);
 const base = {
 questionText: form.questionText.trim(),
 questionType: form.questionType,
 points,
 orderIndex,
 imageUrl:
 form.questionType === "image_recognition" && form.imageUrl.trim()
 ? form.imageUrl.trim()
 : null,
 source: form.source.trim() || null,
 factCheckUrl: form.factCheckUrl.trim() || null,
 };

 if (form.questionType === "multiple_choice") {
 const choices = form.choices.map((c) => c.trim()).filter(Boolean);
 return {
 ...base,
 options: { choices },
 correctAnswer: form.correctAnswer.trim(),
 };
 }
}
```

```

if (form.questionType === "matching") {
 const pairs = form.pairs
 .map((p) => ({ left: p.left.trim(), right: p.right.trim() }))
 .filter((p) => p.left && p.right);
 const correctAnswer = pairs
 .map((p) => `${p.left}:${p.right}`)
 .sort((a, b) => a.localeCompare(b))
 .join("|");
 return { ...base, options: { pairs }, correctAnswer };
}

if (form.questionType === "true_false") {
 return {
 ...base,
 options: null,
 correctAnswer: form.correctAnswer === "false" ? "false" : "true",
 };
}

// write_in / image_recognition
const alternates = form.alternateAnswers
 .split(",")
 .map((s) => s.trim())
 .filter(Boolean);
return {
 ...base,
 options: alternates.length ? { alternateAnswers: alternates } : null,
 correctAnswer: form.correctAnswer.trim(),
}

```



```
};
}
```

```
function payloadFromExisting(q: Question, orderIndex: number) {
 const opts = q.options as {
 choices?: string[];
 pairs?: { left: string; right: string }[];
 alternateAnswers?: string[];
 } | null;
 return {
 questionText: q.questionText,
 questionType: q.questionType,
 points: q.points,
 orderIndex,
 imageUrl: q.imageUrl ?? null,
 options: opts ?? null,
 correctAnswer: q.correctAnswer,
 };
}
```

```
function validateForm(form: QuestionFormState): string | null {
 if (!form.questionText.trim()) return "Question text is required";
 if (form.questionType === "multiple_choice") {
 const choices = form.choices.map((c) => c.trim()).filter(Boolean);
 if (choices.length < 2) return "Add at least two choices";
 if (!form.correctAnswer.trim()) return "Pick the correct choice";
 }
}
```

```

 if (!choices.includes(form.correctAnswer.trim()))
 return "Correct answer must be one of the choices";
} else if (form.questionType === "matching") {
 const pairs = form.pairs.filter((p) => p.left.trim() && p.right.trim());
 if (pairs.length < 2) return "Add at least two complete pairs";
} else if (form.questionType === "true_false") {
 if (form.correctAnswer !== "true" && form.correctAnswer !== "false")
 return "Pick true or false";
} else {
 if (!form.correctAnswer.trim()) return "Correct answer is required";
 if (form.questionType === "image_recognition" && !form.imageUrl.trim())
 return "Image URL is required";
}
return null;
}

```

// — Question type meta —————

```

const TYPE_META: Record<
 QuestionType,
 { label: string; Icon: typeof CheckSquare; color: string }
> = {
 multiple_choice: { label: "Multiple Choice", Icon: CheckSquare, color: "text-primary" },
 true_false: { label: "True / False", Icon: ToggleLeft, color: "text-secondary" },
 write_in: { label: "Write-In", Icon: PenLine, color: "text-accent" },
 image_recognition: { label: "Image", Icon: ImageIcon, color: "text-orange-400" },

```

```
 matching: { label: "Matching", Icon: ArrowLeftRight, color: "text-purple-400" },
};
```

```
const CHOICE_LABELS = ["A", "B", "C", "D", "E", "F"];
```

```
// — OpenTDB categories
```

---

```
const OPENTDB_CATEGORIES = [
 { id: 9, name: "General Knowledge" },
 { id: 10, name: "Books" },
 { id: 11, name: "Film" },
 { id: 12, name: "Music" },
 { id: 14, name: "Television" },
 { id: 15, name: "Video Games" },
 { id: 17, name: "Science & Nature" },
 { id: 21, name: "Sports" },
 { id: 22, name: "Geography" },
 { id: 23, name: "History" },
 { id: 25, name: "Art" },
 { id: 26, name: "Celebrities" },
 { id: 27, name: "Animals" },
 { id: 28, name: "Vehicles" },
] as const;
```

```
// — QuestionForm
```

---

```

function QuestionForm({
 initial,
 onSubmit,
 pending,
 submitLabel,
}: {
 initial: QuestionFormState;
 onSubmit: (form: QuestionFormState) => void;
 pending: boolean;
 submitLabel: string;
}) {
 const [form, setForm] = useState<QuestionFormState>(initial);
 const { toast } = useToast();
 const set = <K extends keyof QuestionFormState>(k: K, v: QuestionFormState[K]) =>
 setForm((f) => ({ ...f, [k]: v }));

 const handleTypeChange = (v: QuestionType) => {
 setForm((f) => ({
 ...f,
 questionType: v,
 correctAnswer: "",
 points: String(DEFAULT_POINTS[v]),
 }));
 };
};

```

```

const validChoices = form.choices.map((c) => c.trim()).filter(Boolean);

return (
 <div className="space-y-5">
 {/* Type selector */}
 <div className="space-y-2">
 <Label>Question Type</Label>
 <div className="grid grid-cols-2 sm:grid-cols-3 gap-2">
 {(Object.keys(TYPE_META) as QuestionType[]).map((t) => {
 const { label, Icon, color } = TYPE_META[t];
 const active = form.questionType === t;
 return (
 <button
 key={t}
 type="button"
 onClick={() => handleTypeChange(t)}
 className={`flex items-center gap-2 rounded-lg border px-3 py-2.5 text-sm font-medium transition-colors text-left ${
 active
 ? "border-primary bg-primary/10 text-foreground"
 : "border-border bg-background/40 text-muted-foreground hover:border-muted-foreground/40 hover:text-foreground"
 }`}
 >
 <Icon className={`h-4 w-4 shrink-0 ${active ? color : ""}`} />
 {label}
 </button>
)
 })}
 </div>
 </div>
)

```

```
);
}}}
</div>
</div>
```

```
{/* Question text */}
<div className="space-y-2">
 <Label>
 {form.questionType === "image_recognition" ? "Caption / Prompt" : "Question"}
 </Label>
 <Textarea
 value={form.questionText}
 onChange={(e) => set("questionText", e.target.value)}
 placeholder={
 form.questionType === "image_recognition"
 ? 'e.g. "Name this landmark" or "Which country is this flag from?"'
 : "Type the question players will see..."
 }
 rows={3}
 />
</div>
```

```
{/* — Multiple Choice — */}
{form.questionType === "multiple_choice" && (
 <div className="space-y-3">
 <Label>Answer Choices</Label>
```

```

<div className="space-y-2">
 {form.choices.map((choice, i) => {
 const isCorrect = form.correctAnswer === choice.trim() && choice.trim();
 return (
 <div key={i} className="flex items-center gap-2">
 <button
 type="button"
 onClick={() => choice.trim() && set("correctAnswer", choice.trim())}
 className={`h-8 w-8 shrink-0 rounded-full border-2 text-sm font-bold
transition-colors ${
 isCorrect
 ? "border-secondary bg-secondary text-secondary-foreground"
 : "border-border text-muted-foreground hover:border-muted-foreground/60"
 }`}
 title="Click to mark as correct"
 >
 {CHOICE_LABELS[i]}
 </button>
 <Input
 value={choice}
 onChange={(e) => {
 const next = [...form.choices];
 next[i] = e.target.value;
 // if this was the correct answer, keep it synced
 const updated = { ...form, choices: next };
 if (form.correctAnswer === form.choices[i]?.trim()) {

```

```

 updated.correctAnswer = e.target.value.trim();
 }
 setForm(updated);
 }}
 placeholder={`Choice ${CHOICE_LABELS[i]} `}
 className={isCorrect ? "border-secondary/60 bg-secondary/5" : ""}
/>
{form.choices.length > 2 && (
 <Button
 type="button"
 variant="ghost"
 size="icon"
 className="shrink-0"
 onClick={() => {
 const next = form.choices.filter((_, j) => j !== i);
 setForm((f) => ({
 ...f,
 choices: next,
 correctAnswer:
 f.correctAnswer === choice.trim() ? "" : f.correctAnswer,
 }));
 }}
 >
 <X className="h-4 w-4" />
 </Button>
)}

```



```
</div>
```

```
);
```

```
}}}
```

```
</div>
```

```
{form.choices.length < 6 && (
```

```
<Button
```

```
 type="button"
```

```
 variant="outline"
```

```
 size="sm"
```

```
 onClick={() => set("choices", [...form.choices, ""])}
>
```

```
<Plus className="mr-1 h-3.5 w-3.5" /> Add choice
```

```
</Button>
```

```
)}

{form.correctAnswer ? (
```

```
<p className="text-sm text-secondary font-medium flex items-center gap-1.5">
```

```
<CheckCircle2 className="h-4 w-4" />
```

```
 Correct: "{form.correctAnswer}"
```

```
</p>
```

```
): (
```

```
<p className="text-xs text-muted-foreground">
```

```
 Click a letter button to mark the correct answer.
```

```
</p>
```

```
)}

</div>
```

```

</div>

))

{/* — True / False — */}

{form.questionType === "true_false" && (

 <div className="space-y-2">

 <Label>Correct Answer</Label>

 <div className="grid grid-cols-2 gap-3">

 {[["true", "false"] as const].map((val) => (

 <button

 key={val}

 type="button"

 onClick={() => set("correctAnswer", val)}

 className={` rounded-xl border-2 py-4 text-center font-bold text-lg transition-
colors ${

 form.correctAnswer === val

 ? val === "true"

 ? "border-secondary bg-secondary/15 text-secondary"

 : "border-destructive bg-destructive/10 text-destructive"

 : "border-border text-muted-foreground hover:border-muted-foreground/50"

 }`

 }

 >

 {val === "true" ? "✓ TRUE" : "✗ FALSE"}

 </button>

)]}

 >

 {val === "true" ? "✓ TRUE" : "✗ FALSE"}

 </button>

)}}

</div>

```

```

</div>

)}}

{/* — Write-In — */}

{form.questionType === "write_in" && (
 <div className="space-y-4">
 <div className="space-y-2">
 <Label>Correct Answer</Label>
 <Input
 value={form.correctAnswer}
 onChange={(e) => set("correctAnswer", e.target.value)}
 placeholder="Primary correct answer (case-insensitive)"
 />
 </div>
 <div className="space-y-2">
 <Label>
 Alternate Acceptable Answers

 (optional, comma-separated)

 </Label>
 <Input
 value={form.alternateAnswers}
 onChange={(e) => set("alternateAnswers", e.target.value)}
 placeholder='e.g. "New York, NYC, The Big Apple"'
 />
 </div>
 </div>
)}

```

```

 <p className="text-xs text-muted-foreground">
 Any of these will be accepted as correct.
 </p>
 </div>
</div>
)}

```

```

{ /* — Image Recognition — */
{form.questionType === "image_recognition" && (
 <div className="space-y-4">
 <div className="space-y-2">
 <Label>Image URL</Label>
 <Input
 value={form.imageUrl}
 onChange={(e) => set("imageUrl", e.target.value)}
 placeholder="https://upload.wikimedia.org/..."
 />
 {form.imageUrl.trim() && (
 <div className="rounded-lg overflow-hidden border border-border max-h-48 flex
items-center justify-center bg-muted/30">
 <img
 src={form.imageUrl.trim()}
 alt="Preview"
 className="max-h-48 object-contain"
 onError={(e) => {
 (e.target as HTMLImageElement).style.display = "none";
 }}
 />
 </div>
)}
 </div>
 </div>
)}
}

```

```
 }}
 />
</div>

)}
</div>

<div className="space-y-2">
 <Label>Correct Answer</Label>

 <Input
 value={form.correctAnswer}
 onChange={(e) => set("correctAnswer", e.target.value)}
 placeholder="What the image shows (case-insensitive)"
 />
</div>

<div className="space-y-2">
 <Label>
 Alternate Answers

 (optional, comma-separated)

 </Label>

 <Input
 value={form.alternateAnswers}
 onChange={(e) => set("alternateAnswers", e.target.value)}
 placeholder='e.g. "Eiffel Tower, La Tour Eiffel"'
 />
</div>
```

```

 </div>

)}

 { /* — Matching — */
 {form.questionType === "matching" && (
 <div className="space-y-3">
 <div className="grid grid-cols-2 gap-x-3 gap-y-1 text-xs font-semibold uppercase
tracking-widest text-muted-foreground px-9">
 Left
 Matches with
 </div>
 {form.pairs.map((pair, i) => (
 <div key={i} className="flex items-center gap-2">
 <span className="w-7 text-center text-sm font-bold text-muted-foreground shrink-
0">
 {i + 1}

 <Input
 value={pair.left}
 onChange={(e) => {
 const next = form.pairs.map((p, j) =>
 j === i ? { ...p, left: e.target.value } : p,
);
 set("pairs", next);
 }}
 placeholder="Left item"
 />

```

```

<ArrowLeftRight className="h-4 w-4 text-muted-foreground shrink-0" />

<Input
 value={pair.right}
 onChange={(e) => {
 const next = form.pairs.map((p, j) =>
 j === i ? { ...p, right: e.target.value } : p,
);
 set("pairs", next);
 }}
 placeholder="Right item"
/>

{form.pairs.length > 2 && (
 <Button
 type="button"
 variant="ghost"
 size="icon"
 className="shrink-0"
 onClick={() => set("pairs", form.pairs.filter((_, j) => j !== i))}
 >
 <X className="h-4 w-4" />
 </Button>
)}

</div>

)}}

{form.pairs.length < 6 && (
 <Button

```

```

 type="button"
 variant="outline"
 size="sm"
 className="ml-9"
 onClick={() => set("pairs", [...form.pairs, { left: "", right: "" }])}
 >
 <Plus className="mr-1 h-3.5 w-3.5" /> Add pair
 </Button>
)}
</div>
)}

```

```

{ /* Source / Citation */
<div className="space-y-1.5 pt-1">
 <Label>
 Source / Citation
 (optional)
 </Label>
 <Input
 value={form.source}
 onChange={(e) => set("source", e.target.value)}
 placeholder='e.g. "Wikipedia: Eiffel Tower" or "Britannica"'
 />
</div>

```



```
{/* Fact-check URL */}

<div className="space-y-1.5">

 <Label>

 Fact-check URL

 (optional)

 </Label>

 <Input

 type="url"

 value={form.factCheckUrl}

 onChange={(e) => set("factCheckUrl", e.target.value)}

 placeholder="https://en.wikipedia.org/wiki/..."

 />

</div>
```

```
{/* Points */}

<div className="flex items-center gap-3 pt-1">

 <div className="space-y-1">

 <Label>Points</Label>

 <Input

 type="number"

 min={1}

 max={100}

 value={form.points}

 onChange={(e) => set("points", e.target.value)}

 className="w-28"

 />

 </div>

</div>
```

```

 />

</div>

<p className="text-xs text-muted-foreground mt-5 max-w-[200px]">
 Default for {TYPE_META[form.questionType].label}:
 {DEFAULT_POINTS[form.questionType]} pts
</p>

</div>

<Button
 className="w-full font-bold"
 disabled={pending}
 onClick={() => {
 const err = validateForm(form);
 if (err) {
 toast({ variant: "destructive", title: err });
 return;
 }
 onSubmit(form);
 }}
>
 {pending ? "Saving..." : submitLabel}
</Button>

</div>

);
}

```

// — SortableQuestionItem —

---

```
function SortableQuestionItem({
 q,
 index,
 onEdit,
 onDelete,
 deleteDisabled,
}: {
 q: Question;
 index: number;
 onEdit: (q: Question) => void;
 onDelete: (id: number) => void;
 deleteDisabled: boolean;
}) {
 const { attributes, listeners, setNodeRef, transform, transition, isDragging } =
 useSortable({ id: q.id });

 const style: React.CSSProperties = {
 transform: CSS.Transform.toString(transform),
 transition,
 opacity: isDragging ? 0.4 : 1,
 zIndex: isDragging ? 10 : undefined,
 };
}
```

```

const { Icon, label, color } = TYPE_META[q.questionType as QuestionType] ??
TYPE_META.write_in;

return (
 <div
 ref={setNodeRef}
 style={style}
 className={`flex items-start gap-3 rounded-lg border bg-card/60 p-3 transition-
shadow ${
 isDragging ? "shadow-xl border-primary/40" : "border-card-border"
 }`}
 >
 { /* Drag handle */ }
 <button
 {...attributes}
 {...listeners}
 className="mt-0.5 cursor-grab active:cursor-grabbing text-muted-foreground/40
hover:text-muted-foreground transition-colors touch-none"
 aria-label="Drag to reorder"
 >
 <GripVertical className="h-5 w-5" />
 </button>

 { /* Number */ }
 <span className="mt-0.5 w-5 text-center font-bold text-muted-foreground tabular-
nums text-sm shrink-0">
 {index + 1}

```

</span>

{/\* Type icon \*/}

<div className={`mt-0.5 shrink-0 \${color}`}>

<Icon className="h-4 w-4" />

</div>

{/\* Content \*/}

<div className="flex-1 min-w-0 space-y-1">

<p className="font-medium leading-snug text-sm">{q.questionText}</p>

<div className="flex flex-wrap items-center gap-2">

<Badge variant="outline" className="uppercase text-[10px]">

{label}

</Badge>

{q.source === "opentdb" && (

<Badge variant="outline" className="text-[10px] bg-blue-500/10 text-blue-400 border-blue-500/30">OpenTDB</Badge>

)}

{q.aiGenerated && (

<Badge variant="outline" className="text-[10px] bg-purple-500/10 text-purple-400 border-purple-500/30">AI Generated</Badge>

)}

<span className="text-xs text-accent font-semibold flex items-center gap-0.5">

<Star className="h-3 w-3" /> {q.points} pts

</span>

{q.verifiedByAdmin ? (

<span className="flex items-center gap-0.5 text-xs text-secondary">

```

 <ShieldCheck className="h-3 w-3" /> Verified

): (

 <AlertTriangle className="h-3 w-3" /> Unverified

)}
{q.questionType !== "matching" && (

 ans: {q.correctAnswer}

)}
</div>

{/* Source citation — shown for AI-generated questions */}
{q.aiGenerated && q.source && q.source !== "opentdb" &&
!q.source.startsWith("manual") && (
 <p className="text-[11px] text-muted-foreground/70 flex items-center gap-1">
 <BookOpen className="h-3 w-3 shrink-0" />
 {q.source}
 </p>
)}
</div>

{/* Actions */}
<div className="flex gap-1 shrink-0">
 <Button

```

```

 variant="ghost"
 size="icon"
 onClick={() => onEdit(q)}
 className="h-8 w-8"
 >
 <Pencil className="h-3.5 w-3.5" />
 </Button>
 <Button
 variant="ghost"
 size="icon"
 className="h-8 w-8 text-destructive hover:text-destructive"
 disabled={deleteDisabled}
 onClick={() => onDelete(q.id)}
 >
 <Trash2 className="h-3.5 w-3.5" />
 </Button>
</div>
</div>
);
}

```

// — QuestionManager

---

```

function QuestionManager({ game }: { game: Game }) {
 const { toast } = useToast();

```

```
const queryClient = useQueryClient();

const [dialogOpen, setDialogOpen] = useState(false);

const [editing, setEditing] = useState<Question | null>(null);

const [localOrder, setLocalOrder] = useState<Question[]>([]);

const { data: questions } = useListGameQuestions(game.id, {
 query: { queryKey: getListGameQuestionsQueryKey(game.id) },
});

const createQuestion = useCreateQuestion();

const updateQuestion = useUpdateQuestion();

const deleteQuestion = useDeleteQuestion();

const sorted = useMemo(
 () => [...(questions ?? [])].sort((a, b) => a.orderIndex - b.orderIndex),
 [questions],
);

// Sync server order to local when server data changes (and no drag in progress)
useEffect(() => {
 setLocalOrder(sorted);
}, [questions]);

const totalPoints = localOrder.reduce((sum, q) => sum + q.points, 0);

const invalidate = () => {
 queryClient.invalidateQueries({ queryKey: getListGameQuestionsQueryKey(game.id) });
```



```
queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });
};
```

```
const sensors = useSensors(
 useSensor(PointerSensor, { activationConstraint: { distance: 5 } }),
 useSensor(KeyboardSensor, { coordinateGetter: sortableKeyboardCoordinates }),
);
```

```
const handleDragEnd = async (event: DragEndEvent) => {
 const { active, over } = event;
 if (!over || active.id === over.id) return;
```

```
 const oldIndex = localOrder.findIndex((q) => q.id === active.id);
 const newIndex = localOrder.findIndex((q) => q.id === over.id);
 const newOrder = arrayMove(localOrder, oldIndex, newIndex);
 setLocalOrder(newOrder);
```

```
 const toUpdate = newOrder
 .map((q, idx) => ({ q, idx }))
 .filter(({ q, idx }) => q.orderIndex !== idx);
```

```
 try {
 await Promise.all(
 toUpdate.map(({ q, idx }) =>
 updateQuestion.mutateAsync({
 questionId: q.id,
```

```

 data: payloadFromExisting(q, idx),
 }),
),
);
 invalidate();
} catch {
 toast({ variant: "destructive", title: "Reorder failed" });
 setLocalOrder(sorted);
}
};

```

```

const handleDelete = (id: number) => {
 if (!window.confirm("Delete this question?")) return;
 deleteQuestion.mutate(
 { questionId: id },
 {
 onSuccess: () => { invalidate(); toast({ title: "Question deleted" }); },
 onError: () => toast({ variant: "destructive", title: "Delete failed" }),
 },
);
};

```

```

return (
 <div className="space-y-4">
 {/* Header */}
 <div className="flex items-center justify-between gap-3 flex-wrap">

```

```

<div>

 <h3 className="font-bold text-lg leading-tight">{game.topic}</h3>

 <div className="flex items-center gap-3 text-sm text-muted-foreground mt-0.5">

 {localOrder.length}{localOrder.length === 1 ? "question" :
"questions"}

 {localOrder.length > 0 && (

 <>

 .

 <Star className="h-3.5 w-3.5" /> {totalPoints} pts total

 </>

)}

 </div>

</div>

<Dialog
 open={dialogOpen}
 onOpenChange={({open}) => {
 setDialogOpen(open);
 if (!open) setEditing(null);
 }}
>

 <DialogTrigger asChild>

 <Button className="font-bold">

 <Plus className="mr-1.5 h-4 w-4" /> Add Question

 </Button>

```

```

</DialogTrigger>

<DialogContent className="max-h-[90dvh] overflow-y-auto sm:max-w-lg">

 <DialogHeader>

 <DialogTitle>{editing ? "Edit Question" : "New Question"}</DialogTitle>

 </DialogHeader>

 <QuestionForm

 key={editing?.id ?? "new"}

 initial={editing ? formFromQuestion(editing) : emptyForm}

 pending={createQuestion.isPending || updateQuestion.isPending}

 submitLabel={editing ? "Save Changes" : "Add Question"}

 onSubmit={(form) => {

 if (editing) {

 updateQuestion.mutate(

 { questionId: editing.id, data: buildPayload(form, editing.orderIndex) },

 {

 onSuccess: () => {

 invalidate();

 setDialogOpen(false);

 setEditing(null);

 toast({ title: "Question updated" });

 },

 onError: () => toast({ variant: "destructive", title: "Update failed" }),

 },

);

 } else {

 const nextIndex =

```

```

 localOrder.length > 0

 ? Math.max(...localOrder.map((q) => q.orderIndex)) + 1
 : 0;

const payload = buildPayload(form, nextIndex);
createQuestion.mutate(
 {
 gameId: game.id,
 data: {
 ...payload,
 source: payload.source ?? "manual",
 verifiedByAdmin: true,
 },
 },
 {
 onSuccess: () => {
 invalidate();
 setDialogOpen(false);
 toast({ title: "Question added" });
 },
 onError: () => toast({ variant: "destructive", title: "Create failed" }),
 },
);
}
}
/>
</DialogContent>

```

```
</Dialog>
```

```
</div>
```

```
{/* Question list */}
```

```
{localOrder.length === 0 ? (
```

```
 <Card className="border-dashed border-primary/30 bg-card/30">
```

```
 <CardContent className="py-12 text-center space-y-2">
```

```
 <HelpCircle className="mx-auto h-10 w-10 text-primary/40" />
```

```
 <p className="font-semibold">No questions yet</p>
```

```
 <p className="text-sm text-muted-foreground max-w-sm mx-auto">
```

```
 Click "Add Question" to write your first verified question for this game.
```

```
 </p>
```

```
 </CardContent>
```

```
 </Card>
```

```
): (
```

```
 <>
```

```
 <p className="text-xs text-muted-foreground flex items-center gap-1.5">
```

```
 <GripVertical className="h-3.5 w-3.5" /> Drag the handle to reorder
```

```
 </p>
```

```
 <DndContext
```

```
 sensors={sensors}
```

```
 collisionDetection={closestCenter}
```

```
 onDragEnd={handleDragEnd}
```

```
 >
```

```
 <SortableContext
```

```
 items={localOrder.map((q) => q.id)}
```

```

 strategy={verticalListSortingStrategy}
 >
 <div className="space-y-2">
 {localOrder.map((q, i) => (
 <SortableQuestionItem
 key={q.id}
 q={q}
 index={i}
 onEdit={(q) => { setEditing(q); setDialogOpen(true); }}
 onDelete={handleDelete}
 deleteDisabled={deleteQuestion.isPending}
 />
))}
 </div>
</SortableContext>
</DndContext>

{/* Total points footer */}
<div className="flex justify-end pt-1">
 <div className="rounded-lg border border-accent/30 bg-accent/5 px-4 py-2 text-sm
font-semibold text-accent flex items-center gap-2">
 <Star className="h-4 w-4" />
 Total possible: {totalPoints} points
 </div>
</div>
</>

```

```

 })
 </div>

);
}

// — ResultsPanel —————

function ResultsPanel({ game, onClose }: { game: Game; onClose: () => void }) {
 const { data: participants } = useListGameParticipants(game.id, {
 query: { queryKey: getListGameParticipantsQueryKey(game.id) },
 });

 const sorted = useMemo(
 () => [...(participants ?? [])].sort((a, b) => b.totalScore - a.totalScore),
 [participants],
);

 return (
 <div className="space-y-4">
 <div className="flex items-center justify-between gap-3">
 <div>
 <h3 className="font-bold text-lg flex items-center gap-2">
 <Trophy className="h-5 w-5 text-accent" /> Results — {game.topic}
 </h3>
 <p className="text-sm text-muted-foreground">
 {sorted.length} {sorted.length === 1 ? "player" : "players"} competed
 </p>

```



```

</div>

<Button variant="ghost" size="sm" onClick={onClose}>

 <X className="mr-1 h-4 w-4" /> Close

</Button>

</div>

{sorted.length === 0 ? (

 <Card className="border-dashed border-card-border bg-card/40">

 <CardContent className="py-10 text-center text-muted-foreground">

 No players joined this game.

 </CardContent>

 </Card>

) : (

 <Card className="border-card-border bg-card/60">

 <CardContent className="p-0 divide-y divide-border">

 {sorted.map((p, i) => (

 <div key={p.id} className="flex items-center gap-3 px-4 py-3">

 {i === 0 ? (

 <Crown className="h-5 w-5 text-accent mx-auto" />

) : (

 `#${i + 1}`

)}

 {p.userName}

 {p.totalScore}


```

```


 pts

 </div>

)}}

</CardContent>

</Card>

)}

</div>

);

}

```

// — Dashboard section —————

```

function DashboardSection({
 games,
 onNavigate,
}: {
 games: Game[];
 onNavigate: (s: Section, game?: Game) => void;
}) {
 const { data: stats } = useGetStatsSummary({
 query: { queryKey: getGetStatsSummaryQueryKey(), refetchInterval: 10000 },
 });

 const { toast } = useToast();

 const queryClient = useQueryClient();

 const createGame = useCreateGame();

```

```
const createQuestion = useCreateQuestion();

const [buildingSample, setBuildingSample] = useState(false);

const handleCreateSampleStructure = async () => {
 setBuildingSample(true);
 try {
 const game = await createGame.mutateAsync({
 data: {
 topic: "Sample Game — Replace This Topic",
 difficulty: "medium",
 createdByAdmin: true,
 },
 });
 type Opts = { [key: string]: unknown } | null;
 const placeholders = [
 {
 questionType: "multiple_choice" as const,
 questionText: "[Q1: Replace with your verified multiple choice question]",
 options: ["Option A — replace", "Option B — replace", "Option C — replace", "Option D — replace"] as unknown as Opts,
 correctAnswer: "Option A — replace",
 points: 100,
 orderIndex: 0,
 },
 {
 questionType: "true_false" as const,
```

```
questionText: "[Q2: Replace with your verified true/false statement]",
options: ["True", "False"] as unknown as Opts,
correctAnswer: "True",
points: 100,
orderIndex: 1,
},
{
 questionType: "write_in" as const,
 questionText: "[Q3: Replace with your verified write-in question]",
 options: null,
 correctAnswer: "[replace with correct answer]",
 points: 100,
 orderIndex: 2,
},
{
 questionType: "image_recognition" as const,
 questionText: "[Q4: Replace with your image recognition question — add an
imageUrl]",
 options: null,
 correctAnswer: "[replace with correct answer]",
 points: 100,
 orderIndex: 3,
 imageUrl: null,
},
{
 questionType: "matching" as const,
```

```

 questionText: "[Q5: Replace with your matching question]",
 options: {
 leftOptions: ["Term A — replace", "Term B — replace", "Term C — replace"],
 rightOptions: ["Match 1 — replace", "Match 2 — replace", "Match 3 — replace"],
 } as Opts,
 correctAnswer: "Term A — replace:Match 1 — replace|Term B — replace:Match 2 —
replace|Term C — replace:Match 3 — replace",
 points: 100,
 orderIndex: 4,
 },
];
for (const q of placeholders) {
 await createQuestion.mutateAsync({ gameId: game.id, data: q });
}
queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });
toast({
 title: "Sample structure created",
 description: `Game #${game.id} created with 5 placeholder questions. Replace all
[bracketed] text with real, verified facts.`,
});
onNavigate("questions");
} catch {
 toast({ title: "Failed to create sample", variant: "destructive" });
} finally {
 setBuildingSample(false);
}
};

```

```

const activeGame = games.find((g) => g.status === "active");
const waitingGames = games.filter((g) => g.status === "waiting");
const completedGames = games.filter((g) => g.status === "completed");
const statCards = [
 { label: "Total Games", value: stats?.totalGames ?? 0, icon: Gamepad2, color: "text-
primary" },
 { label: "Live Now", value: stats?.activeGames ?? 0, icon: Radio, color: "text-secondary" },
 { label: "Total Players", value: stats?.totalPlayers ?? 0, icon: Users, color: "text-accent" },
 { label: "Answers", value: stats?.totalAnswers ?? 0, icon: Zap, color: "text-primary" },
];

return (
 <div className="space-y-6">
 {activeGame && (
 <Card className="border-primary/60 bg-primary/5">
 <CardContent className="p-4 flex items-center justify-between gap-3 flex-wrap">
 <div className="flex items-center gap-3">
 <div className="h-2.5 w-2.5 rounded-full bg-primary animate-pulse" />
 <div>
 <p className="text-xs font-semibold uppercase tracking-widest text-primary">
 Game In Progress
 </p>
 <p className="font-bold text-lg">{activeGame.topic}</p>
 </div>
 </div>
 </div>
 <div className="flex items-center gap-2">

```

```

<Badge variant="outline" className="uppercase">{activeGame.difficulty}</Badge>

 {activeGame.questionCount} questions

<Button size="sm" onClick={() => onNavigate("manage")}>
 Manage <ChevronRight className="ml-1 h-3.5 w-3.5" />
</Button>

</div>

</CardContent>

</Card>

)}

```

```

<div className="grid grid-cols-2 lg:grid-cols-4 gap-3">
 {statCards.map((s, i) => (
 <motion.div
 key={s.label}
 initial={{ opacity: 0, y: 8 }}
 animate={{ opacity: 1, y: 0 }}
 transition={{ delay: i * 0.06 }}
 >
 <Card className="border-card-border bg-card/60">
 <CardContent className="p-4 flex items-center gap-3">
 <s.icon className={`h-6 w-6 shrink-0 ${s.color}`} />
 <div>
 <div className="text-2xl font-bold tabular-nums">{s.value}</div>

```

```
 <div className="text-xs uppercase tracking-widest text-muted-foreground
leading-tight">
```

```
 {s.label}
```

```
 </div>
```

```
 </div>
```

```
 </CardContent>
```

```
</Card>
```

```
</motion.div>
```

```
)))
```

```
</div>
```

```
<div className="grid gap-4 md:grid-cols-3">
```

```
{[
```

```
 { id: "create" as Section, icon: PlusCircle, title: "New Game", sub: "Set up a fresh
round", color: "bg-primary/10 text-primary", border: "hover:border-primary/50" },
```

```
 { id: "manage" as Section, icon: Gamepad2, title: "Manage Games", sub:
` ${waitingGames.length} waiting · ${completedGames.length} done`, color: "bg-
secondary/10 text-secondary", border: "hover:border-secondary/50" },
```

```
 { id: "questions" as Section, icon: HelpCircle, title: "Add Questions", sub: "Build your
question bank", color: "bg-accent/10 text-accent", border: "hover:border-accent/50" },
```

```
].map((item) => (
```

```
 <Card
```

```
 key={item.id}
```

```
 className={` border-card-border bg-card/60 cursor-pointer transition-colors
${item.border} ` }
```

```
 onClick={() => onNavigate(item.id)}
```

```
 >
```

```
 <CardContent className="p-5 flex items-center gap-4">
```



```

<div className={`rounded-lg p-3 ${item.color}`}>
 <item.icon className="h-6 w-6" />
</div>

<div>
 <p className="font-bold">{item.title}</p>
 <p className="text-sm text-muted-foreground">{item.sub}</p>
</div>

<ChevronRight className="ml-auto h-4 w-4 text-muted-foreground" />
</CardContent>
</Card>
)}}
</div>

```

```

{import.meta.env.DEV && (
 <Card className="border-dashed border-yellow-500/40 bg-yellow-500/5">
 <CardContent className="p-4 flex items-center justify-between gap-4 flex-wrap">
 <div className="flex items-center gap-3">
 <div className="rounded-lg p-2 bg-yellow-500/10 text-yellow-400">
 <FlaskConical className="h-5 w-5" />
 </div>
 <div>
 <p className="font-semibold text-sm">Dev: Create Sample Structure</p>
 <p className="text-xs text-muted-foreground">
 Creates one game with 5 typed placeholder questions. Replace all{" "}
 [bracketed] text with real
 verified facts before going live.
 </p>

```

```

 </p>
 </div>
</div>

<Button
 size="sm"
 variant="outline"
 className="border-yellow-500/40 text-yellow-400 hover:bg-yellow-500/10 shrink-
0"
 disabled={buildingSample}
 onClick={handleCreateSampleStructure}
>
 {buildingSample ? "Creating..." : "Create Sample"}
</Button>
</CardContent>
</Card>
)}
</div>

);
}

```

// — Create Game section —————

```

function CreateGameSection({ onCreate }: { onCreate: (game: Game) => void }) {
 const [categoryId, setCategoryId] = useState<string>("9");
 const [customTopic, setCustomTopic] = useState("");
 const [difficulty, setDifficulty] = useState<Game["difficulty"]>("medium");

```

```
const [amount, setAmount] = useState("10");
const [created, setCreated] = useState<Game | null>(null);
const [working, setWorking] = useState(false);
const [workingLabel, setWorkingLabel] = useState("");
const [importedCount, setImportedCount] = useState<number | null>(null);
const [importError, setImportError] = useState<string | null>(null);
const [importSource, setImportSource] = useState<"opentdb" | "gemini" | "manual" |
null>(null);
const [retryCountdown, setRetryCountdown] = useState(0);
const { toast } = useToast();
const queryClient = useQueryClient();
const createGame = useCreateGame();
const importQuestions = useImportOpenTdbQuestions();
const generateQuestions = useGenerateGeminiQuestions();

useEffect(() => {
 if (retryCountdown <= 0) return;
 const t = setTimeout(() => setRetryCountdown((n) => Math.max(0, n - 1)), 1000);
 return () => clearTimeout(t);
}, [retryCountdown]);

const isCustom = categoryId === "custom";
const selectedCategory = OPENTDB_CATEGORIES.find((c) => String(c.id) === categoryId);
const topicName = isCustom ? customTopic.trim() : (selectedCategory?.name ?? "");
const canSubmit = topicName.length > 0 && !createGame.isPending && !working;
```

```

const handleSubmit = async (e: React.FormEvent) => {
 e.preventDefault();
 if (!canSubmit) return;

 let game: Game;
 try {
 setWorkingLabel("Creating game...");
 setWorking(true);
 game = await createGame.mutateAsync({
 data: { topic: topicName, difficulty, createdByAdmin: true },
 });
 queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });
 queryClient.invalidateQueries({ queryKey: getGetStatsSummaryQueryKey() });
 } catch (err: unknown) {
 const status = err && typeof err === "object" && "status" in err ? (err as { status: number }).status : 0;

 toast({
 variant: "destructive",
 title: status === 403 ? "Session expired" : "Failed to create game",
 description: status === 403 ? "Please log out and log back in with the admin code." :
undefined,
 });
 setWorking(false);
 return;
 }

 if (!isCustom && selectedCategory) {

```

```

// OpenTDB path

setWorkingLabel("Importing from Open Trivia Database...");

try {

 const result = await importQuestions.mutateAsync({

 gameId: game.id,

 data: { categoryId: selectedCategory.id, difficulty, amount: Number(amount) },

 });

 queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });

 setImportedCount(result.imported);

 setImportSource("opentdb");

 toast({ title: ` ${result.imported} questions imported from Open Trivia DB! ` });

} catch (err: unknown) {

 const msg =

 err instanceof Error ? err.message : "Could not fetch questions from Open Trivia Database.";

 setImportError(msg);

 setImportSource("opentdb");

 toast({ variant: "destructive", title: "Import failed — add questions manually" });

}

} else {

 // Gemini path for custom topics

 setWorkingLabel("Generating fact-based questions with AI... This may take 10–15 seconds");

 try {

 const result = await generateQuestions.mutateAsync({

 gameId: game.id,

 data: { topic: topicName, difficulty, amount: Number(amount) },

 });

 }

```

```

});

queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });

setImportedCount(result.imported);

setImportSource("gemini");

toast({ title: ` ${result.imported} questions generated with Gemini AI! ` });
} catch (err: unknown) {

 const msg = err instanceof Error ? err.message : "Could not generate questions.";

 setImportError(msg);

 setImportSource("gemini");

 const status = err && typeof err === "object" && "status" in err ? (err as { status: number
}).status : 0;

 const isRateLimit = status === 429 || msg.includes("Too many requests") ||
msg.includes("429") || msg.includes("rate limit");

 if (isRateLimit) {

 setRetryCountdown(60);

 toast({ variant: "destructive", title: "Gemini rate limited — retry unlocks in 60 s" });

 } else {

 toast({ variant: "destructive", title: "Generation failed — add questions manually" });

 }

}

setWorking(false);

setCreated(game);

};

const handleRetryGeneration = async () => {

```

```

if (!created) return;

setWorking(true);

setImportError(null);

setImportedCount(null);

setWorkingLabel("Retrying AI question generation... This may take up to 30 seconds");

try {

 const result = await generateQuestions.mutateAsync({

 gameId: created.id,

 data: {

 topic: created.topic,

 difficulty: (created.difficulty ?? "medium") as "easy" | "medium" | "hard",

 amount: Number(amount),

 },

 });

 queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });

 setImportedCount(result.imported);

 setImportSource("gemini");

 toast({ title: `${result.imported} questions generated!` });

} catch (err: unknown) {

 const msg = err instanceof Error ? err.message : "Could not generate questions.";

 setImportError(msg);

 setImportSource("gemini");

 const status = err && typeof err === "object" && "status" in err ? (err as { status: number }).status : 0;

 const isRateLimit = status === 429 || msg.includes("Too many requests") ||
msg.includes("429") || msg.includes("rate limit");

 if (isRateLimit) {

```

```
 setRetryCountdown(60);

 toast({ variant: "destructive", title: "Still rate limited — retry unlocks in 60 s" });
 } else {
 toast({ variant: "destructive", title: "Generation failed — add questions manually" });
 }
} finally {
 setWorking(false);
}
};
```

```
const handleReset = () => {
 setCreated(null);
 setImportedCount(null);
 setImportError(null);
 setImportSource(null);
 setCustomTopic("");
};
```

```
if (created) {
 return (
 <motion.div
 initial={{ opacity: 0, scale: 0.95 }}
 animate={{ opacity: 1, scale: 1 }}
 className="space-y-4"
 >
 <Card className="border-2 border-secondary/50 bg-secondary/5">
```



```

<CardContent className="py-10 text-center space-y-3">
 {working ? (
 <>
 <Loader2 className="mx-auto h-14 w-14 text-primary animate-spin" />
 <h3 className="text-2xl font-bold tracking-tight">{workingLabel}</h3>
 <p className="text-muted-foreground text-sm">This may take a few
seconds...</p>
 </>
) : importedCount !== null ? (
 <>
 <CheckCircle2 className="mx-auto h-14 w-14 text-secondary" />
 <h3 className="text-2xl font-bold tracking-tight">Ready to Go!</h3>
 <p className="text-muted-foreground">
 {created.topic}{" "}
 — {importedCount}
questions{" "}
 {importSource === "gemini" ? "generated by Gemini AI" : "imported from Open
Trivia Database"}.
 </p>
 {importSource === "gemini" && (
 <div className="flex items-center justify-center gap-2 text-xs text-yellow-400 bg-
yellow-400/10 rounded-lg px-3 py-2 max-w-sm mx-auto">
 <AlertTriangle className="h-3.5 w-3.5 shrink-0" />
 AI-generated — please verify all questions in the Review tab before going live.
 </div>
)}
 <p className="text-xs text-muted-foreground">

```

Game ID: <span className="font-mono text-secondary">#{created.id}</span>

{ " · "}Review or edit questions before going live.

</p>

<div className="flex justify-center gap-3 pt-2">

<Button className="font-bold" onClick={() => onCreate(created)}>

<ListChecks className="mr-2 h-4 w-4" /> Review Questions

</Button>

<Button variant="outline" onClick={handleReset}>

Create Another

</Button>

</div>

</>

): importError ? (

<>

<AlertTriangle className="mx-auto h-14 w-14 text-orange-400" />

<h3 className="text-2xl font-bold tracking-tight">Game Created</h3>

<p className="text-muted-foreground max-w-sm mx-auto text-sm">

<span className="font-semibold text-foreground">{created.topic}</span> is  
ready, but

{importSource === "gemini" ? " Gemini" : " auto-import"} hit a snag:{ " "}

{importError.includes("Too many requests") ? "rate limited by Gemini API" :  
importError}

</p>

<div className="flex flex-col sm:flex-row justify-center gap-3 pt-2">

{importSource === "gemini" && (

<Button

className="font-bold"

```

 onClick={handleRetryGeneration}
 disabled={working || retryCountdown > 0}
 >
 {working ? (
 <><Loader2 className="mr-2 h-4 w-4 animate-spin" />Retrying...</>
) : retryCountdown > 0 ? (
 <><RefreshCw className="mr-2 h-4 w-4" />Retry in {retryCountdown}s</>
) : (
 <><RefreshCw className="mr-2 h-4 w-4" />Retry Generation</>
)}
 </Button>
)}

<Button
 variant={importSource === "gemini" ? "outline" : "default"}
 className="font-bold"
 onClick={() => onCreate(created)}
 >
 <Pencil className="mr-2 h-4 w-4" /> Add Questions Manually
 </Button>

 <Button variant="ghost" onClick={handleReset}>
 Create Another
 </Button>
</div>
</>

): null}
</CardContent>

```

```

 </Card>

 </motion.div>

);
}

return (
 <div className="space-y-6 max-w-xl">
 <div>
 <h2 className="text-xl font-bold tracking-tight">Create a New Game</h2>
 <p className="text-muted-foreground text-sm mt-1">
 Standard categories auto-import from Open Trivia Database. Custom topics use
 Gemini AI.
 </p>
 </div>
 <form onSubmit={handleSubmit} className="space-y-5">

 {/* Category / Topic */}
 <div className="space-y-2">
 <Label>Category</Label>
 <Select value={categoryId} onChange={setCategoryId}>
 <SelectTrigger className="h-12">
 <SelectValue placeholder="Select a category..." />
 </SelectTrigger>
 <SelectContent>
 <SelectItem value="custom"> ✨ Custom Topic — Gemini AI generates
 questions</SelectItem>

```

```
<Separator className="my-1" />

{OPENTDB_CATEGORIES.map((cat) => (

 <SelectItem key={cat.id} value={String(cat.id)}>

 {cat.name}

 </SelectItem>

))}

</SelectContent>

</Select>

</div>
```

```
{/* Custom topic text input */}

{isCustom && (

 <div className="space-y-2">

 <Label htmlFor="customTopic">Custom Topic</Label>

 <Input

 id="customTopic"

 value={customTopic}

 onChange={(e) => setCustomTopic(e.target.value)}

 placeholder="e.g. Harry Potter, The Office, 80s Music, Local History..."

 className="h-12 text-base"

 autoFocus

 />

 </div>

)}


```

```
<div className="grid grid-cols-2 gap-4">
```

```

<div className="space-y-2">
 <Label>Difficulty</Label>
 <Select value={difficulty} onChange={(v) => setDifficulty(v as
Game["difficulty"])}>
 <SelectTrigger className="h-12"><SelectValue /></SelectTrigger>
 <SelectContent>
 <SelectItem value="easy">Easy (5 pts each)</SelectItem>
 <SelectItem value="medium">Medium (10 pts each)</SelectItem>
 <SelectItem value="hard">Hard (15 pts each)</SelectItem>
 </SelectContent>
 </Select>
</div>

<div className="space-y-2">
 <Label>Questions to {isCustom ? "Generate" : "Import"}</Label>
 <Select value={amount} onChange={setAmount}>
 <SelectTrigger className="h-12"><SelectValue /></SelectTrigger>
 <SelectContent>
 <SelectItem value="5">5 questions</SelectItem>
 <SelectItem value="10">10 questions</SelectItem>
 <SelectItem value="15">15 questions</SelectItem>
 {!isCustom && <SelectItem value="20">20 questions</SelectItem>}
 </SelectContent>
 </Select>
</div>
</div>

```

```

{ /* Info card */
{isCustom ? (
 <Card className="border-purple-500/30 bg-purple-500/5">
 <CardContent className="p-4 text-sm space-y-1.5 flex gap-3">
 <Lightbulb className="h-4 w-4 text-purple-400 shrink-0 mt-0.5" />
 <div className="space-y-1">
 <p className="font-semibold text-purple-400">Gemini AI will generate {amount}
questions</p>
 <p className="text-muted-foreground">
 Questions will be marked as AI-generated and unverified. Review them in the
 Review Questions tab before going live.
 </p>
 </div>
 </CardContent>
 </Card>
) : (
 <Card className="border-primary/20 bg-primary/5">
 <CardContent className="p-4 text-sm space-y-1 flex gap-3">
 <Database className="h-4 w-4 text-primary shrink-0 mt-0.5" />
 <div>
 <p className="font-semibold text-primary">Auto-import from Open Trivia
Database</p>
 <p className="text-muted-foreground">
 {amount} {difficulty} questions about{" "}
 <span className="font-medium text-
foreground">{selectedCategory?.name}{" "}
 will be fetched from the free, community-verified database.

```

```

 </p>
 </div>
</CardContent>
</Card>
)}

<Button
 type="submit"
 className="w-full h-12 font-bold text-base"
 disabled={!canSubmit}
>
 {createGame.isPending || working ? (
 <><Loader2 className="mr-2 h-4 w-4 animate-spin" /> {workingLabel ||
"Working..."}</>
) : isCustom ? (
 <><Lightbulb className="mr-2 h-4 w-4" /> Create & Generate with Gemini</>
) : (
 <><Database className="mr-2 h-4 w-4" /> Create & Import from OpenTDB</>
)}
</Button>
</form>
</div>

);
}

// — Manage Games section

```

---



```

function ManageGamesSection({
 games,
 onManageQuestions,
}: {
 games: Game[];
 onManageQuestions: (game: Game) => void;
}) {
 const { toast } = useToast();
 const queryClient = useQueryClient();
 const [viewingResults, setViewingResults] = useState<Game | null>(null);
 const [confirmStart, setConfirmStart] = useState<{ game: Game; unverifiedCount:
number } | null>(null);
 const [checking, setChecking] = useState<number | null>(null);
 const updateGame = useUpdateGame();
 const deleteGame = useDeleteGame();
 const hasActive = games.some((g) => g.status === "active");

 const invalidate = () => {
 queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });
 queryClient.invalidateQueries({ queryKey: getGetStatsSummaryQueryKey() });
 };

 const doGoLive = (game: Game) => {
 updateGame.mutate(
 { gameId: game.id, data: { status: "active" } },

```

```

{
 onSuccess: () => { invalidate(); toast({ title: ` "${game.topic}" is now live!` }); },
 onError: () => toast({ variant: "destructive", title: "Failed to start" }),
},
);
setConfirmStart(null);
};

const handleGoLive = async (game: Game) => {
 // Check the "require all verified" setting from localStorage
 let requireVerify = true;

 try { requireVerify = localStorage.getItem(REQUIRE_VERIFY_KEY) !== "false"; } catch { /*
ignore */ }

 if (!requireVerify) {
 doGoLive(game);
 return;
 }

 setChecking(game.id);

 try {
 const resp = await fetch(`/api/games/${game.id}/questions`, { credentials: "include" });
 if (resp.ok) {
 const qs = (await resp.json()) as Question[];
 const unverified = qs.filter((q) => !q.verifiedByAdmin).length;
 if (unverified > 0) {

```

```

 setConfirmStart({ game, unverifiedCount: unverified });
 setChecking(null);
 return;
 }
}
} catch {
 // fail open — proceed without verification check
}
setChecking(null);
doGoLive(game);
};

if (viewingResults) {
 return <ResultsPanel game={viewingResults} onClose={() => setViewingResults(null)} />;
}

const ordered = [...games].sort((a, b) => {
 const rank: Record<string, number> = { active: 0, waiting: 1, completed: 2 };
 return (rank[a.status] ?? 3) - (rank[b.status] ?? 3);
});

return (
 <div className="space-y-4">
 <div>
 <h2 className="text-xl font-bold tracking-tight">Manage Games</h2>
 <p className="text-muted-foreground text-sm mt-1">

```

Start, end, or delete games. Only one game can be live at a time.

</p>

</div>

{ordered.length === 0 ? (

<Card className="border-dashed border-primary/30 bg-card/40">

<CardContent className="py-14 text-center space-y-2">

<Gamepad2 className="mx-auto h-10 w-10 text-primary/40" />

<p className="font-semibold text-lg">No games yet</p>

<p className="text-sm text-muted-foreground">

Head to "Create Game" to set up your first round.

</p>

</CardContent>

</Card>

): (

<div className="space-y-3">

{ordered.map((game, i) => {

const isWaiting = game.status === "waiting";

const isActive = game.status === "active";

const isDone = game.status === "completed";

const noQuestions = game.questionCount === 0;

return (

<motion.div

key={game.id}

initial={{ opacity: 0, y: 8 }}

animate={{ opacity: 1, y: 0 }}

transition={{ delay: i \* 0.04 }}

>

<Card

```
className={`border-card-border bg-card/60 ${
 isActive ? "border-primary/40 ring-1 ring-primary/20" : ""
}}`
```

>

<CardContent className="p-4 space-y-3">

<div className="flex items-start justify-between gap-3 flex-wrap">

<div className="space-y-1 min-w-0">

<div className="flex items-center gap-2 flex-wrap">

<h3 className="font-bold text-base">{game.topic}</h3>

<Badge

```
className={`uppercase text-[10px] ${
```

```
 isActive
```

```
 ? "bg-primary text-primary-foreground"
```

```
 : isWaiting
```

```
 ? "bg-secondary/20 text-secondary border border-secondary/40"
```

```
 : "bg-muted text-muted-foreground"
```

```
 }}`
```

>

```
{isActive ? "LIVE" : game.status}
```

</Badge>

<Badge variant="outline" className="uppercase text-[10px]">

```
{game.difficulty}
```

</Badge>

</div>

```

<p className="text-sm text-muted-foreground">
 Game #{game.id} · {game.questionCount}" "
 {game.questionCount === 1 ? "question" : "questions"}
</p>
</div>
</div>

{isWaiting && noQuestions && (
 <div className="flex items-center gap-2 rounded-md bg-destructive/10 border
border-destructive/20 px-3 py-2 text-sm text-destructive">
 <AlertTriangle className="h-4 w-4 shrink-0" />
 No questions added — add questions before going live
 </div>
)}

{isWaiting && hasActive && (
 <div className="flex items-center gap-2 rounded-md bg-accent/10 border
border-accent/20 px-3 py-2 text-sm text-accent">
 <AlertTriangle className="h-4 w-4 shrink-0" />
 Another game is already live — end it first
 </div>
)}

<div className="flex items-center gap-2 flex-wrap">
 <Button variant="outline" size="sm" onClick={() => onManageQuestions(game)}>
 <ListChecks className="mr-1 h-4 w-4" /> Questions
 </Button>
 {isDone && (
 <Button variant="outline" size="sm" onClick={() => setViewingResults(game)}>
 <Trophy className="mr-1 h-4 w-4" /> View Results
)
 }
}

```

```

 </Button>

)}

 {isWaiting && (

 <Button

 size="sm"

 className="font-bold"

 disabled={updateGame.isPending || checking === game.id || noQuestions ||
hasActive}

 onClick={() => handleGoLive(game)}

 >

 {checking === game.id ? (

 <Loader2 className="mr-1 h-4 w-4 animate-spin" />

) : (

 <Play className="mr-1 h-4 w-4" />

)}

 Go Live

 </Button>

)}

 {isActive && (

 <Button

 size="sm"

 variant="secondary"

 className="font-bold"

 disabled={updateGame.isPending}

 onClick={() =>

 updateGame.mutate(

```

```

 { gameId: game.id, data: { status: "completed" } },
 { onSuccess: () => { invalidate(); toast({ title: "Game ended" }); } },
)
}
>
<Flag className="mr-1 h-4 w-4" /> End Game
</Button>
)}
<Button
 variant="ghost"
 size="sm"
 className="text-destructive hover:text-destructive ml-auto"
 disabled={deleteGame.isPending}
 onClick={() => {
 if (window.confirm(`Delete "${game.topic}"? This removes all questions,
participants, and answers.`)) {
 deleteGame.mutate(
 { gameId: game.id },
 {
 onSuccess: () => { invalidate(); toast({ title: "Game deleted" }); },
 onError: () => toast({ variant: "destructive", title: "Delete failed" }),
 },
);
 }
 }}
>

```



```

 <Trash2 className="mr-1 h-4 w-4" /> Delete
 </Button>
 </div>
 </CardContent>
</Card>
</motion.div>

);
}}
</div>
})

```

```

{/* Go Live confirmation dialog */}
{confirmStart && (
 <Dialog open onOpenChange={() => setConfirmStart(null)}>
 <DialogContent className="sm:max-w-md">
 <DialogHeader>
 <DialogTitle className="flex items-center gap-2">
 <AlertTriangle className="h-5 w-5 text-yellow-400" />
 Unverified questions
 </DialogTitle>
 </DialogHeader>
 <div className="space-y-3 py-2">
 <p className="text-sm text-muted-foreground">
 {confirmStart.unverifiedCount}{" "}
 {confirmStart.unverifiedCount === 1 ? "question has" : "questions have"} not been
 verified yet in{" "}

```

```
 {confirmStart.game.topic}.
```

```
 </p>
```

```
 <p className="text-xs text-muted-foreground">
```

```
 Use "Review Questions" to verify them before going live, or start anyway.
```

```
 </p>
```

```
 </div>
```

```
 <div className="flex gap-2 justify-end pt-1">
```

```
 <Button variant="outline" onClick={() => setConfirmStart(null)}>
```

```
 Cancel
```

```
 </Button>
```

```
 <Button
```

```
 variant="destructive"
```

```
 onClick={() => doGoLive(confirmStart.game)}
```

```
 disabled={updateGame.isPending}
```

```
 >
```

```
 <Play className="mr-1 h-4 w-4" /> Start Anyway
```

```
 </Button>
```

```
 </div>
```

```
</DialogContent>
```

```
</Dialog>
```

```
)}
```

```
</div>
```

```
);
```

```
}
```

// — Questions section —

---

```
function QuestionsSection({
 games,
 preferGameId,
}: {
 games: Game[];
 preferGameId?: number;
}) {
 const waitingGames = games.filter((g) => g.status === "waiting");
 const [selectedId, setSelectedId] = useState<number | null>(() => {
 if (preferGameId) return preferGameId;
 return waitingGames[0]?.id ?? null;
 });

 useEffect(() => {
 if (preferGameId) { setSelectedId(preferGameId); return; }
 if (selectedId === null && waitingGames.length > 0) {
 setSelectedId(waitingGames[0]!.id);
 }
 }, [games, preferGameId]);

 const selectedGame = games.find((g) => g.id === selectedId) ?? null;

 return (
 <div className="space-y-5">
```

```
<div>

 <h2 className="text-xl font-bold tracking-tight">Add Questions</h2>

 <p className="text-muted-foreground text-sm mt-1">

 Select a waiting game and build its question set manually.

 </p>

</div>
```

```
{/* Verified facts notice */}

<Card className="border-destructive/30 bg-destructive/5">

 <CardContent className="p-4 flex gap-3">

 <ShieldAlert className="h-5 w-5 text-destructive shrink-0 mt-0.5" />

 <div className="space-y-1">

 <p className="font-semibold text-sm text-destructive">

 All questions must be based on verified real-world facts

 </p>

 <p className="text-xs text-muted-foreground">

 Do not use AI-generated or unverified content. Every answer must be

 factually correct and confirmable from a reliable source. Players

 trust the host to get it right.

 </p>

 </div>

 </CardContent>

</Card>
```

```
{waitingGames.length === 0 ? (

 <Card className="border-dashed border-primary/30 bg-card/40">
```

```

<CardContent className="py-12 text-center space-y-2">
 <Gamepad2 className="mx-auto h-10 w-10 text-primary/40" />
 <p className="font-semibold">No games in waiting status</p>
 <p className="text-sm text-muted-foreground">
 Create a new game first, or games that are already live won't
 appear here — end them to add more questions.
 </p>
</CardContent>
</Card>
):(
<>
<div className="space-y-2">
 <Label>Select Game</Label>
 <Select
 value={selectedId !== null ? String(selectedId) : ""}
 onChange={(v) => setSelectedId(Number(v))}
 >
 <SelectTrigger className="h-11 max-w-sm">
 <SelectValue placeholder="Choose a game..." />
 </SelectTrigger>
 <SelectContent>
 {waitingGames.map((g) => (
 <SelectItem key={g.id} value={String(g.id)}>
 {g.topic}

 ({g.questionCount} questions)

```

```


 </SelectedItem>

)})

</SelectContent>

</Select>

</div>

```

```

{selectedGame ? (
 <QuestionManager game={selectedGame} />
) : (
 <p className="text-sm text-muted-foreground">Select a game above.</p>
)}
</>
)}
</div>

);
}

```

// — Review Questions section —————

```

function ReviewSection({ games }: { games: Game[] }) {
 const [selectedGameId, setSelectedGameId] = useState<number | null>(null);
 const [filter, setFilter] = useState<"all" | "unverified" | "opentdb" | "ai" | "manual">("all");
 const [sortBy, setSortBy] = useState<"order" | "verified" | "type" | "dateAdded">("order");
 const [selected, setSelected] = useState<Set<number>>(new Set());
 const [editingQuestion, setEditingQuestion] = useState<Question | null>(null);

```

```
const [editDialogOpen, setEditDialogOpen] = useState(false);
```

```
const { toast } = useToast();
```

```
const queryClient = useQueryClient();
```

```
const updateQuestion = useUpdateQuestion();
```

```
const deleteQuestion = useDeleteQuestion();
```

```
// Regenerate modal state
```

```
const [regenQ, setRegenQ] = useState<Question | null>(null);
```

```
const [regenDiff, setRegenDiff] = useState<"same" | "easy" | "medium" | "hard">("same");
```

```
const [regenType, setRegenType] = useState<string>("same");
```

```
const [regenLoading, setRegenLoading] = useState(false);
```

```
const [regenPreview, setRegenPreview] = useState<RegenerateQuestionPreview | null>(null);
```

```
const [regenError, setRegenError] = useState<string | null>(null);
```

```
// Enhance modal state
```

```
const [enhQ, setEnhQ] = useState<Question | null>(null);
```

```
const [enhLoading, setEnhLoading] = useState(false);
```

```
const [enhResult, setEnhResult] = useState<EnhanceQuestionResult | null>(null);
```

```
const [enhError, setEnhError] = useState<string | null>(null);
```

```
const [enhAcceptText, setEnhAcceptText] = useState(false);
```

```
const [enhAcceptOptions, setEnhAcceptOptions] = useState(false);
```

```
const [enhAcceptSource, setEnhAcceptSource] = useState(false);
```

```
// Generate More modal state
```

```
const [genMoreOpen, setGenMoreOpen] = useState(false);
```

```

const [genMoreCount, setGenMoreCount] = useState(5);

const [genMoreDiff, setGenMoreDiff] = useState<"easy" | "medium" | "hard" |
"same">("same");

const [genMoreAvoid, setGenMoreAvoid] = useState(true);

// Fact check state

type FcResult = { question: Question; verdict: string; confidence: string; explanation:
string; correctAnswerIfWrong: string | null };

type FcFailed = { question: Question; error: string };

const [fcState, setFcState] = useState<{

 running: boolean;

 current: number;

 total: number;

 results: FcResult[];

 failed: FcFailed[];

 rateLimited: boolean;

} | null>(null);

const regenMutation = useRegenerateQuestion();

const enhanceMutation = useEnhanceQuestion();

const factCheckMutation = useFactCheckQuestion();

const generateMore = useGenerateGeminiQuestions();

useEffect(() => {

 if (selectedGameId === null && games.length > 0) {

 setSelectedGameId(games[0]!.id);

 }

```



```
}, [games]);
```

```
const { data: rawQuestions = [] } = useListGameQuestions(selectedGameId ?? 0, {
 query: {
 queryKey: getListGameQuestionsQueryKey(selectedGameId ?? 0),
 enabled: selectedGameId !== null,
 },
});
```

```
const invalidate = () => {
 if (selectedGameId !== null) {
 queryClient.invalidateQueries({ queryKey:
getListGameQuestionsQueryKey(selectedGameId) });
 }
 queryClient.invalidateQueries({ queryKey: getListGamesQueryKey() });
};
```

```
const filtered = rawQuestions.filter((q) => {
 switch (filter) {
 case "unverified": return !q.verifiedByAdmin;
 case "opentdb": return q.source === "opentdb";
 case "ai": return !!q.aiGenerated;
 case "manual": return q.source === "manual" || (!q.source && !q.aiGenerated);
 default: return true;
 }
});
```

```
const displayList = [...filtered].sort((a, b) => {
 if (sortBy === "verified") return Number(b.verifiedByAdmin) - Number(a.verifiedByAdmin);
 if (sortBy === "type") return a.questionType.localeCompare(b.questionType);
 if (sortBy === "dateAdded") return b.id - a.id;
 return a.orderIndex - b.orderIndex;
});
```

```
const verifiedCount = rawQuestions.filter((q) => q.verifiedByAdmin).length;
const unverifiedCount = rawQuestions.length - verifiedCount;
```

```
const toggleSelect = (id: number) => {
 setSelected((prev) => {
 const next = new Set(prev);
 if (next.has(id)) next.delete(id); else next.add(id);
 return next;
 });
};

const allSelected = displayList.length > 0 && displayList.every((q) => selected.has(q.id));
const toggleSelectAll = () => {
 if (allSelected) setSelected(new Set());
 else setSelected(new Set(displayList.map((q) => q.id)));
};
```

```
const handleVerify = (q: Question, verified: boolean) => {
 updateQuestion.mutate(

```

```

 { questionId: q.id, data: { verifiedByAdmin: verified } },
 {
 onSuccess: () => { invalidate(); toast({ title: verified ? "Marked verified" : "Marked
unverified" }); },
 onError: () => toast({ variant: "destructive", title: "Update failed" }),
 },
);
};

```

```

const handleDelete = (id: number) => {
 if (!window.confirm("Delete this question?")) return;
 deleteQuestion.mutate(
 { questionId: id },
 {
 onSuccess: () => {
 invalidate();
 setSelected((prev) => { const n = new Set(prev); n.delete(id); return n; });
 toast({ title: "Question deleted" });
 },
 onError: () => toast({ variant: "destructive", title: "Delete failed" }),
 },
);
};

```

```

const handleBulkVerify = async () => {
 const ids = Array.from(selected);

```

```

await Promise.allSettled(
 ids.map((id) => {
 const q = rawQuestions.find((q) => q.id === id);
 if (!q?.verifiedByAdmin) {
 return updateQuestion.mutateAsync({ questionId: id, data: { verifiedByAdmin: true } });
 }
 return Promise.resolve();
 }),
);
invalidate();
setSelected(new Set());
toast({ title: `Verified ${ids.length} questions` });
};

```

```

const handleBulkDelete = async () => {
 if (!window.confirm(`Delete ${selected.size} selected questions?`)) return;
 const ids = Array.from(selected);
 await Promise.allSettled(ids.map((id) => deleteQuestion.mutateAsync({ questionId: id })));
 invalidate();
 setSelected(new Set());
 toast({ title: `Deleted ${ids.length} questions` });
};

```

```

const handleRunRegen = async () => {
 if (!regenQ || !selectedGameId) return;

```

```

setRegenLoading(true);
setRegenPreview(null);
setRegenError(null);
const game = games.find((g) => g.id === selectedGameId);
const difficulty =
 regenDiff === "same"
 ? ((game?.difficulty ?? "medium") as "easy" | "medium" | "hard")
 : regenDiff;
const questionType =
 regenType === "same"
 ? (regenQ.questionType as "multiple_choice" | "true_false" | "write_in")
 : (regenType as "multiple_choice" | "true_false" | "write_in");
try {
 const preview = await regenMutation.mutateAsync({
 gameId: selectedGameId,
 questionId: regenQ.id,
 data: { difficulty, questionType },
 });
 setRegenPreview(preview);
} catch (err) {
 const msg = err instanceof Error ? err.message : "Generation failed. Please try again.";
 setRegenError(msg.includes("Too many requests") ? "Rate limited — please wait a moment and try again." : msg);
} finally {
 setRegenLoading(false);
}

```

```
};
```

```
const handleAcceptRegen = () => {
 if (!regenQ || !regenPreview) return;
 const opts =
 regenPreview.options && regenPreview.options.length > 0
 ? ({ choices: regenPreview.options } as unknown as Record<string, unknown>)
 : null;
 updateQuestion.mutate(
 {
 questionId: regenQ.id,
 data: {
 questionText: regenPreview.questionText,
 questionType: regenPreview.questionType as "multiple_choice" | "true_false" |
"write_in",
 correctAnswer: regenPreview.correctAnswer,
 options: opts,
 points: regenPreview.points,
 source: regenPreview.source,
 verifiedByAdmin: false,
 },
 },
 {
 onSuccess: () => {
 invalidate();
 setRegenQ(null);
 }
 }
)
}
```

```

 setRegenPreview(null);

 toast({ title: "Question replaced" });

 },

 onError: () => toast({ variant: "destructive", title: "Update failed" }),

);

};

```

```

const handleRunEnhance = async (q: Question) => {
 if (!selectedGameId) return;

 setEnhQ(q);
 setEnhLoading(true);
 setEnhResult(null);
 setEnhError(null);
 setEnhAcceptText(false);
 setEnhAcceptOptions(false);
 setEnhAcceptSource(false);

 try {
 const result = await enhanceMutation.mutateAsync({ gameId: selectedGameId,
questionId: q.id });

 setEnhResult(result);
 } catch (err) {

 const msg = err instanceof Error ? err.message : "Enhancement failed. Please try again.";

 setEnhError(msg.includes("Too many requests") ? "Rate limited — please wait a
moment and try again." : msg);
 } finally {

 setEnhLoading(false);
 }
}

```

```

 }
};

const handleApplyEnhancements = () => {
 if (!enhQ || !enhResult) return;

 type Patch = {
 questionText?: string;
 options?: Record<string, unknown> | null;
 source?: string | null;
 };

 const patch: Patch = {};

 if (enhAcceptText) patch.questionText = enhResult.improvedQuestionText;

 if (enhAcceptOptions && enhResult.improvedOptions &&
 enhResult.improvedOptions.length > 0) {
 patch.options = { choices: enhResult.improvedOptions } as Record<string, unknown>;
 }

 if (enhAcceptSource && enhResult.suggestedSource) patch.source =
 enhResult.suggestedSource;

 if (Object.keys(patch).length === 0) {
 toast({ title: "No improvements selected" });

 return;
 }

 updateQuestion.mutate(
 { questionId: enhQ.id, data: patch as Parameters<typeof
 updateQuestion.mutate>[0]["data"] },
 {
 onSuccess: () => {

```



```

 invalidate();

 setEnhQ(null);

 setEnhResult(null);

 toast({ title: "Enhancements applied" });

 },

 onError: () => toast({ variant: "destructive", title: "Update failed" }),

},

);

};

```

```

const handleGenerateMore = async () => {

 if (!selectedGameId) return;

 const game = games.find((g) => g.id === selectedGameId);

 if (!game) return;

 const difficulty =

 genMoreDiff === "same"

 ? ((game.difficulty ?? "medium") as "easy" | "medium" | "hard")

 : genMoreDiff;

 const existingQs = genMoreAvoid ? rawQuestions.map((q) => q.questionText) : undefined;

 try {

 const result = await generateMore.mutateAsync({

 gameId: selectedGameId,

 data: { topic: game.topic, difficulty, amount: genMoreCount, existingQuestions:

existingQs },

 });

 invalidate();

```

```

 setGenMoreOpen(false);

 toast({ title: ` Added ${result.imported} questions — total now ${rawQuestions.length +
result.imported}` });

 } catch {

 toast({ variant: "destructive", title: "Generation failed. Please try again." });

 }

};

```

```

const handleFactCheckAll = async () => {

 if (!selectedGameId) return;

 const toCheck = rawQuestions.filter((q) => !q.verifiedByAdmin);

 if (toCheck.length === 0) {

 toast({ title: "All questions are already verified." });

 return;

 }

 setFcState({ running: true, current: 0, total: toCheck.length, results: [], failed: [],
rateLimited: false });

 const results: FcResult[] = [];

 const failed: FcFailed[] = [];

 for (let i = 0; i < toCheck.length; i++) {

 const q = toCheck[i]!;

 setFcState((prev) => (prev ? { ...prev, current: i + 1, rateLimited: false } : null));

 let done = false;

 for (let attempt = 0; attempt < 2; attempt++) {

 try {

 const r = await factCheckMutation.mutateAsync({ gameId: selectedGameId,
questionId: q.id });

```

```

results.push({
 question: q,
 verdict: r.verdict,
 confidence: r.confidence,
 explanation: r.explanation,
 correctAnswerIfWrong: r.correctAnswerIfWrong ?? null,
});

done = true;

break;
} catch (err) {

 const msg = err instanceof Error ? err.message : String(err);

 if ((msg.includes("Too many requests") || msg.includes("429")) && attempt === 0) {

 setFcState((prev) => (prev ? { ...prev, rateLimited: true } : null));

 await new Promise((res) => setTimeout(res, 15000));

 } else {

 failed.push({ question: q, error: "Check failed" });

 break;

 }

}

}

if (!done && !failed.some((f) => f.question.id === q.id)) {

 failed.push({ question: q, error: "Check failed" });

}

setFcState((prev) => (prev ? { ...prev, results: [...results], failed: [...failed] } : null));

if (i < toCheck.length - 1) await new Promise((res) => setTimeout(res, 2000));

}

```

```

 setFcState((prev) => (prev ? { ...prev, running: false, rateLimited: false } : null));
};

const handleRetryFailed = async () => {
 if (!selectedGameId || !fcState || fcState.failed.length === 0) return;
 const toRetry = [...fcState.failed];
 setFcState((prev) =>
 prev ? { ...prev, running: true, failed: [], current: 0, total: toRetry.length, rateLimited: false
 } : null,
);
 const newFailed: FcFailed[] = [];
 const newResults = [...fcState.results];
 for (let i = 0; i < toRetry.length; i++) {
 const entry = toRetry[i]!;
 setFcState((prev) => (prev ? { ...prev, current: i + 1 } : null));
 try {
 const r = await factCheckMutation.mutateAsync({ gameId: selectedGameId,
 questionId: entry.question.id });
 newResults.push({
 question: entry.question,
 verdict: r.verdict,
 confidence: r.confidence,
 explanation: r.explanation,
 correctAnswerIfWrong: r.correctAnswerIfWrong ?? null,
 });
 } catch {
 newFailed.push(entry);
 }
 }
}

```

```

 }

 setFcState((prev) => (prev ? { ...prev, results: [...newResults], failed: [...newFailed] } :
null));

 if (i < toRetry.length - 1) await new Promise((res) => setTimeout(res, 2000));

 }

 setFcState((prev) => (prev ? { ...prev, running: false } : null));

};

const getSourceBadge = (q: Question) => {

 if (q.aiGenerated) return { label: "AI Generated", cls: "bg-purple-500/15 text-purple-400
border-purple-500/30" };

 if (q.source === "opentdb") return { label: "OpenTDB", cls: "bg-blue-500/15 text-blue-400
border-blue-500/30" };

 return { label: "Manual", cls: "bg-green-500/15 text-green-400 border-green-500/30" };

};

const FILTERS: { key: typeof filter; label: string }[] = [

 { key: "all", label: "All" },

 { key: "unverified", label: "Unverified" },

 { key: "opentdb", label: "OpenTDB" },

 { key: "ai", label: "AI Generated" },

 { key: "manual", label: "Manual" },

];

const filterCount = (key: typeof filter) => rawQuestions.filter((q) => {

 switch (key) {

 case "unverified": return !q.verifiedByAdmin;

 }

});

```

```

 case "opentdb": return q.source === "opentdb";
 case "ai": return !!q.aiGenerated;
 case "manual": return q.source === "manual" || (!q.source && !q.aiGenerated);
 default: return true;
 }
}).length;

return (
 <div className="space-y-5">
 <div>
 <h2 className="text-xl font-bold tracking-tight">Review Questions</h2>
 <p className="text-muted-foreground text-sm mt-1">
 Verify, edit, and manage questions across all games.
 </p>
 </div>

 {games.length === 0 ? (
 <Card className="border-dashed border-primary/30 bg-card/40">
 <CardContent className="py-14 text-center space-y-2">
 <Gamepad2 className="mx-auto h-10 w-10 text-primary/40" />
 <p className="font-semibold">No games yet</p>
 <p className="text-sm text-muted-foreground">Create a game first to review its
questions.</p>
 </CardContent>
 </Card>
) : (

```

```

<>

{ /* Game + sort selectors */}

<div className="flex items-end gap-3 flex-wrap">

 <div className="space-y-1.5 flex-1 min-w-[200px]">

 <Label>Game</Label>

 <Select

 value={selectedGameId !== null ? String(selectedGameId) : ""}

 onChange={v => { setSelectedGameId(Number(v)); setSelected(new Set());

 }}

 >

 <SelectTrigger className="h-10">

 <SelectValue placeholder="Select a game..." />

 </SelectTrigger>

 <SelectContent>

 {games.map((g) => (

 <SelectItem key={g.id} value={String(g.id)}>

 {g.topic}

 <span className="ml-2 text-muted-foreground text-xs
capitalize">{g.status}

 </SelectItem>

))}

 </SelectContent>

 </Select>

 </div>

 <div className="space-y-1.5">

 <Label>Sort by</Label>

 <Select value={sortBy} onChange={v => setSortBy(v as typeof sortBy)}>

```

```

<SelectTrigger className="h-10 w-44">
 <SelectValue />
</SelectTrigger>
<SelectContent>
 <SelectItem value="order">Question order</SelectItem>
 <SelectItem value="dateAdded">Date added (newest first)</SelectItem>
 <SelectItem value="type">Question type</SelectItem>
 <SelectItem value="verified">Verification status</SelectItem>
</SelectContent>
</Select>
</div>
</div>

```

```

{/* Verification stats */}
{selectedGameId !== null && rawQuestions.length > 0 && (
 <div className="flex items-center justify-between rounded-lg border border-card-
border bg-card/50 px-4 py-2.5">
 <div className="flex items-center gap-4 text-sm flex-wrap">
 {rawQuestions.length} total

 <ShieldCheck className="h-4 w-4" />
 {verifiedCount} verified

 {unverifiedCount > 0 && (

 <AlertTriangle className="h-3.5 w-3.5" />

```



```

 {unverifiedCount} unverified

)}
 </div>

 <div className="flex items-center gap-2 shrink-0">

 <div className="h-2 w-20 bg-muted/30 rounded-full overflow-hidden hidden sm:block">

 <div

 className="h-full bg-secondary rounded-full transition-all"

 style={{ width: `${rawQuestions.length ? (verifiedCount / rawQuestions.length) *
100 : 0}%` }}

 />

 </div>

 <Button

 size="sm"

 variant="ghost"

 className="h-7 px-2 text-xs gap-1"

 onClick={() => setGenMoreOpen(true)}

 disabled={fcState?.running}

 >

 <Sparkles className="h-3 w-3" /> Generate More

 </Button>

 <Button

 size="sm"

 variant="ghost"

 className="h-7 px-2 text-xs gap-1"

 onClick={handleFactCheckAll}

```

```

 disabled={fcState?.running || rawQuestions.length === 0}
 >
 <FlaskConical className="h-3 w-3" /> Fact Check All
 </Button>
 </div>
 </div>
)}

```

```

{ /* Filter tabs */
<div className="flex gap-2 flex-wrap">
 {FILTERS.map((f) => (
 <button
 key={f.key}
 onClick={() => { setFilter(f.key); setSelected(new Set()); }}
 className={` rounded-full px-3 py-1 text-xs font-medium transition-colors border
$ {
 filter === f.key
 ? "bg-primary text-primary-foreground border-primary"
 : "bg-card/60 text-muted-foreground border-card-border hover:border-muted-
foreground/50"
 } `}
 >
 {f.label} ({filterCount(f.key)})
 </button>
)})
</div>

```

```

 { /* Bulk action bar */
 <AnimatePresence>
 {selected.size > 0 && (
 <motion.div
 initial={{ opacity: 0, y: -8 }}
 animate={{ opacity: 1, y: 0 }}
 exit={{ opacity: 0, y: -8 }}
 className="flex items-center gap-3 rounded-lg border border-primary/30 bg-
primary/5 px-4 py-2.5"
 >
 {selected.size} selected
 <Button size="sm" variant="outline" onClick={handleBulkVerify}
disabled={updateQuestion.isPending}>
 <ShieldCheck className="mr-1 h-3.5 w-3.5" /> Verify All
 </Button>
 <Button
 size="sm"
 variant="ghost"
 className="text-destructive hover:text-destructive"
 onClick={handleBulkDelete}
 disabled={deleteQuestion.isPending}
 >
 <Trash2 className="mr-1 h-3.5 w-3.5" /> Delete Selected
 </Button>
 <Button size="sm" variant="ghost" onClick={() => setSelected(new Set())}>
 <X className="h-3.5 w-3.5" />
 </Button>

```

```

 </motion.div>

)}

</AnimatePresence>

{ /* Fact check progress / results */}

{fcState && (

 <div className="rounded-lg border border-card-border bg-card/60 p-4 space-y-3">

 {fcState.running ? (

 <div className="flex items-center gap-3">

 <Loader2 className="h-4 w-4 animate-spin text-primary shrink-0" />

 <div className="flex-1">

 <p className="text-sm font-medium">

 {fcState.rateLimited

 ? "Rate limited — waiting 15 seconds..."

 : `Checking question ${fcState.current} of ${fcState.total}...`}

 </p>

 <div className="mt-1.5 h-1.5 w-40 bg-muted/30 rounded-full overflow-hidden">

 <div

 className="h-full bg-primary rounded-full transition-all"

 style={{ width: `${fcState.total ? (fcState.current / fcState.total) * 100 : 0}%` }}

 />

 </div>

 </div>

 </div>

) : (

 <div className="space-y-3">

```

```

<div className="flex items-center justify-between">

 <p className="text-sm font-semibold">Fact Check Complete</p>

 <Button size="sm" variant="ghost" className="h-7 w-7 p-0" onClick={() =>
setFcState(null)}>

 <X className="h-3.5 w-3.5" />

 </Button>

</div>

<div className="flex gap-4 text-xs flex-wrap">

 <CheckCircle2 className="h-3.5 w-3.5" />

 {fcState.results.filter((r) => r.verdict === "CORRECT").length} correct

 <AlertTriangle className="h-3.5 w-3.5" />

 {fcState.results.filter((r) => r.verdict === "UNCERTAIN").length} uncertain

 <X className="h-3.5 w-3.5" />

 {fcState.results.filter((r) => r.verdict === "INCORRECT").length} incorrect

 {fcState.failed.length > 0 && (

 {fcState.failed.length}
failed

)}

</div>

{fcState.results.filter((r) => r.verdict !== "CORRECT").length > 0 && (

 <div className="space-y-2 max-h-64 overflow-y-auto">

```

```

{fcState.results
 .filter((r) => r.verdict !== "CORRECT")
 .map(({ question, verdict, confidence, explanation, correctAnswerIfWrong }) => (
 <div
 key={question.id}
 className={`rounded-md border px-3 py-2 text-xs space-y-1 ${
 verdict === "INCORRECT"
 ? "border-destructive/30 bg-destructive/5"
 : "border-yellow-400/20 bg-yellow-400/5"
 }`}
 >
 <div className="flex items-start gap-2">
 {verdict === "INCORRECT" ? (
 <X className="h-3.5 w-3.5 shrink-0 text-destructive mt-0.5" />
) : (
 <AlertTriangle className="h-3.5 w-3.5 shrink-0 text-yellow-400 mt-0.5" />
)}
 <div className="flex-1 min-w-0">
 <p className="font-medium leading-snug truncate">{question.questionText}</p>
 <p className="text-muted-foreground mt-0.5">{explanation}</p>
 {correctAnswerIfWrong && (
 <p className="mt-1">
 Suggested answer:{" "}
 {correctAnswerIfWrong}
 </p>
)}
 </div>
 </div>
)
)
)

```

```

 }}
 <span className="inline-block mt-0.5 text-[10px] text-muted-foreground
uppercase tracking-wide">
 Confidence: {confidence}

</div>
</div>
</div>
)}
</div>
)}
{fcState.failed.length > 0 && (
 <Button size="sm" variant="outline" onClick={handleRetryFailed}>
 <RefreshCw className="mr-1.5 h-3.5 w-3.5" />
 Retry {fcState.failed.length} failed {fcState.failed.length === 1 ? "question" :
"questions"}
 </Button>
)}
</div>
)}
</div>
)}

{/* Question list */}
{selectedGameId === null ? (
 <p className="text-sm text-muted-foreground">Select a game above.</p>
) : displayList.length === 0 ? (

```

```

<Card className="border-dashed border-card-border bg-card/30">
 <CardContent className="py-10 text-center text-muted-foreground text-sm">
 No questions match this filter.
 </CardContent>
</Card>
): (
 <div className="space-y-2">
 { /* Select-all row */ }
 <div className="flex items-center gap-2 px-1">
 <button
 onClick={toggleSelectAll}
 className="text-muted-foreground hover:text-foreground transition-colors"
 >
 {allSelected
 ? <CheckSquare className="h-4 w-4 text-primary" />
 : <Square className="h-4 w-4" />}
 </button>

 {allSelected ? "Deselect all" : `Select all ${displayList.length}`}

 </div>

 {displayList.map((q) => {
 const { icon, label: typeLabel, color } = TYPE_META[q.questionType as
QuestionType] ?? TYPE_META.write_in;
 const src = getSourceBadge(q);

```



```

const isSelected = selected.has(q.id);

return (
 <motion.div
 key={q.id}
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className={`rounded-lg border bg-card/60 p-3.5 transition-colors ${
 isSelected ? "border-primary/40 bg-primary/5" : "border-card-border"
 }`}
 >
 <div className="flex items-start gap-3">
 {/* Checkbox */}
 <button
 onClick={() => toggleSelect(q.id)}
 className="mt-0.5 shrink-0 text-muted-foreground hover:text-primary
transition-colors"
 >
 {isSelected
 ? <CheckSquare className="h-4 w-4 text-primary" />
 : <Square className="h-4 w-4" />}
 </button>

 {/* Type icon */}
 <div className={`mt-0.5 shrink-0 ${color}`}>
 <Icon className="h-4 w-4" />
 </div>
 </div>
 </div>

```

```

 { /* Content */

 <div className="flex-1 min-w-0 space-y-2">

 <p className="font-medium leading-snug text-sm">{q.questionText}</p>

 <div className="flex flex-wrap items-center gap-2">

 <Badge variant="outline" className="uppercase text-
[10px]">{typeLabel}</Badge>

 <Badge variant="outline" className={` text-[10px]
${src.cls} `}>{src.label}</Badge>

 <span className="text-xs text-accent font-semibold flex items-center gap-
0.5">

 <Star className="h-3 w-3" /> {q.points} pts

 {q.verifiedByAdmin ? (

 <span className="flex items-center gap-1 text-xs text-secondary font-
medium">

 <ShieldCheck className="h-3.5 w-3.5" /> Verified

) : (

 <span className="flex items-center gap-1 text-xs text-yellow-400 font-
medium">

 <AlertTriangle className="h-3.5 w-3.5" /> Unverified

)}

 </div>

 {q.questionType !== "matching" && (

 <p className="text-xs text-muted-foreground">

```

Answer: <span className="text-secondary font-medium">{q.correctAnswer}</span>

</p>

)}</div>

{q.source && q.source !== "manual" && q.source !== "opentdb" && (

<p className="text-xs text-muted-foreground">Source: {q.source}</p>

)}</div>

{q.factCheckUrl && (

<a

href={q.factCheckUrl}

target="\_blank"

rel="noopener noreferrer"

className="flex items-center gap-1 text-xs text-primary hover:underline w-

fit"

>

<Link className="h-3 w-3" /> Fact-check link

</a>

)}</div>

</div>

{/\* Actions \*/}

<div className="flex flex-col sm:flex-row gap-1 shrink-0">

<Button

size="sm"

variant={q.verifiedByAdmin ? "ghost" : "outline"}

className={

q.verifiedByAdmin

```

 ? "h-8 px-2 text-yellow-400 hover:text-yellow-400"
 : "h-8 px-2 text-secondary border-secondary/40"
 }
 onClick={() => handleVerify(q, !q.verifiedByAdmin)}
 disabled={updateQuestion.isPending}
>
 {q.verifiedByAdmin ? (
 <<AlertTriangle className="h-3 w-3 mr-1" />Unverify</>
) : (
 <<ShieldCheck className="h-3 w-3 mr-1" />Verify</>
)}
</Button>
{q.aiGenerated && (
 <Button
 variant="ghost"
 size="sm"
 className="h-8 px-2 text-xs text-purple-400 hover:text-purple-300"
 onClick={() => {
 setRegenQ(q);
 setRegenDiff("same");
 setRegenType("same");
 setRegenPreview(null);
 setRegenError(null);
 }}
 >
 <RefreshCw className="mr-1 h-3 w-3" />Regen

```

```

</Button>

)}

<Button
 variant="ghost"
 size="sm"
 className="h-8 px-2 text-xs text-blue-400 hover:text-blue-300"
 onClick={() => handleRunEnhance(q)}
 disabled={enhanceMutation.isPending && enhQ?.id === q.id}
>

 <Wand2 className="mr-1 h-3 w-3" />Enhance
</Button>

<Button
 variant="ghost"
 size="icon"
 className="h-8 w-8"
 onClick={() => { setEditingQuestion(q); setEditDialogOpen(true); }}
>

 <Pencil className="h-3.5 w-3.5" />
</Button>

<Button
 variant="ghost"
 size="icon"
 className="h-8 w-8 text-destructive hover:text-destructive"
 onClick={() => handleDelete(q.id)}
 disabled={deleteQuestion.isPending}
>

```

```

 <Trash2 className="h-3.5 w-3.5" />

 </Button>

 </div>

 </div>

</motion.div>

);

}}

</div>

})

```

```

{/* — Regenerate dialog — */}

<Dialog open={regenQ !== null} onChange={({open) => { if (!open) {
setRegenQ(null); setRegenPreview(null); setRegenError(null); } }}>

 <DialogContent className="sm:max-w-md">

 <DialogHeader>

 <DialogTitle className="flex items-center gap-2">

 <RefreshCw className="h-4 w-4 text-purple-400" /> Regenerate Question

 </DialogTitle>

 </DialogHeader>

 {regenQ && (

 <div className="space-y-4">

 <div className="rounded-md border border-card-border bg-card/50 p-3">

 <p className="text-xs text-muted-foreground mb-1">Current question</p>

 <p className="text-sm font-medium leading-snug">{regenQ.questionText}</p>

 </div>

```

```

 { /* Difficulty */ }

 <div className="space-y-2">

 <p className="text-sm font-medium">Difficulty</p>

 <div className="flex flex-wrap gap-2">

 {[["same", "easy", "medium", "hard"] as const].map((d) => (

 <button

 key={d}

 onClick={() => setRegenDiff(d)}

 className={`rounded-full px-3 py-1 text-xs font-medium border transition-
colors capitalize ${

 regenDiff === d

 ? "bg-primary text-primary-foreground border-primary"

 : "bg-card/60 text-muted-foreground border-card-border hover:border-
muted-foreground/50"

 }`}

 >

 {d === "same" ? "Same as current" : d}

 </button>

)]}

 </div>

 </div>

```

```

 { /* Question type */ }

 <div className="space-y-2">

 <p className="text-sm font-medium">Question type</p>

 <Select value={regenType} onChange={setRegenType}>

 <SelectTrigger className="h-9">

```

```

 <SelectValue />
 </SelectTrigger>
 <SelectContent>
 <SelectItem value="same">Same as current</SelectItem>
 <SelectItem value="multiple_choice">Multiple Choice</SelectItem>
 <SelectItem value="true_false">True / False</SelectItem>
 <SelectItem value="write_in">Write-in</SelectItem>
 </SelectContent>
</Select>
</div>

```

```

 { /* Error */
 {regenError && (
 <div className="flex items-center gap-2 rounded-md bg-destructive/10 border
border-destructive/20 px-3 py-2 text-xs text-destructive">
 <AlertTriangle className="h-3.5 w-3.5 shrink-0" /> {regenError}
 </div>
)}
)}

```

```

 { /* Preview */
 {regenPreview && (
 <div className="rounded-md border border-primary/30 bg-primary/5 p-3 space-
y-2">
 <p className="text-xs text-primary font-medium">New question preview</p>
 <p className="text-sm font-medium leading-
snug">{regenPreview.questionText}</p>
 <p className="text-xs text-muted-foreground">

```



```
 Answer: <span className="text-secondary font-
medium">{regenPreview.correctAnswer}
```

```
</p>
```

```
{regenPreview.options && regenPreview.options.length > 0 && (
```

```
 <p className="text-xs text-muted-foreground">
```

```
 Options: {regenPreview.options.join(" · ")}</p>
```

```
</p>
```

```
)}
```

```
 <p className="text-xs text-muted-foreground">Source:
{regenPreview.source}</p>
```

```
</div>
```

```
)}
```

```
{/* Action buttons */}
```

```
<div className="flex gap-2 flex-wrap">
```

```
 <Button
```

```
 className="flex-1"
```

```
 onClick={handleRunRegen}
```

```
 disabled={regenLoading}
```

```
>
```

```
 {regenLoading ? (
```

```
 <><Loader2 className="mr-2 h-4 w-4 animate-spin" />Generating...</>
```

```
) : regenPreview ? (
```

```
 <><RefreshCw className="mr-2 h-4 w-4" />Try Again</>
```

```
) : (
```

```
 <><Sparkles className="mr-2 h-4 w-4" />Generate</>
```

```
)}
```

```

</Button>

{regenPreview && (
 <Button
 variant="outline"
 className="flex-1 border-secondary/40 text-secondary"
 onClick={handleAcceptRegen}
 disabled={updateQuestion.isPending}
 >
 <CheckCircle2 className="mr-2 h-4 w-4" />Accept
 </Button>
)}
</div>
</div>
)}
</DialogContent>
</Dialog>

```

```

{/* — Enhance dialog — */}

<Dialog open={enhQ !== null} onOpenChange={(open) => { if (!open) { setEnhQ(null);
setEnhResult(null); setEnhError(null); } }}>

 <DialogContent className="sm:max-w-xl max-h-[90dvh] overflow-y-auto">

 <DialogHeader>

 <DialogTitle className="flex items-center gap-2">

 <Wand2 className="h-4 w-4 text-blue-400" /> Enhance Question

 </DialogTitle>

 </DialogHeader>

```

```

{enhQ && (
 <div className="space-y-4">
 {/* Loading */}
 {enhLoading && (
 <div className="flex items-center gap-3 py-4">
 <Loader2 className="h-5 w-5 animate-spin text-primary" />
 <p className="text-sm text-muted-foreground">Analyzing with Gemini...</p>
 </div>
)}

 {/* Error */}
 {enhError && (
 <div className="flex items-center gap-2 rounded-md bg-destructive/10 border
border-destructive/20 px-3 py-2 text-xs text-destructive">
 <AlertTriangle className="h-3.5 w-3.5 shrink-0" /> {enhError}
 </div>
)}

 {enhResult && (
 <>
 {/* Fact check banner */}
 <div
 className={`flex items-start gap-2 rounded-md border px-3 py-2.5 ${
 enhResult.factCheckResult === "VERIFIED"
 ? "bg-secondary/10 border-secondary/30 text-secondary"
 : enhResult.factCheckResult === "LIKELY_INCORRECT"

```

```

 ? "bg-destructive/10 border-destructive/30 text-destructive"
 : "bg-yellow-400/10 border-yellow-400/20 text-yellow-400"
 }` }
>
{enhResult.factCheckResult === "VERIFIED" ? (
 <ShieldCheck className="h-4 w-4 shrink-0 mt-0.5" />
) : enhResult.factCheckResult === "LIKELY_INCORRECT" ? (
 <AlertTriangle className="h-4 w-4 shrink-0 mt-0.5" />
) : (
 <HelpCircle className="h-4 w-4 shrink-0 mt-0.5" />
)}
<div>
 <p className="text-sm font-semibold">
 {enhResult.factCheckResult === "VERIFIED"
 ? "Fact Verified"
 : enhResult.factCheckResult === "LIKELY_INCORRECT"
 ? "Likely Incorrect"
 : "Uncertain"}
 </p>
 {enhResult.factCheckNotes && (
 <p className="text-xs mt-0.5 opacity-85">{enhResult.factCheckNotes}</p>
)}
</div>
</div>

{/* Side-by-side comparison */}

```

```

<div className="grid grid-cols-1 sm:grid-cols-2 gap-3">

 {/* Original */}

 <div className="space-y-2">

 <p className="text-xs font-semibold text-muted-foreground uppercase tracking-wide">Original</p>

 <div className="rounded-md border border-card-border bg-card/50 p-3 text-sm space-y-1.5">

 <p className="font-medium leading-snug">{enhQ.questionText}</p>

 {enhQ.source && <p className="text-xs text-muted-foreground">Source: {enhQ.source}</p>}

 </div>

 </div>

 {/* Suggested */}

 <div className="space-y-2">

 <p className="text-xs font-semibold text-primary uppercase tracking-wide">Suggested</p>

 <div className="rounded-md border border-primary/30 bg-primary/5 p-3 text-sm space-y-1.5">

 <p className="font-medium leading-snug">{enhResult.improvedQuestionText}</p>

 {enhResult.suggestedSource && (

 <p className="text-xs text-muted-foreground">Source: {enhResult.suggestedSource}</p>

)}

 </div>

 </div>

</div>

```

```

{enhResult.suggestions && (
 <div className="rounded-md bg-muted/20 border border-card-border px-3
py-2 text-xs text-muted-foreground">
 Tips:
{enhResult.suggestions}
 </div>
)}

```

```

{/* Checkboxes */}
<div className="space-y-2">
 <p className="text-sm font-medium">Apply improvements</p>
 <label className="flex items-start gap-2.5 cursor-pointer group">
 <input
 type="checkbox"
 className="mt-0.5 accent-primary"
 checked={enhAcceptText}
 onChange={(e) => setEnhAcceptText(e.target.checked)}
 />
 <span className="text-sm group-hover:text-foreground text-muted-
foreground transition-colors">
 Use improved question text

 </label>
 {enhResult.improvedOptions && enhResult.improvedOptions.length > 0 && (
 <label className="flex items-start gap-2.5 cursor-pointer group">
 <input
 type="checkbox"

```

```

 className="mt-0.5 accent-primary"
 checked={enhAcceptOptions}
 onChange={(e) => setEnhAcceptOptions(e.target.checked)}
 />

 <span className="text-sm group-hover:text-foreground text-muted-
foreground transition-colors">

 Use improved answer options

</label>

)}

{enhResult.suggestedSource && (
 <label className="flex items-start gap-2.5 cursor-pointer group">
 <input
 type="checkbox"
 className="mt-0.5 accent-primary"
 checked={enhAcceptSource}
 onChange={(e) => setEnhAcceptSource(e.target.checked)}
 />

 <span className="text-sm group-hover:text-foreground text-muted-
foreground transition-colors">

 Use suggested source

 </label>

)}
</div>

{/* Buttons */}

```

```

 <div className="flex gap-2">
 <Button
 onClick={handleApplyEnhancements}
 disabled={updateQuestion.isPending || (!enhAcceptText &&
!enhAcceptOptions && !enhAcceptSource)}
 className="flex-1"
 >
 Apply Selected Changes
 </Button>
 <Button variant="ghost" onClick={() => { setEnhQ(null); setEnhResult(null); }}>
 Dismiss
 </Button>
 </div>
 </>
)}
</div>
)}
</DialogContent>
</Dialog>

```

```

{/* — Generate More dialog — */}

```

```

<Dialog open={genMoreOpen} onOpenChange={setGenMoreOpen}>

```

```

 <DialogContent className="sm:max-w-sm">

```

```

 <DialogHeader>

```

```

 <DialogTitle className="flex items-center gap-2">

```

```

 <Sparkles className="h-4 w-4 text-primary" /> Generate More Questions

```



```

</DialogTitle>

</DialogHeader>

<div className="space-y-4">

 {selectedGameId !== null && (

 <p className="text-sm text-muted-foreground">

 Adding to:

 {games.find((g) => g.id === selectedGameId)?.topic}

 </p>

)}

 <div className="space-y-1.5">

 <Label>Number of questions (1–10)</Label>

 <Input

 type="number"

 min={1}

 max={10}

 value={genMoreCount}

 onChange={({e} => setGenMoreCount(Math.max(1, Math.min(10,
Number(e.target.value))))}

 className="h-9"

 />

 </div>

 <div className="space-y-1.5">

 <Label>Difficulty</Label>

 <Select value={genMoreDiff} onChange={({v} => setGenMoreDiff(v as typeof
genMoreDiff))>

 <SelectTrigger className="h-9">

```

```

 <SelectValue />
 </SelectTrigger>
 <SelectContent>
 <SelectItem value="same">Same as game setting</SelectItem>
 <SelectItem value="easy">Easy</SelectItem>
 <SelectItem value="medium">Medium</SelectItem>
 <SelectItem value="hard">Hard</SelectItem>
 </SelectContent>
</Select>
</div>

<label className="flex items-center gap-2.5 cursor-pointer">
 <input
 type="checkbox"
 className="accent-primary"
 checked={genMoreAvoid}
 onChange={(e) => setGenMoreAvoid(e.target.checked)}
 />

 Avoid duplicating existing
questions
</label>

<Button
 className="w-full"
 onClick={handleGenerateMore}
 disabled={generateMore.isPending}
>
 {generateMore.isPending ? (

```

```

 <><Loader2 className="mr-2 h-4 w-4 animate-spin" />Generating...</>

) : (
 <><Sparkles className="mr-2 h-4 w-4" />Generate {genMoreCount}
Questions</>

 })

</Button>

{generateMore.isPending && (
 <p className="text-xs text-center text-muted-foreground">This may take 10–20
seconds...</p>

)}

</div>

</DialogContent>

</Dialog>

{/* Edit dialog */}

<Dialog

 open={editDialogOpen}

 onOpenChange={(open) => { setEditDialogOpen(open); if (!open)
setEditingQuestion(null); }}

 >

 <DialogContent className="max-h-[90dvh] overflow-y-auto sm:max-w-lg">

 <DialogHeader>

 <DialogTitle>Edit Question</DialogTitle>

 </DialogHeader>

 {editingQuestion && (

 <QuestionForm

 key={editingQuestion.id}

```

```

 initial={formFromQuestion(editingQuestion)}
 pending={updateQuestion.isPending}
 submitLabel="Save Changes"
 onSubmit={({form}) => {
 updateQuestion.mutate(
 { questionId: editingQuestion.id, data: buildPayload(form,
editingQuestion.orderIndex) },
 {
 onSuccess: () => {
 invalidate();
 setEditDialogOpen(false);
 setEditingQuestion(null);
 toast({ title: "Question updated" });
 },
 onError: () => toast({ variant: "destructive", title: "Update failed" }),
 },
);
 }}
 />
)}
</DialogContent>
</Dialog>
</>
)}
</div>
);

```

```
}
```

```
// — Settings section —————
```

---

```
const REQUIRE_VERIFY_KEY = "trivia-require-verify";
```

```
function SettingsSection() {
```

```
 const { toast } = useToast();
```

```
 const [triviaCode, setTriviaCode] = useState("");
```

```
 const [adminCode, setAdminCode] = useState("");
```

```
 const [showTrivia, setShowTrivia] = useState(false);
```

```
 const [showAdmin, setShowAdmin] = useState(false);
```

```
 const [currentTrivia, setCurrentTrivia] = useState("");
```

```
 const [currentAdmin, setCurrentAdmin] = useState("");
```

```
 const [loading, setLoading] = useState(true);
```

```
 const [saving, setSaving] = useState(false);
```

```
 const [requireVerify, setRequireVerify] = useState<boolean>(() => {
```

```
 try { return localStorage.getItem(REQUIRE_VERIFY_KEY) !== "false"; } catch { return true; }
```

```
 });
```

```
 useEffect(() => {
```

```
 fetch("/api/settings", { credentials: "include" })
```

```
 .then((r) => r.json())
```

```
 .then((data) => {
```

```
 setCurrentTrivia(data.triviaAccessCode ?? "");
```

```
 setCurrentAdmin(data.adminAccessCode ?? "");
```

```
 setTriviaCode(data.triviaAccessCode ?? "");
 setAdminCode(data.adminAccessCode ?? "");
 setLoading(false);
 })
 .catch(() => {
 toast({ variant: "destructive", title: "Could not load settings" });
 setLoading(false);
 });
}, []);
```

```
const handleSave = async (e: React.FormEvent) => {
 e.preventDefault();
 const t = triviaCode.trim();
 const a = adminCode.trim();

 if (t.length < 4) { toast({ variant: "destructive", title: "Trivia code must be at least 4 characters" }); return; }

 if (a.length < 4) { toast({ variant: "destructive", title: "Admin code must be at least 4 characters" }); return; }

 if (t === a) { toast({ variant: "destructive", title: "Trivia and admin codes must be different" }); return; }

 setSaving(true);

 try {
 const res = await fetch("/api/settings", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 credentials: "include",
 body: JSON.stringify({ triviaAccessCode: t, adminAccessCode: a }),
 });
```

```

});

if (!res.ok) {
 const data = await res.json();
 toast({ variant: "destructive", title: data.error ?? "Save failed" });
 return;
}

const updated = await res.json();
setCurrentTrivia(updated.triviaAccessCode);
setCurrentAdmin(updated.adminAccessCode);
toast({ title: "Settings saved" });
} catch {
 toast({ variant: "destructive", title: "Network error" });
} finally {
 setSaving(false);
}
};

if (loading) {
 return (
 <div className="space-y-4 max-w-lg">
 <div className="h-8 w-48 bg-muted/50 animate-pulse rounded" />
 <div className="h-32 bg-muted/30 animate-pulse rounded-lg" />
 </div>
);
}

```

```
const unchanged = triviaCode.trim() === currentTrivia && adminCode.trim() ===
currentAdmin;
```

```
return (
```

```
<div className="space-y-6 max-w-lg">
```

```
<div>
```

```
<h2 className="text-xl font-bold tracking-tight">Settings</h2>
```

```
<p className="text-muted-foreground text-sm mt-1">
```

```
 Change the access codes players and admins use to enter the app.
```

```
</p>
```

```
</div>
```

```
<form onSubmit={handleSave} className="space-y-5">
```

```
<Card className="border-card-border bg-card/60">
```

```
<CardContent className="p-5 space-y-4">
```

```
<div className="space-y-2">
```

```
<Label htmlFor="trivia-code">
```

```
 Player Access Code
```

```

```

```
 (shared with players)
```

```

```

```
</Label>
```

```
<div className="relative">
```

```
<Input
```

```
 id="trivia-code"
```

```
 type={showTrivia ? "text" : "password"}
```

```
 value={triviaCode}
```



```

 onChange={(e) => setTriviaCode(e.target.value.toUpperCase())}

 className="pr-10 uppercase tracking-widest font-mono"
 />

 <button

 type="button"

 onClick={() => setShowTrivia((v) => !v)}

 className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground
hover:text-foreground transition-colors"

 >

 {showTrivia ? <EyeOff className="h-4 w-4" /> : <Eye className="h-4 w-4" />}

 </button>

 </div>

</div>

<Separator />

<div className="space-y-2">

 <Label htmlFor="admin-code">

 Admin Code

 (hosts only — keep private)

 </Label>

 <div className="relative">

 <Input

 id="admin-code"

 type={showAdmin ? "text" : "password"}

 value={adminCode}

```

```

 onChange={(e) => setAdminCode(e.target.value.toUpperCase())}

 className="pr-10 uppercase tracking-widest font-mono"
 />

 <button

 type="button"

 onClick={() => setShowAdmin((v) => !v)}

 className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground
hover:text-foreground transition-colors"

 >

 {showAdmin ? <EyeOff className="h-4 w-4" /> : <Eye className="h-4 w-4" />}

 </button>

 </div>

</div>

</CardContent>

</Card>

<Button type="submit" className="font-bold" disabled={unchanged || saving}>

 {saving ? "Saving..." : "Save Changes"}

</Button>

{unchanged && (

 <p className="text-xs text-muted-foreground">No changes to save.</p>

)}

</form>

{/* — Game Start Rules — */}

<div className="space-y-3">

 <div>

```

```
<h3 className="font-semibold text-base">Game Start Rules</h3>
```

```
<p className="text-muted-foreground text-sm mt-0.5">
```

Controls what happens when you click Go Live on a game.

```
</p>
```

```
</div>
```

```
<Card className="border-card-border bg-card/60">
```

```
<CardContent className="p-5">
```

```
<div className="flex items-start justify-between gap-4">
```

```
<div className="space-y-1">
```

```
<p className="font-medium text-sm">Require all questions verified</p>
```

```
<p className="text-xs text-muted-foreground max-w-sm">
```

When enabled, going live with unverified questions shows a warning and requires confirmation. Disable only if you trust all questions are accurate.

```
</p>
```

```
</div>
```

```
<Switch
```

```
checked={requireVerify}
```

```
onCheckedChange={v => {
```

```
 setRequireVerify(v);
```

```
 try { localStorage.setItem(REQUIRE_VERIFY_KEY, String(v)); } catch { /* ignore */ }
```

```
 toast({ title: v ? "Verification check enabled" : "Verification check disabled" });
```

```
}}
```

```
/>
```

```
</div>
```

```
{!requireVerify && (
```

```
 <div className="mt-3 flex items-center gap-2 rounded-md bg-yellow-400/10
border border-yellow-400/20 px-3 py-2 text-xs text-yellow-400">
```

```
 <AlertTriangle className="h-3.5 w-3.5 shrink-0" />
```

```
 Verification check is off — games can go live with unverified questions.
```

```
 </div>
```

```
)}
```

```
</CardContent>
```

```
</Card>
```

```
</div>
```

```
</div>
```

```
);
```

```
}
```

```
// ——— Results section —————
```

```
type LeaderboardEntry = {
```

```
 rank: number;
```

```
 userId: number;
```

```
 userName: string;
```

```
 totalScore: number;
```

```
 correctCount: number;
```

```
 totalAnswered: number;
```

```
};
```

```
type GameResultsData = {
```

```
 game: Game;
```

```
participants: LeaderboardEntry[];
totalQuestions: number;
};
```

```
type QuestionStat = {
 id: number;
 questionText: string;
 questionType: string;
 points: number;
 orderIndex: number;
 totalAnswered: number;
 correctCount: number;
 percentCorrect: number | null;
};
```

```
function ResultsSection({ games }: { games: Game[] }) {
 const [selectedGameId, setSelectedGameId] = useState<number | null>(null);
 const [showQuestions, setShowQuestions] = useState(false);

 const completedGames = games.filter((g) => g.status === "completed");

 const { data: resultsData, isLoading: loadingResults } = useQuery<GameResultsData>({
 queryKey: ["admin-results", selectedGameId],
 queryFn: async () => {
 const res = await fetch(`/api/games/${selectedGameId}/results`);
 if (!res.ok) throw new Error("Failed to fetch results");
 }
 });
```

```
 return res.json();
 },
 enabled: selectedGameId !== null,
});
```

```
const { data: questionStats = [], isLoading: loadingStats } = useQuery<QuestionStat[]>({
 queryKey: ["admin-question-stats", selectedGameId],
 queryFn: async () => {
 const res = await fetch(`/api/games/${selectedGameId}/questions/stats`);
 if (!res.ok) throw new Error("Failed to fetch stats");
 return res.json();
 },
 enabled: selectedGameId !== null && showQuestions,
});
```

```
const handleExport = () => {
 if (!selectedGameId) return;
 window.open(`/api/games/${selectedGameId}/results/export.csv`, "_blank");
};
```

```
const medals = ["🥇", "🥈", "🥉"];
```

```
return (
 <div className="space-y-6">
 <div>
 <h2 className="text-2xl font-bold tracking-tight">Game Results</h2>
```

```
<p className="text-muted-foreground text-sm mt-1">
```

```
 View leaderboards and question analytics for completed games.
```

```
</p>
```

```
</div>
```

```
{/* Game picker */}
```

```
<Card>
```

```
 <CardContent className="pt-5">
```

```
 {completedGames.length === 0 ? (
```

```
 <p className="text-muted-foreground text-sm text-center py-4">
```

```
 No completed games yet. Finish a game to see results here.
```

```
 </p>
```

```
) : (
```

```
 <div className="space-y-3">
```

```
 <label className="text-sm font-medium">Select a completed game</label>
```

```
 <div className="grid gap-2 sm:grid-cols-2 lg:grid-cols-3">
```

```
 {completedGames.map((g) => (
```

```
 <button
```

```
 key={g.id}
```

```
 onClick={() => {
```

```
 setSelectedGameId(g.id);
```

```
 setShowQuestions(false);
```

```
 }})
```

```
 className={cn(
```

```
 "flex items-start gap-3 p-3 rounded-lg border text-left transition-colors",
```

```
 selectedGameId === g.id
```

```

 ? "border-primary bg-primary/10"
 : "border-border hover:border-primary/50 hover:bg-muted/50"
)}
 >
 <Trophy className="h-4 w-4 mt-0.5 shrink-0 text-amber-500" />
 <div className="min-w-0">
 <p className="font-medium text-sm truncate">{g.topic}</p>
 <p className="text-xs text-muted-foreground">
 {g.questionCount} question{g.questionCount !== 1 ? "s" : ""}
 </p>
 </div>
</button>
)})
</div>
</div>
)}
</CardContent>
</Card>

{/* Results display */}
{selectedGameId && (
 <>
 {loadingResults ? (
 <Card>
 <CardContent className="py-12 text-center text-muted-foreground text-sm">
 Loading results...

```



```

</CardContent>

</Card>

) : resultsData ? (

<>

{ /* Header */}

<div className="flex items-center justify-between gap-3 flex-wrap">

 <div>

 <h3 className="text-lg font-bold">{resultsData.game.topic}</h3>

 <p className="text-sm text-muted-foreground">

 {resultsData.participants.length} player{resultsData.participants.length !== 1 ?

"s" : ""}.{" "}

 {resultsData.totalQuestions} question{resultsData.totalQuestions !== 1 ? "s" : ""}

 </p>

 </div>

 <Button variant="outline" size="sm" onClick={handleExport} className="gap-2">

 <Download className="h-4 w-4" />

 Export CSV

 </Button>

</div>

{ /* Leaderboard */}

<Card>

 <CardHeader className="pb-3">

 <CardTitle className="text-base flex items-center gap-2">

 <Trophy className="h-4 w-4 text-amber-500" />

 Final Leaderboard

```

```

</CardTitle>

</CardHeader>

<CardContent className="p-0">

 {resultsData.participants.length === 0 ? (

 <p className="text-muted-foreground text-sm text-center py-6">

 No participants recorded.

 </p>

) : (

 <div className="divide-y divide-border">

 {resultsData.participants.map((p: LeaderboardEntry) => (

 <div key={p.userId} className="flex items-center gap-3 px-4 py-3">

 {p.rank <= 3 ? medals[p.rank - 1] : `#${p.rank}`}

 <div className="flex-1 min-w-0">

 <p className="font-semibold truncate">{p.userName}</p>

 <p className="text-xs text-muted-foreground">

 {p.correctCount}/{resultsData.totalQuestions} correct

 </p>

 </div>

 {p.totalScore} pts

 </div>

))}

 </div>

)}

</div>

```

```

 })
 </CardContent>
</Card>

{/* Question stats accordion */}

<Card>

 <button

 className="w-full flex items-center justify-between px-5 py-4 text-left"

 onClick={() => setShowQuestions((v) => !v)}

 >

 <div className="flex items-center gap-2 font-semibold text-sm">

 <BarChart3 className="h-4 w-4 text-primary" />

 Question Analytics

 </div>

 {showQuestions ? (

 <ChevronUp className="h-4 w-4 text-muted-foreground" />

) : (

 <ChevronDown className="h-4 w-4 text-muted-foreground" />

)}

 </button>

 {showQuestions && (

 <CardContent className="pt-0 pb-4">

 {loadingStats ? (

 <p className="text-sm text-muted-foreground text-center py-4">Loading...</p>

) : questionStats.length === 0 ? (

```

```
 <p className="text-sm text-muted-foreground text-center py-4">No question data.</p>
```

```
) : (
```

```
 <div className="space-y-3">
```

```
 {questionStats.map((q: QuestionStat, idx: number) => (
```

```
 <div key={q.id} className="space-y-1">
```

```
 <div className="flex items-start justify-between gap-3">
```

```
 <p className="text-sm font-medium leading-snug">
```

```
 Q{idx + 1}
```

```
 {q.questionText}
```

```
 </p>
```

```

```

```
 {q.percentCorrect !== null ? `${q.percentCorrect}%` : "—"}
```

```

```

```
 </div>
```

```
 <div className="h-2 rounded-full bg-muted overflow-hidden">
```

```
 <div
```

```
 className="h-full rounded-full bg-primary transition-all"
```

```
 style={{ width: `${q.percentCorrect ?? 0}%` }}
```

```
 />
```

```
 </div>
```

```
 <p className="text-xs text-muted-foreground">
```

```
 {q.correctCount}/{q.totalAnswered} players correct · {q.points} pt{q.points
 !== 1 ? "s" : ""}
```

```
 </p>
```

```
 </div>
```

```
)}}
 </div>

 }}
 </CardContent>

 }}
</Card>

</>

): null}

</>

)}}
</div>

);
}
```

// — Sidebar nav —————

// — Help section —————

```
function HelpSection() {

 const sources = [

 { name: "Wikipedia", url: "https://en.wikipedia.org", note: "Well-sourced factual articles" },

 { name: "Britannica", url: "https://www.britannica.com", note: "Encyclopedia with editorial oversight" },

 { name: "Guinness World Records", url: "https://www.guinnessworldrecords.com", note: "Verified record holders" },

 { name: "World Bank Open Data", url: "https://data.worldbank.org", note: "Country statistics and indicators" },
```

```
{ name: "Statista", url: "https://www.statista.com", note: "Statistics with cited sources" },
 { name: "Official government sites", url: "", note: "e.g. census.gov, ons.gov.uk, cia.gov/the-
world-factbook" },
];
```

```
const questionTips = [
 { type: "Multiple Choice", icon: CheckSquare, color: "text-primary", tips: ["Write 4 options
— one correct, three plausible wrong.", "Make all options the same length/format so the
answer doesn't stand out.", "Avoid 'all of the above' or 'none of the above' — they muddy
scoring."] },
 { type: "True / False", icon: ToggleLeft, color: "text-secondary", tips: ["State the claim
clearly and unambiguously — no trick wording.", "Mix True and False answers across your
round.", "Avoid statements that are 'mostly true' or need context to be correct."] },
 { type: "Write-In", icon: PenLine, color: "text-accent", tips: ["The grader is case- and
whitespace-insensitive.", "Pick answers that have one clearly correct form (e.g. a name, a
year, a number).", "Avoid answers that are commonly spelled multiple ways unless you
accept all forms."] },
 { type: "Image Recognition", icon: ImageIcon, color: "text-yellow-400", tips: ["Use a stable,
publicly accessible image URL (e.g. Wikimedia Commons).", "Test that the image loads
before saving the question.", "Keep the expected answer simple and unambiguous."] },
 { type: "Matching", icon: ArrowLeftRight, color: "text-pink-400", tips: ["Aim for 3–5 pairs —
more becomes difficult to read on mobile.", "Left side should be the 'question' (term,
country, symbol), right side the 'answer'", "Pairs are sorted alphabetically by left side before
grading — keep terms distinct."] },
];
```

```
return (
 <div className="space-y-8">
 <div>
 <h2 className="text-xl font-bold tracking-tight">Host Guide</h2>
```

```
<p className="text-muted-foreground text-sm mt-1">
```

How to build a great Trivia Night — from game setup to question writing.

```
</p>
```

```
</div>
```

```
{/* Quick start */}
```

```
<Card className="border-primary/20 bg-primary/5">
```

```
<CardHeader className="pb-3">
```

```
<CardTitle className="text-base flex items-center gap-2">
```

```
<BookOpen className="h-4 w-4 text-primary" />
```

Quick-start workflow

```
</CardTitle>
```

```
</CardHeader>
```

```
<CardContent className="space-y-0 text-sm">
```

```
{[
```

["Create a game", 'Go to Create Game, enter a topic (e.g. World Geography), pick difficulty, and set the question count. The game starts in Waiting status.'],

["Add questions", 'Go to Add Questions, select your new game, and add one question at a time. Use real, verified facts — no guessing.'],

["Go live", 'When all questions are ready, go to Manage Games and click Go Live. Players in the lobby will be notified instantly.'],

["End the game", 'When players have finished answering, click End Game. Everyone is redirected to the results page.'],

```
].map(([step, desc], i) => (
```

```
<div key={i} className="flex gap-3 py-2.5 border-b border-border/30 last:border-0">
```

```

```

```

 {i + 1}

 <div>
 <p className="font-semibold">{step}</p>
 <p className="text-muted-foreground">{desc}</p>
 </div>
</div>
)}}
</CardContent>
</Card>

```

```

{ /* Factual integrity notice */ }
<Card className="border-yellow-500/30 bg-yellow-500/5">
 <CardContent className="p-4 flex gap-3">
 <ShieldAlert className="h-5 w-5 text-yellow-400 shrink-0 mt-0.5" />
 <div className="text-sm space-y-1">
 <p className="font-semibold text-yellow-300">All questions must be factually
verified</p>
 <p className="text-muted-foreground">
 Before adding any question, confirm the answer from a primary or authoritative
source.
 Do not use AI-generated content, personal memory, or unverified trivia apps as your
source.
 Incorrect questions damage player trust and undermine the event.
 </p>
 </div>
 </CardContent>

```



</Card>

```
{/* Difficulty guide */}
```

<div>

<h3 className="font-semibold mb-3 flex items-center gap-2">

<Lightbulb className="h-4 w-4 text-accent" />

Difficulty guide

</h3>

<div className="grid gap-3 md:grid-cols-3">

{[

{ level: "Easy", color: "border-green-500/30 bg-green-500/5", badge: "bg-green-500/20 text-green-400", desc: "General knowledge most adults know. Good for first rounds or mixed-age crowds. Example: What is the capital of France?" },

{ level: "Medium", color: "border-yellow-500/30 bg-yellow-500/5", badge: "bg-yellow-500/20 text-yellow-400", desc: "Requires some background knowledge. Appropriate for a typical pub quiz. Example: In what year did the Berlin Wall fall?" },

{ level: "Hard", color: "border-red-500/30 bg-red-500/5", badge: "bg-red-500/20 text-red-400", desc: "Specialist or obscure knowledge. Use sparingly — one or two per round. Example: What is the chemical symbol for tungsten?" },

].map(({ level, color, badge, desc }) => (

<Card key={level} className={`border \${color}`}>

<CardContent className="p-4 space-y-2">

<span className={`text-xs font-bold uppercase tracking-widest px-2 py-0.5 rounded-full \${badge}`}>

{level}

</span>

<p className="text-sm text-muted-foreground">{desc}</p>

</CardContent>

```
</Card>
```

```
)))
```

```
</div>
```

```
</div>
```

```
{/* Question type tips */}
```

```
<div>
```

```
<h3 className="font-semibold mb-3 flex items-center gap-2">
```

```
<Star className="h-4 w-4 text-accent" />
```

```
Tips by question type
```

```
</h3>
```

```
<div className="space-y-3">
```

```
{questionTips.map(({ type, icon: Icon, color, tips }) => (
```

```
<Card key={type} className="border-border/40 bg-card/50">
```

```
<CardContent className="p-4">
```

```
<div className="flex items-center gap-2 mb-2">
```

```
<Icon className={`h-4 w-4 ${color}`} />
```

```
<p className="font-semibold text-sm">{type}</p>
```

```
</div>
```

```
<ul className="space-y-1">
```

```
{tips.map((tip, i) => (
```

```
<li key={i} className="text-sm text-muted-foreground flex gap-2">
```

```
•
```

```
{tip}
```

```

```

```
)))
```

```


 </CardContent>

 </Card>

)}

</div>

</div>

```

```

{ /* Suggested sources */}

<div>

 <h3 className="font-semibold mb-3 flex items-center gap-2">

 <ExternalLink className="h-4 w-4 text-secondary" />

 Suggested sources

 </h3>

 <Card className="border-border/40 bg-card/50">

 <CardContent className="p-0">

 {sources.map(({ name, url, note }, i) => (

 <div

 key={name}

 className={`flex items-start gap-3 px-4 py-3 ${i < sources.length - 1 ? "border-b
border-border/30" : ""}`}

 >

 <CheckCircle2 className="h-4 w-4 text-secondary shrink-0 mt-0.5" />

 <div className="min-w-0">

 {url ? (

 <a

 href={url}

```

```

 target="_blank"

 rel="noopener noreferrer"

 className="text-sm font-semibold text-secondary hover:underline inline-flex
items-center gap-1"

 >

 {name}

 <ExternalLink className="h-3 w-3" />

): (

 <p className="text-sm font-semibold">{name}</p>

 })

 <p className="text-xs text-muted-foreground">{note}</p>

</div>

</div>

)}}

</CardContent>

</Card>

</div>

</div>

);

}

```

```
// — Nav items
```

---

```

const NAV_ITEMS: { id: Section; label: string; icon: typeof LayoutDashboard }[] = [

 { id: "dashboard", label: "Dashboard", icon: LayoutDashboard },

```

```
{ id: "create", label: "Create Game", icon: PlusCircle },
{ id: "manage", label: "Manage Games", icon: Gamepad2 },
{ id: "questions", label: "Add Questions", icon: HelpCircle },
{ id: "review", label: "Review Questions", icon: ShieldCheck },
{ id: "settings", label: "Settings", icon: Settings },
{ id: "results", label: "Results", icon: BarChart3 },
{ id: "help", label: "Help & Guide", icon: BookOpen },
];
```

```
// — AdminDashboard
```

---

```
function AdminDashboard() {
 const { logout } = useAuth();
 const [, setLocation] = useLocation();
 const [section, setSection] = useState<Section>("dashboard");
 const [preferredGameId, setPreferredGameId] = useState<number |
undefined>(undefined);
 const [sidebarOpen, setSidebarOpen] = useState(false);

 const { data: games = [] } = useListGames(undefined, {
 query: { queryKey: getListGamesQueryKey(), refetchInterval: 10000 },
 });

 const activeGame = games.find((g) => g.status === "active");

 const navigate = (s: Section, gameId?: number) => {
```

```
setSection(s);
if (gameId !== undefined) setPreferredGameId(gameId);
setSidebarOpen(false);
};
```

```
const handleCreated = (game: Game) => {
 setPreferredGameId(game.id);
 setSection("questions");
};
```

```
const renderSection = () => {
 switch (section) {
 case "dashboard":
 return <DashboardSection games={games} onNavigate={(s) => navigate(s)} />;
 case "create":
 return <CreateGameSection onCreated={handleCreated} />;
 case "manage":
 return (
 <ManageGamesSection
 games={games}
 onManageQuestions={(g) => navigate("questions", g.id)}
 />
);
 case "questions":
 return <QuestionsSection games={games} preferGameId={preferredGameId} />;
 case "review":
```

```

 return <ReviewSection games={games} />;

 case "settings":

 return <SettingsSection />;

 case "results":

 return <ResultsSection games={games} />;

 case "help":

 return <HelpSection />;

 }

};

return (

 <div className="min-h-[100dvh] flex flex-col">

 {/* Top bar */}

 <header className="border-b border-border bg-card/60 backdrop-blur sticky top-0 z-20">

 <div className="flex items-center justify-between px-4 h-14 gap-3">

 <div className="flex items-center gap-3">

 <button

 className="lg:hidden p-1.5 rounded-md hover:bg-muted transition-colors"

 onClick={() => setSidebarOpen((v) => !v)}

 aria-label="Toggle sidebar"

 >

 <div className="space-y-1">

 <div className="w-5 h-0.5 bg-foreground" />

 <div className="w-5 h-0.5 bg-foreground" />

 <div className="w-5 h-0.5 bg-foreground" />

```

```

 </div>

 </button>

 <div className="flex items-center gap-2">

 <Shield className="h-5 w-5 text-primary shrink-0" />

 HOST CONTROL

 </div>

</div>

{activeGame && (

 <button

 onClick={() => navigate("manage")}

 className="hidden md:flex items-center gap-2 rounded-full border border-
primary/30 bg-primary/10 px-3 py-1 text-sm font-medium text-primary hover:bg-primary/20
transition-colors"

 >

 {activeGame.topic} is live

 </button>

)}

</div>

</header>

<div className="flex flex-1 relative">

 {/* Sidebar */}

 <aside

 className={`

```



```

fixed inset-y-0 left-0 z-30 w-56 bg-card border-r border-border flex flex-col pt-14
transition-transform duration-200 lg:translate-x-0 lg:static lg:z-auto lg:pt-0
${sidebarOpen ? "translate-x-0" : "-translate-x-full"}
`
}
>
<nav className="flex-1 p-3 space-y-1 overflow-y-auto">
 {NAV_ITEMS.map((item) => (
 <button
 key={item.id}
 onClick={() => navigate(item.id)}
 className={`
 w-full flex items-center gap-3 rounded-lg px-3 py-2.5 text-sm font-medium
 transition-colors text-left
 ${
 section === item.id
 ? "bg-primary/10 text-primary"
 : "text-muted-foreground hover:bg-muted hover:text-foreground"
 }
 `}
 >
 <item.icon className="h-4 w-4 shrink-0" />
 {item.label}
 </button>
))}
</nav>
<div className="p-3 border-t border-border space-y-2">

```

```

<p className="text-[11px] text-muted-foreground px-3">
 {games.length} game{games.length !== 1 ? "s" : ""} .{" "}
 {activeGame ? "1 live" : "none live"}
</p>

<button
 onClick={async () => { await logout(); setLocation("/"); }}
 className="w-full flex items-center gap-3 rounded-lg px-3 py-2.5 text-sm font-
medium transition-colors text-left text-muted-foreground hover:bg-muted hover:text-
foreground"
 >
 <LogOut className="h-4 w-4 shrink-0" />
 Sign out
</button>
</div>
</aside>

{/* Sidebar overlay on mobile */}
{sidebarOpen && (
 <div
 className="fixed inset-0 z-20 bg-background/60 backdrop-blur-sm lg:hidden"
 onClick={() => setSidebarOpen(false)}
 />
)}

{/* Main content */}
<main className="flex-1 overflow-y-auto">
 <AnimatePresence mode="wait">

```

```

<motion.div
 key={section}
 initial={{ opacity: 0, y: 8 }}
 animate={{ opacity: 1, y: 0 }}
 exit={{ opacity: 0, y: -8 }}
 transition={{ duration: 0.15 }}
 className="p-6 max-w-4xl"
>

 {renderSection()}
</motion.div>
</AnimatePresence>
</main>
</div>
</div>
);
}

```

// — AdminGate

---

```

function AdminGate() {
 const [, setLocation] = useLocation();
 useEffect(() => { setLocation("/admin-login"); }, []);
 return null;
}

```

// — Export

---

```
export default function Admin() {
 const { isAdmin } = useAuth();
 return isAdmin ? <AdminDashboard /> : <AdminGate />;
}
```

```

`artifacts/trivia-game/src/pages/AdminLogin.tsx`

```typescript

```
import { useState } from "react";
import { useLocation, Link } from "wouter";
import { useAuth } from "../lib/auth";
import { Footer } from "@components/Footer";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { useToast } from "@hooks/use-toast";
import { Shield, Loader2, AlertCircle, ArrowLeft } from "lucide-react";

export default function AdminLogin() {
```

```
const [code, setCode] = useState("");
const [error, setError] = useState("");
const [pending, setPending] = useState(false);
const { loginAdmin } = useAuth();
const [, setLocation] = useLocation();
const { toast } = useToast();

const handleSubmit = async (e: React.FormEvent) => {
 e.preventDefault();
 const trimmed = code.trim().toUpperCase();
 if (!trimmed) {
 setError("Enter your admin code");
 return;
 }
 setError("");
 setPending(true);
 try {
 const res = await fetch("/api/admin/login", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 credentials: "include",
 body: JSON.stringify({ code: trimmed }),
 });

 if (res.status === 401) {
 setError("That code isn't right — try again");
 }
 }
}
```

```
 return;
 }
 if (!res.ok) {
 toast({ variant: "destructive", title: "Something went wrong — please retry" });
 return;
 }
```

```
 loginAdmin();
 setLocation("/admin");
 } catch {
 toast({ variant: "destructive", title: "Connection error — please retry" });
 } finally {
 setPending(false);
 }
};
```

```
return (
 <div className="min-h-[100dvh] flex flex-col items-center justify-center p-4">
 <div className="w-full max-w-md space-y-8 animate-in fade-in slide-in-from-bottom-4 duration-700">
 <div className="text-center space-y-2">
 <div className="flex items-center justify-center gap-3">
 <Shield className="h-10 w-10 text-primary" />
 <h1 className="text-4xl font-bold tracking-tighter">
 HOST ACCESS
 </h1>
 </div>
 </div>
 </div>
 </div>
```

```
</div>
```

```
<p className="text-muted-foreground">
```

```
 Enter your admin code to manage tonight's games
```

```
</p>
```

```
</div>
```

```
<Card className="border-primary/20 bg-card/50 backdrop-blur">
```

```
 <CardHeader className="pb-2">
```

```
 <CardTitle className="text-base font-medium text-muted-foreground uppercase
tracking-widest">
```

```
 Admin Code
```

```
 </CardTitle>
```

```
 </CardHeader>
```

```
 <CardContent>
```

```
 <form onSubmit={handleSubmit} className="space-y-4">
```

```
 <div className="space-y-2">
```

```
 <Input
```

```
 value={code}
```

```
 onChange={(e) => {
```

```
 setCode(e.target.value);
```

```
 setError("");
```

```
 }}
```

```
 placeholder="ENTER ADMIN CODE"
```

```
 autoCapitalize="characters"
```

```
 autoComplete="off"
```

```
 autoFocus
```

```

aria-invalid={!error}

className={`h-14 text-center text-2xl uppercase tracking-widest bg-background
border-primary/30 focus-visible:ring-primary ${
 error ? "border-destructive focus-visible:ring-destructive" : ""
}`}
/>

{error && (
 <p className="flex items-center gap-1.5 text-sm text-destructive">
 <AlertCircle className="h-4 w-4 shrink-0" />
 {error}
 </p>
)}
</div>

<Button
 type="submit"
 className="w-full h-14 text-lg font-bold tracking-wide"
 disabled={pending}
>
 {pending ? (
 <>
 <Loader2 className="mr-2 h-5 w-5 animate-spin" />
 VERIFYING...
 </>
) : (
 "UNLOCK DASHBOARD"
)}

```



```

 })
 </Button>
</form>
</CardContent>
</Card>

<div className="text-center">
 <Link
 href="/"
 className="inline-flex items-center gap-1.5 text-sm text-muted-foreground
hover:text-foreground transition-colors"
 >
 <ArrowLeft className="h-4 w-4" />
 Back to player login
 </Link>
</div>
</div>
<Footer />
</div>
);
}

...

```

```
`artifacts/trivia-game/src/pages/not-found.tsx`
```

```
` `` typescript
```

```
import { Card, CardContent } from "@components/ui/card";
```

```
import { AlertCircle } from "lucide-react";
```

```
export default function NotFound() {
```

```
 return (
```

```
 <div className="min-h-screen w-full flex items-center justify-center bg-gray-50">
```

```
 <Card className="w-full max-w-md mx-4">
```

```
 <CardContent className="pt-6">
```

```
 <div className="flex mb-4 gap-2">
```

```
 <AlertCircle className="h-8 w-8 text-red-500" />
```

```
 <h1 className="text-2xl font-bold text-gray-900">404 Page Not Found</h1>
```

```
 </div>
```

```
 <p className="mt-4 text-sm text-gray-600">
```

```
 Did you forget to add the page to the router?
```

```
 </p>
```

```
 </CardContent>
```

```
 </Card>
```

```
 </div>
```

```
);
```

```
}
```

```
` ``
```

---

## `artifacts/trivia-game/src/components/Footer.tsx`

```typescript

export function Footer() {

return (

<footer className="py-5 px-6 text-center border-t border-border/20 mt-8">

<p className="text-xs text-muted-foreground/50 tracking-wide">

All trivia questions are sourced from verified real-world facts.{" "}

No AI-generated content.

</p>

</footer>

);

}

```

---

## `artifacts/trivia-game/src/hooks/useGameSocket.ts`

```typescript

import { useEffect, useRef } from "react";

import { getSocket, type ServerToClientEvents } from "../lib/socket";

```
// — useLobbySocket —————  
  
export function useLobbySocket(callbacks: {  
  onGameStarted?: (p: ServerToClientEvents["game:started"] extends (p: infer P) => void ? P  
: never) => void;  
}) {  
  const cbRef = useRef(callbacks);  
  cbRef.current = callbacks;  
  
  useEffect(() => {  
    const socket = getSocket();  
    socket.connect();  
    socket.emit("lobby:join");  
  
    function onGameStarted(p: { gameId: number; topic: string }) {  
      cbRef.current.onGameStarted?.(p);  
    }  
  
    socket.on("game:started", onGameStarted);  
  
    return () => {  
      socket.off("game:started", onGameStarted);  
      socket.disconnect();  
    };  
  }, []);  
}
```

```
}
```

```
// — useGameSocket
```

```
export function useGameSocket(  
  gameId: number | null,  
  callbacks: {  
    onAnswerSubmitted?: (p: {  
      gameId: number;  
      questionId: number;  
      playerName: string;  
      isCorrect: boolean;  
    }) => void;  
    onGameEnded?: (p: { gameId: number }) => void;  
  },  
) {  
  const cbRef = useRef(callbacks);  
  cbRef.current = callbacks;  
  
  useEffect(() => {  
    if (!gameId) return;  
  
    const socket = getSocket();  
    socket.connect();  
    socket.emit("game:join", gameId);
```

```

function onAnswerSubmitted(p: {
  gameId: number;
  questionId: number;
  playerName: string;
  isCorrect: boolean;
}) {
  cbRef.current.onAnswerSubmitted?.(p);
}

function onGameEnded(p: { gameId: number }) {
  if (p.gameId === gameId) cbRef.current.onGameEnded?.(p);
}

socket.on("answer:submitted", onAnswerSubmitted);
socket.on("game:ended", onGameEnded);

return () => {
  socket.off("answer:submitted", onAnswerSubmitted);
  socket.off("game:ended", onGameEnded);
  socket.disconnect();
};
}, [gameId]);
}

...

```

```
## `artifacts/trivia-game/src/hooks/use-mobile.tsx`
```

```
` `` typescript
```

```
import * as React from "react"
```

```
const MOBILE_BREAKPOINT = 768
```

```
export function useIsMobile() {
```

```
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined)
```

```
  React.useEffect(() => {
```

```
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
```

```
    const onChange = () => {
```

```
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
```

```
    }
```

```
    mql.addEventListener("change", onChange)
```

```
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
```

```
    return () => mql.removeEventListener("change", onChange)
```

```
  }, [])
```

```
  return !!isMobile
```

```
}
```

```
` ``
```

`artifacts/trivia-game/src/hooks/use-toast.ts`

```typescript

import \* as React from "react"

import type {  
 ToastActionElement,  
 ToastProps,  
} from "@components/ui/toast"

const TOAST\_LIMIT = 1  
const TOAST\_REMOVE\_DELAY = 1000000

type ToasterToast = ToastProps & {  
 id: string  
 title?: React.ReactNode  
 description?: React.ReactNode  
 action?: ToastActionElement  
}

const actionTypes = {  
 ADD\_TOAST: "ADD\_TOAST",  
 UPDATE\_TOAST: "UPDATE\_TOAST",



```
DISMISS_TOAST: "DISMISS_TOAST",
REMOVE_TOAST: "REMOVE_TOAST",
} as const
```

```
let count = 0
```

```
function genId() {
 count = (count + 1) % Number.MAX_SAFE_INTEGER
 return count.toString()
}
```

```
type ActionType = typeof actionTypes
```

```
type Action =
```

```
| {
 type: ActionType["ADD_TOAST"]
 toast: ToasterToast
}
| {
 type: ActionType["UPDATE_TOAST"]
 toast: Partial<ToasterToast>
}
| {
 type: ActionType["DISMISS_TOAST"]
 toastId?: ToasterToast["id"]
}
```

```
| {
 type: ActionType["REMOVE_TOAST"]
 toastId?: ToasterToast["id"]
}
```

```
interface State {
 toasts: ToasterToast[]
}
```

```
const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>()
```

```
const addToRemoveQueue = (toastId: string) => {
 if (toastTimeouts.has(toastId)) {
 return
 }
}
```

```
const timeout = setTimeout(() => {
 toastTimeouts.delete(toastId)
 dispatch({
 type: "REMOVE_TOAST",
 toastId: toastId,
 })
}, TOAST_REMOVE_DELAY)
```

```
toastTimeouts.set(toastId, timeout)
}
```

```
export const reducer = (state: State, action: Action): State => {
 switch (action.type) {
 case "ADD_TOAST":
 return {
 ...state,
 toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
 }

 case "UPDATE_TOAST":
 return {
 ...state,
 toasts: state.toasts.map((t) =>
 t.id === action.toast.id ? { ...t, ...action.toast } : t
),
 }

 case "DISMISS_TOAST": {
 const { toastId } = action

 // ! Side effects ! - This could be extracted into a dismissToast() action,
 // but I'll keep it here for simplicity

 if (toastId) {
 addToRemoveQueue(toastId)
 } else {
 state.toasts.forEach((toast) => {
```

```
 addToRemoveQueue(toast.id)
 })
}
```

```
return {
 ...state,
 toasts: state.toasts.map((t) =>
 t.id === toastId || toastId === undefined
 ? {
 ...t,
 open: false,
 }
 : t
),
}
```

```
case "REMOVE_TOAST":
```

```
 if (action.toastId === undefined) {
```

```
 return {
 ...state,
 toasts: [],
 }
 }
```

```
}
```

```
return {
```

```
 ...state,
```

```
 toasts: state.toasts.filter((t) => t.id !== action.toastId),
```

```
 }
 }
}
```

```
const listeners: Array<(state: State) => void> = []
```

```
let memoryState: State = { toasts: [] }
```

```
function dispatch(action: Action) {
 memoryState = reducer(memoryState, action)
 listeners.forEach((listener) => {
 listener(memoryState)
 })
}
```

```
type Toast = Omit<ToasterToast, "id">
```

```
function toast({ ...props }: Toast) {
 const id = genId()
```

```
 const update = (props: ToasterToast) =>
 dispatch({
 type: "UPDATE_TOAST",
 toast: { ...props, id },
 })
```

```
 const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id })
```

```
dispatch({
 type: "ADD_TOAST",
 toast: {
 ...props,
 id,
 open: true,
 onOpenChange: (open) => {
 if (!open) dismiss()
 },
 },
})
```

```
return {
 id: id,
 dismiss,
 update,
}
}
```

```
function useToast() {
 const [state, setState] = React.useState<State>(memoryState)
```

```
 React.useEffect(() => {
 listeners.push(setState)
 return () => {
```

```
const index = listeners.indexOf(setState)

if (index > -1) {
 listeners.splice(index, 1)
}

}, [state])

return {
 ...state,
 toast,
 dismiss: (toastId?: string) => dispatch({ type: "DISMISS_TOAST", toastId }),
}
}
```

```
export { useToast, toast }
```

```
` ``
```

```

```

```
`artifacts/api-server/src/index.ts`
```

```
` `` `typescript
```

```
import { createServer } from "node:http";
```

```
import app from "./app";
```

```
import { initSocket } from "./lib/socket";
```

```
import { logger } from "../lib/logger";

const rawPort = process.env["PORT"];

if (!rawPort) {
 throw new Error(
 "PORT environment variable is required but was not provided.",
);
}

const port = Number(rawPort);

if (Number.isNaN(port) || port <= 0) {
 throw new Error(`Invalid PORT value: "${rawPort}"`);
}

const httpServer = createServer(app);

initSocket(httpServer);

httpServer.on("error", (err) => {
 logger.error({ err }, "Error listening on port");
 process.exit(1);
});

httpServer.listen(port, () => {
```



```
 logger.info({ port }, "Server listening");
 });

 ...

```

```
`artifacts/api-server/src/app.ts`
```

```
` `` `typescript
```

```
import express, { type Express } from "express";
import cors from "cors";
import pinoHttp from "pino-http";
import router from "./routes";
import { logger } from "./lib/logger";
import { sessionMiddleware } from "./lib/session";
```

```
const app: Express = express();
```

```
// Trust the Replit reverse proxy so express-rate-limit can read real client IPs
```

```
app.set("trust proxy", 1);
```

```
app.use(
 pinoHttp({
 logger,
 serializers: {
```

```
req(req) {
 return {
 id: req.id,
 method: req.method,
 url: req.url?.split("?")[0],
 };
},
res(res) {
 return {
 statusCode: res.statusCode,
 };
},
},
}),
);

app.use(cors({ origin: true, credentials: true }));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(sessionMiddleware);

app.use("/api", router);

export default app;

` ``
```

---

## `artifacts/api-server/src/types/session.d.ts`

```typescript

import "express-session";

declare module "express-session" {

interface SessionData {

userId?: number;

userName?: string;

isAdmin?: boolean;

}

}

```

---

## `artifacts/api-server/src/lib/logger.ts`

```typescript

import pino from "pino";

const isProduction = process.env.NODE_ENV === "production";

```

export const logger = pino({
  level: process.env.LOG_LEVEL ?? "info",
  redact: [
    "req.headers.authorization",
    "req.headers.cookie",
    "res.headers['set-cookie']",
  ],
  ...(isProduction
    ? {}
    : {
        transport: {
          target: "pino-pretty",
          options: { colorize: true },
        },
      }),
});

```

```

---

```
`artifacts/api-server/src/lib/serialize.ts`
```

```
```typescript
```

```

export function toJsonSafe<T>(value: T): unknown {
  return JSON.parse(JSON.stringify(value));
}

```

```
}
```

```
```
```

```

```

```
`artifacts/api-server/src/lib/session.ts`
```

```
```typescript
```

```
import session from "express-session";
```

```
const SESSION_SECRET = process.env["SESSION_SECRET"];
```

```
if (!SESSION_SECRET) {
```

```
  throw new Error("SESSION_SECRET environment variable is required");
```

```
}
```

```
export const sessionMiddleware = session({
```

```
  secret: SESSION_SECRET,
```

```
  resave: false,
```

```
  saveUninitialized: false,
```

```
  cookie: {
```

```
    httpOnly: true,
```

```
    secure: process.env["NODE_ENV"] === "production",
```

```
    sameSite: "lax",
```

```
    maxAge: 7 * 24 * 60 * 60 * 1000,
```

```
  },
```

```
});
```

```
`, `
```

```
---
```

```
## `artifacts/api-server/src/lib/socket.ts`
```

```
` `` typescript
```

```
import { Server as SocketIOServer } from "socket.io";
```

```
import type { Server as HTTPServer } from "node:http";
```

```
import type { Request, Response } from "express";
```

```
import type { NextFunction } from "express";
```

```
import { eq, and } from "drizzle-orm";
```

```
import { db, gameParticipantsTable } from "@workspace/db";
```

```
import { logger } from "../logger";
```

```
import { sessionMiddleware } from "../session";
```

```
// — Typed event maps —————
```

```
export type ServerToClientEvents = {
```

```
  "game:started": (payload: { gameId: number; topic: string }) => void;
```

```
  "game:ended": (payload: { gameId: number }) => void;
```

```
  "answer:submitted": (payload: {
```

```
    gameId: number;
```

```
    questionId: number;
```

```
    playerName: string;
    isCorrect: boolean;
  }) => void;
  "player:joined": (payload: {
    gameId: number;
    playerName: string;
    participantCount: number;
  }) => void;
};

export type ClientToServerEvents = {
  "lobby:join": () => void;
  "game:join": (gameId: number) => void;
};
```

// — Singleton —

```
type IO = SocketIOServer<ClientToServerEvents, ServerToClientEvents>;
```

```
let io: IO | null = null;
```

```
export function initSocket(server: HTTPServer): IO {
  io = new SocketIOServer<ClientToServerEvents, ServerToClientEvents>(server, {
    path: "/api/socket.io",
    cors: { origin: true, credentials: true },
    transports: ["polling", "websocket"],
```

```
});
```

```
io.use((socket, next) => {  
  sessionMiddleware(  
    socket.request as Request,  
    {} as Response,  
    next as NextFunction,  
  );  
});
```

```
io.on("connection", (socket) => {  
  logger.debug({ socketId: socket.id }, "Socket connected");
```

```
  const req = socket.request as Request;
```

```
  socket.on("lobby:join", () => {  
    if (!req.session.userId && !req.session.isAdmin) {  
      logger.debug({ socketId: socket.id }, "Unauthenticated lobby:join rejected");  
      return;  
    }  
    void socket.join("lobby");  
  });
```

```
  socket.on("game:join", (gameId: number) => {  
    if (!req.session.userId && !req.session.isAdmin) {  
      logger.debug({ socketId: socket.id }, "Unauthenticated game:join rejected");
```



```

    return;
}

if (req.session.isAdmin) {
    void socket.join(` game:${gameId}`);
    return;
}

// Non-admin: verify the caller is a participant of this game
db.select({ id: gameParticipantsTable.id })
    .from(gameParticipantsTable)
    .where(
        and(
            eq(gameParticipantsTable.gameId, gameId),
            eq(gameParticipantsTable.userId, req.session.userId!),
        ),
    )
    .then(([participant]) => {
        if (!participant) {
            logger.debug({ socketId: socket.id, gameId }, "game:join rejected: not a participant");
            return;
        }
        void socket.join(` game:${gameId}`);
    })
    .catch((err) => {
        logger.error({ err, socketId: socket.id }, "game:join participant check failed");
    });
});

```

```

    socket.on("disconnect", () => {
        logger.debug({ socketId: socket.id }, "Socket disconnected");
    });
});

return io;
}

export function getIo(): IO {
    if (!io) throw new Error("Socket.io not initialized — call initSocket first");
    return io;
}

/** Emit without crashing if socket not initialized (safe for edge cases). */
export function safeEmit(
    room: string,
    event: keyof ServerToClientEvents,
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    payload: any,
): void {
    try {
        // Using `any` here because socket.io's overloaded emit generics don't
        // play nicely with our conditional type extraction in strict mode.
        // eslint-disable-next-line @typescript-eslint/no-explicit-any
        (getIo().to(room) as any).emit(event, payload);
    }
}

```

```
} catch {  
  // Socket not initialized — silently skip  
}  
}  
  
` `` `   
  
---  
  
## ` artifacts/api-server/src/middleware/authRateLimit.ts`  
  
` `` ` typescript  
import rateLimit from "express-rate-limit";  
  
export const authRateLimit = rateLimit({  
  windowMs: 15 * 60 * 1000,  
  limit: 10,  
  standardHeaders: "draft-8",  
  legacyHeaders: false,  
  message: { error: "Too many attempts. Please try again later." },  
  skipSuccessfulRequests: false,  
});  
  
` `` `   
  
---
```

```
## `artifacts/api-server/src/middleware/providerRateLimit.ts`
```

```
` `` typescript
```

```
import rateLimit from "express-rate-limit";
```

```
/**
```

```
 * Strict limit for the bulk Gemini question-generation endpoint.
```

```
 * Each call may retry across multiple models and consume significant quota.
```

```
 * 5 requests per 10 minutes per IP.
```

```
 */
```

```
export const geminiGenerateRateLimit = rateLimit({
```

```
  windowMs: 10 * 60 * 1000,
```

```
  limit: 5,
```

```
  standardHeaders: "draft-8",
```

```
  legacyHeaders: false,
```

```
  message: { error: "Too many generation requests. Please wait before generating again." },
```

```
});
```

```
/**
```

```
 * Moderate limit for single-question Gemini operations (regenerate, enhance, fact-check).
```

```
 * 20 requests per 10 minutes per IP.
```

```
 */
```

```
export const geminiOperationRateLimit = rateLimit({
```

```
  windowMs: 10 * 60 * 1000,
```

```
  limit: 20,
```

```

    standardHeaders: "draft-8",
    legacyHeaders: false,
    message: { error: "Too many AI requests. Please wait before trying again." },
  });

/**
 * Limit for the OpenTDB import endpoint.
 * Prevents scripted upstream fetches and DB insert floods.
 * 10 requests per 10 minutes per IP.
 */
export const opentdbRateLimit = rateLimit({
  windowMs: 10 * 60 * 1000,
  limit: 10,
  standardHeaders: "draft-8",
  legacyHeaders: false,
  message: { error: "Too many import requests. Please wait before importing again." },
});

\ \ \

---

## `artifacts/api-server/src/middleware/requireAdmin.ts`

```typescript
import type { Request, Response, NextFunction } from "express";

```

```
export function requireAdmin(
 req: Request,
 res: Response,
 next: NextFunction,
) : void {
 if (!req.session.isAdmin) {
 res.status(403).json({ error: "Admin access required" });
 return;
 }
 next();
}
```

...

---

## `artifacts/api-server/src/middleware/requireAuth.ts`

```typescript

```
import type { Request, Response, NextFunction } from "express";
```

```
export function requireAuth(  
  req: Request,  
  res: Response,  
  next: NextFunction,
```

```

): void {
  if (!req.session.userId && !req.session.isAdmin) {
    res.status(401).json({ error: "Not authenticated" });
    return;
  }
  next();
}

```

...

`artifacts/api-server/src/middleware/requireUser.ts`

```typescript

```
import type { Request, Response, NextFunction } from "express";
```

```
export function requireUser(
```

```
 req: Request,
```

```
 res: Response,
```

```
 next: NextFunction,
```

```
): void {
```

```
 if (!req.session.userId) {
```

```
 res.status(401).json({ error: "Not authenticated" });
```

```
 return;
```

```
 }
```

```
 next();
 }

 ...

```

```
`artifacts/api-server/src/routes/index.ts`
```

```
` `` `typescript
```

```
import { Router, type IRouter } from "express";
import healthRouter from "./health";
import authRouter from "./auth";
import sessionRouter from "./session";
import usersRouter from "./users";
import gamesRouter from "./games";
import questionsRouter from "./questions";
import playRouter from "./play";
import statsRouter from "./stats";
import settingsRouter from "./settings";
import resultsRouter from "./results";
import opentdbRouter from "./opentdb";
import geminiRouter from "./gemini";

const router: IRouter = Router();
```



```
router.use(healthRouter);
router.use(authRouter);
router.use(sessionRouter);
router.use(settingsRouter);
router.use(usersRouter);
router.use(gamesRouter);
router.use(opentdbRouter);
router.use(geminiRouter);
router.use(questionsRouter);
router.use(playRouter);
router.use(resultsRouter);
router.use(statsRouter);
```

```
export default router;
```

```
` `` `
```

```

```

```
`artifacts/api-server/src/routes/auth.ts`
```

```
` `` `typescript
```

```
import { Router, type IRouter } from "express";
import { db, adminSettingsTable } from "@workspace/db";
import {
 VerifyAccessCodeBody,
```

```
VerifyAccessCodeResponse,
} from "@workspace/api-zod";
import { authRateLimit } from "../middleware/authRateLimit";

const router: IRouter = Router();

router.post("/auth/verify", authRateLimit, async (req, res): Promise<void> => {
 const parsed = VerifyAccessCodeBody.safeParse(req.body);
 if (!parsed.success) {
 res.status(400).json({ error: parsed.error.message });
 return;
 }

 const [settings] = await db.select().from(adminSettingsTable).limit(1);

 if (!settings) {
 res.json(VerifyAccessCodeResponse.parse({ valid: false, role: "none" }));
 return;
 }

 const code = parsed.data.code.trim();

 if (code === settings.adminAccessCode) {
 res.json(VerifyAccessCodeResponse.parse({ valid: true, role: "admin" }));
 return;
 }
}
```

```
if (code === settings.triviaAccessCode) {
 res.json(VerifyAccessCodeResponse.parse({ valid: true, role: "player" }));
 return;
}
```

```
res.json(VerifyAccessCodeResponse.parse({ valid: false, role: "none" }));
});
```

```
export default router;
```

```
````
```

```
---
```

```
## `artifacts/api-server/src/routes/session.ts`
```

```
```typescript
```

```
import { Router, type IRouter } from "express";

import { eq } from "drizzle-orm";

import { db, adminSettingsTable, usersTable } from "@workspace/db";

import { toJsonSafe } from "../lib/serialize";

import { authRateLimit } from "../middleware/authRateLimit";
```

```
const router: IRouter = Router();
```

```
// POST /api/auth/login — verify trivia code + create/retrieve user, start session
router.post("/auth/login", authRateLimit, async (req, res): Promise<void> => {
 const code =
 typeof req.body?.code === "string" ? req.body.code.trim() : "";
 const name =
 typeof req.body?.name === "string" ? req.body.name.trim() : "";

 if (!code || !name) {
 res.status(400).json({ error: "Name and access code are required" });
 return;
 }

 if (name.length > 50) {
 res.status(400).json({ error: "Name must be 50 characters or fewer" });
 return;
 }

 const [settings] = await db.select().from(adminSettingsTable).limit(1);
 if (!settings || code !== settings.triviaAccessCode) {
 res.status(401).json({ error: "Invalid access code" });
 return;
 }

 // Always create a new user row — name is a display label, not an identity key.
 // Reusing an existing row by name would let any caller impersonate another
 // player just by knowing their display name.
 const [user] = await db.insert(usersTable).values({ name }).returning();
```

```
req.session.userId = user!.id;
req.session.userName = user!.name;
req.session.isAdmin = false;
```

```
res.json(toJsonSafe({ id: user!.id, name: user!.name }));
});
```

// GET /api/auth/me — return current session user (null if not logged in)

```
router.get("/auth/me", async (req, res): Promise<void> => {
 if (!req.session.userId) {
 res.json({ user: null, isAdmin: false });
 return;
 }
}
```

```
const [user] = await db
 .select()
 .from(usersTable)
 .where(eq(usersTable.id, req.session.userId));
```

```
if (!user) {
 req.session.destroy(() => {});
 res.json({ user: null, isAdmin: false });
 return;
}
```

```
res.json(toJsonSafe({ user: { id: user.id, name: user.name }, isAdmin: false }));
});
```

```
// POST /api/auth/logout — clear player session
```

```
router.post("/auth/logout", (req, res): void => {
 req.session.destroy(() => {
 res.clearCookie("connect.sid");
 res.json({ ok: true });
 });
});
```

```
// POST /api/admin/login — verify admin code, start admin session
```

```
router.post("/admin/login", authRateLimit, async (req, res): Promise<void> => {
 const code =
 typeof req.body?.code === "string" ? req.body.code.trim() : "";

 if (!code) {
 res.status(400).json({ error: "Access code is required" });
 return;
 }

 const envCode = process.env.ADMIN_ACCESS_CODE?.trim();
 if (envCode) {
 if (code !== envCode) {
 res.status(401).json({ error: "Invalid admin code" });
 return;
 }
 }
}
```

```
 }
 } else {
 const [settings] = await db.select().from(adminSettingsTable).limit(1);
 if (!settings || code !== settings.adminAccessCode) {
 res.status(401).json({ error: "Invalid admin code" });
 return;
 }
 }
}
```

```
req.session.isAdmin = true;
req.session.userId = undefined;
req.session.userName = undefined;
```

```
res.json({ ok: true });
});
```

```
// GET /api/admin/me — return admin session state
router.get("/admin/me", (req, res): void => {
 res.json({ isAdmin: req.session.isAdmin === true });
});
```

```
// POST /api/admin/logout — clear admin session
router.post("/admin/logout", (req, res): void => {
 req.session.destroy(() => {
 res.clearCookie("connect.sid");
 res.json({ ok: true });
 });
});
```

```
});
```

```
});
```

```
export default router;
```

```
```\n
```

```
---
```

```
## `artifacts/api-server/src/routes/games.ts`
```

```
```typescript
```

```
import { Router, type IRouter } from "express";
```

```
import { eq, desc, count } from "drizzle-orm";
```

```
import {
```

```
 db,
```

```
 gamesTable,
```

```
 gameParticipantsTable,
```

```
} from "@workspace/db";
```

```
import { safeEmit } from "../lib/socket";
```

```
import {
```

```
 ListGamesQueryParams,
```

```
 ListGamesResponse,
```

```
 CreateGameBody,
```

```
 CreateGameResponse,
```

```
 GetGameParams,
```



```

 GetGameResponse,
 UpdateGameParams,
 UpdateGameBody,
 UpdateGameResponse,
 DeleteGameParams,
 } from "@workspace/api-zod";
import { toJsonSafe } from "../lib/serialize";
import { requireAdmin } from "../middleware/requireAdmin";
import { requireAuth } from "../middleware/requireAuth";

const router: IRouter = Router();

router.get("/games", requireAuth, async (req, res): Promise<void> => {
 const query = ListGamesQueryParams.safeParse(req.query);
 if (!query.success) {
 res.status(400).json({ error: query.error.message });
 return;
 }

 const status = query.data.status;
 const games = status
 ? await db
 .select()
 .from(gamesTable)
 .where(eq(gamesTable.status, status))
 .orderBy(desc(gamesTable.createdAt))

```

```

: await db.select().from(gamesTable).orderBy(desc(gamesTable.createdAt));

res.json(ListGamesResponse.parse(toJsonSafe(games)));
});

router.post("/games", requireAdmin, async (req, res): Promise<void> => {
 const parsed = CreateGameBody.safeParse(req.body);
 if (!parsed.success) {
 res.status(400).json({ error: parsed.error.message });
 return;
 }

 const [game] = await db
 .insert(gamesTable)
 .values({
 topic: parsed.data.topic.trim(),
 difficulty: parsed.data.difficulty,
 createdByAdmin: parsed.data.createdByAdmin ?? true,
 })
 .returning();

 res.status(201).json(CreateGameResponse.parse(toJsonSafe(game)));
});

router.get("/games/:gameId", requireAuth, async (req, res): Promise<void> => {
 const params = GetGameParams.safeParse(req.params);

```

```
if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
}
```

```
const [game] = await db
 .select()
 .from(gamesTable)
 .where(eq(gamesTable.id, params.data.gameId));
```

```
if (!game) {
 res.status(404).json({ error: "Game not found" });
 return;
}
```

```
const [participants] = await db
 .select({ value: count() })
 .from(gameParticipantsTable)
 .where(eq(gameParticipantsTable.gameId, game.id));
```

```
res.json(
 GetGameResponse.parse(toJsonSafe({
 ...game,
 participantCount: participants?.value ?? 0,
 })),
);
```

```
});
```

```
router.patch("/games/:gameId", requireAdmin, async (req, res): Promise<void> => {
 const params = UpdateGameParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
 }
}
```

```
 const parsed = UpdateGameBody.safeParse(req.body);
 if (!parsed.success) {
 res.status(400).json({ error: parsed.error.message });
 return;
 }
```

```
 const [game] = await db
 .update(gamesTable)
 .set(parsed.data)
 .where(eq(gamesTable.id, params.data.gameId))
 .returning();
```

```
 if (!game) {
 res.status(404).json({ error: "Game not found" });
 return;
 }
```

```
res.json(UpdateGameResponse.parse(toJsonSafe(game)));
```

```
// Real-time: notify relevant rooms when status changes
```

```
if (game.status === "active") {
 safeEmit("lobby", "game:started", { gameId: game.id, topic: game.topic });
} else if (game.status === "completed") {
 safeEmit(`game:${game.id}`, "game:ended", { gameId: game.id });
}
});
```

```
router.delete("/games/:gameId", requireAdmin, async (req, res): Promise<void> => {
 const params = DeleteGameParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
 }
}
```

```
const [game] = await db
 .delete(gamesTable)
 .where(eq(gamesTable.id, params.data.gameId))
 .returning();
```

```
if (!game) {
 res.status(404).json({ error: "Game not found" });
 return;
}
```

```
res.sendStatus(204);
});
```

```
export default router;
```

```
````
```

```
---
```

```
## `artifacts/api-server/src/routes/questions.ts`
```

```
```typescript
```

```
import { Router, type IRouter } from "express";
import { eq, asc, count } from "drizzle-orm";
import { db, gamesTable, questionsTable } from "@workspace/db";
import {
 ListGameQuestionsParams,
 ListGameQuestionsResponse,
 CreateQuestionParams,
 CreateQuestionBody,
 CreateQuestionResponse,
 UpdateQuestionParams,
 UpdateQuestionBody,
 UpdateQuestionResponse,
 DeleteQuestionParams,
```

```
} from "@workspace/api-zod";
```

```
import { requireAdmin } from "../middleware/requireAdmin";
```

```
import { requireAuth } from "../middleware/requireAuth";
```

```
const router: IRouter = Router();
```

```
async function syncQuestionCount(gameId: number): Promise<void> {
```

```
 const [row] = await db
```

```
 .select({ value: count() })
```

```
 .from(questionsTable)
```

```
 .where(eq(questionsTable.gameId, gameId));
```

```
 await db
```

```
 .update(gamesTable)
```

```
 .set({ questionCount: row?.value ?? 0 })
```

```
 .where(eq(gamesTable.id, gameId));
```

```
}
```

```
router.get("/games/:gameId/questions", requireAuth, async (req, res): Promise<void> => {
```

```
 const params = ListGameQuestionsParams.safeParse(req.params);
```

```
 if (!params.success) {
```

```
 res.status(400).json({ error: params.error.message });
```

```
 return;
```

```
}
```

```
const questions = await db
```

```

.select()
.from(questionsTable)
.where(eq(questionsTable.gameId, params.data.gameId))
.orderBy(asc(questionsTable.orderIndex));

const isAdmin = req.session.isAdmin === true;

const response = isAdmin
 ? questions
 : questions.map(({ correctAnswer: _ca, ...rest }) => rest);

res.json(ListGameQuestionsResponse.parse(response));
});

router.post("/games/:gameId/questions", requireAdmin, async (req, res): Promise<void> =>
{
 const params = CreateQuestionParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
 }

 const parsed = CreateQuestionBody.safeParse(req.body);
 if (!parsed.success) {
 res.status(400).json({ error: parsed.error.message });
 return;
 }

```



```
}
```

```
const [game] = await db
 .select()
 .from(gamesTable)
 .where(eq(gamesTable.id, params.data.gameId));
```

```
if (!game) {
 res.status(404).json({ error: "Game not found" });
 return;
}
```

```
const [question] = await db
 .insert(questionsTable)
 .values({
 gameId: game.id,
 questionText: parsed.data.questionText,
 questionType: parsed.data.questionType,
 correctAnswer: parsed.data.correctAnswer,
 options: parsed.data.options ?? null,
 imageUrl: parsed.data.imageUrl ?? null,
 points: parsed.data.points,
 orderIndex: parsed.data.orderIndex,
 })
 .returning();
```

```
await syncQuestionCount(game.id);
```

```
res.status(201).json(CreateQuestionResponse.parse(question));
});
```

```
router.patch("/questions/:questionId", requireAdmin, async (req, res): Promise<void> => {
 const params = UpdateQuestionParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
 }
}
```

```
const parsed = UpdateQuestionBody.safeParse(req.body);
if (!parsed.success) {
 res.status(400).json({ error: parsed.error.message });
 return;
}
```

```
const [question] = await db
 .update(questionsTable)
 .set(parsed.data)
 .where(eq(questionsTable.id, params.data.questionId))
 .returning();
```

```
if (!question) {
 res.status(404).json({ error: "Question not found" });
}
```

```
 return;
 }
```

```
 res.json(UpdateQuestionResponse.parse(question));
 });
```

```
router.delete("/questions/:questionId", requireAdmin, async (req, res): Promise<void> => {
 const params = DeleteQuestionParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
 }
}
```

```
const [question] = await db
 .delete(questionsTable)
 .where(eq(questionsTable.id, params.data.questionId))
 .returning();
```

```
if (!question) {
 res.status(404).json({ error: "Question not found" });
 return;
}
```

```
await syncQuestionCount(question.gameId);
```

```
res.sendStatus(204);
```

```
});
```

```
export default router;
```

```
```\n
```

```
---
```

```
## `artifacts/api-server/src/routes/play.ts`
```

```
```typescript
```

```
import { Router, type IRouter } from "express";
```

```
import { rateLimit } from "express-rate-limit";
```

```
import { eq, and, desc, asc } from "drizzle-orm";
```

```
import {
```

```
 db,
```

```
 gamesTable,
```

```
 usersTable,
```

```
 questionsTable,
```

```
 answersTable,
```

```
 gameParticipantsTable,
```

```
} from "@workspace/db";
```

```
import { safeEmit } from "../lib/socket";
```

```
import { requireUser } from "../middleware/requireUser";
```

```
import { requireAuth } from "../middleware/requireAuth";
```

```
const answerRateLimit = rateLimit({
 windowMs: 60_000,
 max: 30,
 message: { error: "Too many answer submissions. Please slow down." },
 standardHeaders: true,
 legacyHeaders: false,
});

import {
 JoinGameParams,
 JoinGameResponse,
 ListGameParticipantsParams,
 ListGameParticipantsResponse,
 SubmitAnswerParams,
 SubmitAnswerBody,
 SubmitAnswerResponse,
 ListUserAnswersParams,
 ListUserAnswersResponse,
} from "@workspace/api-zod";
import { toJsonSafe } from "../lib/serialize";

const router: IRouter = Router();

function normalize(value: string): string {
 return value.trim().toLowerCase().replace(/\s+/g, " ");
}
```

```

function gradeAnswer(
 questionType: string,
 userAnswer: string,
 correctAnswer: string,
 points: number,
 alternates: string[],
): { isCorrect: boolean; pointsEarned: number } {
 // Matching: compare pair-by-pair for partial credit
 if (questionType === "matching") {
 const parsePairs = (ans: string): Record<string, string> =>
 ans.split("|").reduce<Record<string, string>>((acc, pair) => {
 const idx = pair.indexOf(":");
 if (idx === -1) return acc;
 acc[normalize(pair.slice(0, idx))] = normalize(pair.slice(idx + 1));
 return acc;
 }, {});

 const correctPairs = parsePairs(correctAnswer);
 const userPairs = parsePairs(userAnswer);
 const total = Object.keys(correctPairs).length;
 if (total === 0) return { isCorrect: true, pointsEarned: points };

 let correctCount = 0;
 for (const [left, right] of Object.entries(userPairs)) {
 if (correctPairs[left] === right) correctCount++;
 }
 }
}

```

```
const isCorrect = correctCount === total;
const pointsEarned = Math.floor((correctCount / total) * points);
return { isCorrect, pointsEarned };
}
```

// All other types: exact match + alternates

```
const isCorrect =
 normalize(userAnswer) === normalize(correctAnswer) ||
 alternates.some((alt) => normalize(userAnswer) === normalize(alt));
return { isCorrect, pointsEarned: isCorrect ? points : 0 };
}
```

// — Join game —————

```
router.post("/games/:gameId/join", requireUser, async (req, res): Promise<void> => {
 const params = JoinGameParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
 }
}
```

```
const sessionUserId = req.session.userId;
```

```
const [game] = await db
 .select()
```

```
.from(gamesTable)
.where(eq(gamesTable.id, params.data.gameId));
```

```
if (!game) {
 res.status(404).json({ error: "Game not found" });
 return;
}
```

```
const [user] = await db
 .select()
 .from(usersTable)
 .where(eq(usersTable.id, sessionUserId));
```

```
if (!user) {
 res.status(404).json({ error: "User not found" });
 return;
}
```

```
const [existing] = await db
 .select()
 .from(gameParticipantsTable)
 .where(
 and(
 eq(gameParticipantsTable.gameId, game.id),
 eq(gameParticipantsTable.userId, user.id),
),
),
```



```
);
```

```
if (existing) {
```

```
 res.status(201).json(JoinGameResponse.parse(toJsonSafe(existing)));
```

```
 return;
```

```
}
```

```
const [participant] = await db
```

```
 .insert(gameParticipantsTable)
```

```
 .values({ gameId: game.id, userId: user.id })
```

```
 .returning();
```

```
res.status(201).json(JoinGameResponse.parse(toJsonSafe(participant)));
```

```
});
```

```
// — List participants —————
```

```
router.get("/games/:gameId/participants", requireAuth, async (req, res): Promise<void> => {
```

```
 const params = ListGameParticipantsParams.safeParse(req.params);
```

```
 if (!params.success) {
```

```
 res.status(400).json({ error: params.error.message });
```

```
 return;
```

```
}
```

```
const rows = await db
```

```
 .select({
```

```

 id: gameParticipantsTable.id,
 gameId: gameParticipantsTable.gameId,
 userId: gameParticipantsTable.userId,
 userName: usersTable.name,
 totalScore: gameParticipantsTable.totalScore,
 joinedAt: gameParticipantsTable.joinedAt,
 })
 .from(gameParticipantsTable)
 .innerJoin(usersTable, eq(gameParticipantsTable.userId, usersTable.id))
 .where(eq(gameParticipantsTable.gameId, params.data.gameId))
 .orderBy(
 desc(gameParticipantsTable.totalScore),
 asc(gameParticipantsTable.joinedAt),
);

res.json(ListGameParticipantsResponse.parse(toJsonSafe(rows)));
});

```

// — Submit answer —————

```

router.post("/games/:gameId/answers", requireUser, answerRateLimit, async (req, res):
Promise<void> => {
 const params = SubmitAnswerParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
 }

```

```
}
```

```
const parsed = SubmitAnswerBody.safeParse(req.body);
```

```
if (!parsed.success) {
```

```
 res.status(400).json({ error: parsed.error.message });
```

```
 return;
```

```
}
```

```
const sessionUserId = req.session.userId!;
```

```
const [game] = await db
```

```
 .select()
```

```
 .from(gamesTable)
```

```
 .where(eq(gamesTable.id, params.data.gameId));
```

```
if (!game) {
```

```
 res.status(404).json({ error: "Game not found" });
```

```
 return;
```

```
}
```

```
if (game.status !== "active") {
```

```
 res.status(403).json({ error: "Answers can only be submitted to an active game" });
```

```
 return;
```

```
}
```

```
const [participant] = await db
```

```
.select()
.from(gameParticipantsTable)
.where(
 and(
 eq(gameParticipantsTable.gameId, game.id),
 eq(gameParticipantsTable.userId, sessionUserId),
),
);
```

```
if (!participant) {
 res.status(403).json({ error: "You must join the game before submitting answers" });
 return;
}
```

```
const [question] = await db
 .select()
 .from(questionsTable)
 .where(eq(questionsTable.id, parsed.data.questionId));
```

```
if (!question || question.gameId !== params.data.gameId) {
 res.status(404).json({ error: "Question not found in this game" });
 return;
}
```

```
const [already] = await db
 .select()
```

```
.from(answersTable)
.where(
 and(
 eq(answersTable.userId, sessionUserId),
 eq(answersTable.questionId, question.id),
),
);
```

```
if (already) {
 res.status(409).json({ error: "Question already answered" });
 return;
}
```

```
const opts = question.options as { alternateAnswers?: string[] } | null;
const alternates = opts?.alternateAnswers ?? [];
```

```
const { isCorrect, pointsEarned } = gradeAnswer(
 question.questionType,
 parsed.data.userAnswer,
 question.correctAnswer,
 question.points,
 alternates,
);
```

```
const [answer] = await db
 .insert(answersTable)
```

```
.values({
 userId: sessionUserId,
 gameId: question.gameId,
 questionId: question.id,
 userAnswer: parsed.data.userAnswer,
 isCorrect,
 pointsEarned,
})
.returning();
```

```
const totalScore = (participant.totalScore ?? 0) + pointsEarned;
```

```
await db
 .update(gameParticipantsTable)
 .set({ totalScore })
 .where(eq(gameParticipantsTable.id, participant.id));
```

```
res.status(201).json(
 SubmitAnswerResponse.parse(
 toJsonSafe({ ...answer, pointsEarned, totalScore })),
),
);
```

```
// Fire-and-forget: emit real-time event to game room
db.select({ name: usersTable.name })
 .from(usersTable)
```

```

.where(eq(usersTable.id, sessionUserId))
.then(([u]) => {
 if (!u) return;
 safeEmit(` game:${question.gameld}`, "answer:submitted", {
 gameld: question.gameld,
 questionId: question.id,
 playerName: u.name,
 isCorrect,
 });
})
.catch(() => { /* non-critical */ });
});

```

// — Per-question answers (for feedback leaderboard) —————

```

router.get(
 "/games/:gameld/questions/:questionId/answers",
 requireUser,
 async (req, res): Promise<void> => {
 const gameld = parseInt(String(req.params.gameld ?? ""), 10);
 const questionId = parseInt(String(req.params.questionId ?? ""), 10);
 if (isNaN(gameld) || isNaN(questionId)) {
 res.status(400).json({ error: "Invalid params" });
 return;
 }
 }
)

```

```
// Only admins or game participants may view per-question answer telemetry
```

```
if (!req.session.isAdmin) {
```

```
 const [participant] = await db
```

```
 .select({ id: gameParticipantsTable.id })
```

```
 .from(gameParticipantsTable)
```

```
 .where(
```

```
 and(
```

```
 eq(gameParticipantsTable.gameId, gameId),
```

```
 eq(gameParticipantsTable.userId, req.session.userId!),
```

```
),
```

```
);
```

```
if (!participant) {
```

```
 res.status(403).json({ error: "Access denied" });
```

```
 return;
```

```
}
```

```
}
```

```
const [question] = await db
```

```
 .select({ id: questionsTable.id })
```

```
 .from(questionsTable)
```

```
 .where(
```

```
 and(
```

```
 eq(questionsTable.id, questionId),
```

```
 eq(questionsTable.gameId, gameId),
```

```
),
```

```
);
```



```
if (!question) {
 res.status(404).json({ error: "Question not found" });
 return;
}
```

```
const rows = await db
 .select({ isCorrect: answersTable.isCorrect })
 .from(answersTable)
 .where(
 and(
 eq(answersTable.gameld, gameld),
 eq(answersTable.questionId, questionId),
),
);
```

```
const totalAnswered = rows.length;
const correctCount = rows.filter((r) => r.isCorrect).length;
```

```
res.json({ totalAnswered, correctCount });
},
);
```

```
// — List user answers —————
```

```
router.get(
```

```
"/games/:gameId/users/:userId/answers",
requireUser,
async (req, res): Promise<void> => {
 const params = ListUserAnswersParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
 }

 if (req.session.userId !== params.data.userId) {
 res.status(403).json({ error: "Access denied" });
 return;
 }

 const [game] = await db
 .select({ status: gamesTable.status })
 .from(gamesTable)
 .where(eq(gamesTable.id, params.data.gameId));

 if (!game) {
 res.status(404).json({ error: "Game not found" });
 return;
 }

 const gameCompleted = game.status === "completed";
```

```

const rows = await db
 .select({
 id: answersTable.id,
 userId: answersTable.userId,
 gameId: answersTable.gameId,
 questionId: answersTable.questionId,
 userAnswer: answersTable.userAnswer,
 isCorrect: answersTable.isCorrect,
 pointsEarned: answersTable.pointsEarned,
 answeredAt: answersTable.answeredAt,
 correctAnswer: questionsTable.correctAnswer,
 })
 .from(answersTable)
 .innerJoin(questionsTable, eq(answersTable.questionId, questionsTable.id))
 .where(
 and(
 eq(answersTable.gameId, params.data.gameId),
 eq(answersTable.userId, params.data.userId),
),
)
 .orderBy(asc(answersTable.answeredAt));

```

```

type AnswerRow = {
 id: number; userId: number; gameId: number; questionId: number;
 userAnswer: string; isCorrect: boolean; pointsEarned: number;
 answeredAt: string; correctAnswer: string;
}

```

```

};

const safeRows = (toJsonSafe(rows) as AnswerRow[]).map((row) =>
 gameCompleted ? row : { ...row, correctAnswer: undefined },
);

res.json(ListUserAnswersResponse.parse(safeRows));
},
);

export default router;

```

```

` ` `

```

```

```

```

`artifacts/api-server/src/routes/results.ts`

```

```

` ` ` typescript

```

```

import { Router, type IRouter } from "express";
import { eq, and, sql, asc } from "drizzle-orm";
import { requireAdmin } from "../middleware/requireAdmin";
import { requireAuth } from "../middleware/requireAuth";
import {
 db,
 gamesTable,
 usersTable,

```

```
questionsTable,
answersTable,
gameParticipantsTable,
} from "@workspace/db";
import { toJsonSafe } from "../lib/serialize";
```

```
const router: IRouter = Router();
```

```
// — GET /games/:gameId/results —————
```

```
// Returns leaderboard enriched with per-user correct/total counts
```

```
router.get("/games/:gameId/results", requireAuth, async (req, res): Promise<void> => {
 const gameId = parseInt(String(req.params.gameId ?? ""), 10);
 if (isNaN(gameId)) { res.status(400).json({ error: "Invalid gameId" }); return; }

 const [game] = await db.select().from(gamesTable).where(eq(gamesTable.id, gameId));
 if (!game) { res.status(404).json({ error: "Game not found" }); return; }
```

```
// Participants ordered by score desc
```

```
const participants = await db
 .select({
 id: gameParticipantsTable.id,
 userId: gameParticipantsTable.userId,
 userName: usersTable.name,
 totalScore: gameParticipantsTable.totalScore,
 joinedAt: gameParticipantsTable.joinedAt,
```

```

 })

 .from(gameParticipantsTable)

 .innerJoin(usersTable, eq(gameParticipantsTable.userId, usersTable.id))

 .where(eq(gameParticipantsTable.gameId, gameId))

 .orderBy(sql` ${gameParticipantsTable.totalScore} DESC`,
asc(gameParticipantsTable.joinedAt));

// Aggregate correct / total per user
const answerStats = await db

 .select({

 userId: answersTable.userId,

 totalAnswered: sql<number>` count(*)::int`,

 correctCount: sql<number>` sum(case when ${answersTable.isCorrect} then 1 else 0
end)::int`,

 })

 .from(answersTable)

 .where(eq(answersTable.gameId, gameId))

 .groupBy(answersTable.userId);

const statsMap = new Map(answerStats.map((s) => [s.userId, s]));

const enriched = participants.map((p, i) => {

 const stats = statsMap.get(p.userId);

 return {

 ...p,

 rank: i + 1,

 correctCount: stats?.correctCount ?? 0,

```

```
 totalAnswered: stats?.totalAnswered ?? 0,
 };
});
```

```
const totalQuestions = await db
 .select({ count: sql<number>` count(*)::int` })
 .from(questionsTable)
 .where(eq(questionsTable.gameId, gameId))
 .then((r) => r[0]?.count ?? 0);
```

```
res.json(toJsonSafe({ game, participants: enriched, totalQuestions }));
});
```

```
// — GET /games/:gameId/questions/stats —————
```

```
// Per-question analytics: correct count, % who got it right (admin use)
```

```
router.get("/games/:gameId/questions/stats", requireAdmin, async (req, res):
Promise<void> => {
 const gameId = parseInt(String(req.params.gameId ?? ""), 10);
 if (isNaN(gameId)) { res.status(400).json({ error: "Invalid gameId" }); return; }
```

```
const questions = await db
 .select({
 id: questionsTable.id,
 questionText: questionsTable.questionText,
 questionType: questionsTable.questionType,
```

```
 points: questionsTable.points,
 orderIndex: questionsTable.orderIndex,
 })
 .from(questionsTable)
 .where(eq(questionsTable.gameId, gameId))
 .orderBy(asc(questionsTable.orderIndex));
```

```
const answerStats = await db
 .select({
 questionId: answersTable.questionId,
 totalAnswered: sql<number>` count(*)::int`,
 correctCount: sql<number>` sum(case when ${answersTable.isCorrect} then 1 else 0
end)::int`,
 })
 .from(answersTable)
 .where(eq(answersTable.gameId, gameId))
 .groupBy(answersTable.questionId);
```

```
const statsMap = new Map(answerStats.map((s) => [s.questionId, s]));
```

```
const result = questions.map((q) => {
 const s = statsMap.get(q.id);
 const totalAnswered = s?.totalAnswered ?? 0;
 const correctCount = s?.correctCount ?? 0;
 return {
 ...q,
```



```

 totalAnswered,
 correctCount,
 percentCorrect: totalAnswered > 0 ? Math.round((correctCount / totalAnswered) * 100) :
null,
 };
});

res.json(result);
});

// — GET /games/:gameId/results/export.csv —————
// CSV download for admin

router.get("/games/:gameId/results/export.csv", requireAdmin, async (req, res):
Promise<void> => {
 const gameId = parseInt(req.params["gameId"] as string ?? "", 10);
 if (isNaN(gameId)) { res.status(400).json({ error: "Invalid gameId" }); return; }

 const [game] = await db.select().from(gamesTable).where(eq(gamesTable.id, gameId));
 if (!game) { res.status(404).json({ error: "Game not found" }); return; }

 const participants = await db
 .select({
 userId: gameParticipantsTable.userId,
 userName: usersTable.name,
 totalScore: gameParticipantsTable.totalScore,
 })

```

```
.from(gameParticipantsTable)
.innerJoin(usersTable, eq(gameParticipantsTable.userId, usersTable.id))
.where(eq(gameParticipantsTable.gameId, gameId))
.orderBy(sql` ${gameParticipantsTable.totalScore} DESC`);
```

```
const answerStats = await db
 .select({
 userId: answersTable.userId,
 totalAnswered: sql<number>` count(*)::int`,
 correctCount: sql<number>` sum(case when ${answersTable.isCorrect} then 1 else 0
end)::int`,
 })
 .from(answersTable)
 .where(eq(answersTable.gameId, gameId))
 .groupBy(answersTable.userId);
```

```
const totalQuestions = await db
 .select({ count: sql<number>` count(*)::int` })
 .from(questionsTable)
 .where(eq(questionsTable.gameId, gameId))
 .then((r) => r[0]?.count ?? 0);
```

```
const statsMap = new Map(answerStats.map((s) => [s.userId, s]));
```

```
const escapeCsv = (v: string | number) => `${String(v).replace(/"/g, '"')}`;
```

```

const header = ["Rank", "Player Name", "Total Score", "Correct", "Total Questions",
"Accuracy %"].map(escapeCsv).join(",");

const rows = participants.map((p, i) => {

 const s = statsMap.get(p.userId);

 const correct = s?.correctCount ?? 0;

 const accuracy = totalQuestions > 0 ? Math.round((correct / totalQuestions) * 100) : 0;

 return [i + 1, p.userName, p.totalScore, correct, totalQuestions,
accuracy].map(escapeCsv).join(",");

});

```

```

const csv = [header, ...rows].join("\n");

const filename = `${game.topic.replace(/^[^a-z0-9]/gi, "_")} _results.csv`;

```

```

res.setHeader("Content-Type", "text/csv");

res.setHeader("Content-Disposition", `attachment; filename="${filename}"`);

res.send(csv);

});

```

```

export default router;

```

```

` ` `

```

```

```

```

`artifacts/api-server/src/routes/gemini.ts`

```

```

` ` `typescript

```

```
import { Router, type IRouter } from "express";

import { and, eq } from "drizzle-orm";

import { requireAdmin } from "../middleware/requireAdmin";

import {
 geminiGenerateRateLimit,
 geminiOperationRateLimit,
} from "../middleware/providerRateLimit";

import { db, gamesTable, questionsTable } from "@workspace/db";

import {
 GenerateGeminiQuestionsBody,
 GenerateGeminiQuestionsParams,
 RegenerateQuestionBody,
 RegenerateQuestionParams,
 EnhanceQuestionParams,
 FactCheckQuestionParams,
} from "@workspace/api-zod";

import {
 generateGeminiQuestions,
 filterValidImageQuestions,
 regenerateSingleQuestion,
 enhanceQuestion,
 factCheckSingleQuestion,
} from "../services/geminiApi";

import { logger } from "../lib/logger";

const router: IRouter = Router();
```

```

async function syncQuestionCount(gameId: number): Promise<void> {
 const rows = await db
 .select({ id: questionsTable.id })
 .from(questionsTable)
 .where(eq(questionsTable.gameId, gameId));
 await db
 .update(gamesTable)
 .set({ questionCount: rows.length })
 .where(eq(gamesTable.id, gameId));
}

```

```

function geminiErrorResponse(
 res: Parameters<Parameters<IRouter["post"]>[1]>[1],
 error: { code: string; message?: string },
): void {
 if (error.code === "no_api_key") {
 res.status(503).json({
 error: "Google Gemini API key not configured. Add GOOGLE_API_KEY to your
environment secrets.",
 });
 return;
 }
 const msg = (error as { message?: string }).message ?? error.code;
 const kind = (error as { kind?: string }).kind;
 if (kind === "rate_limit_minute" || kind === "rate_limit_daily") {

```

```
 res.status(429).json({ error: msg });
 return;
}
if (kind === "model_unavailable") {
 res.status(503).json({ error: msg });
 return;
}
res.status(502).json({ error: ` Gemini error: ${msg} ` });
}
```

// — Bulk generate questions

---

```
router.post(
 "/games/:gameId/questions/generate-gemini",
 requireAdmin,
 geminiGenerateRateLimit,
 async (req, res): Promise<void> => {
 const params = GenerateGeminiQuestionsParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: "Invalid game ID" });
 return;
 }
 }
```

```
 const body = GenerateGeminiQuestionsBody.safeParse(req.body);
 if (!body.success) {
 res.status(400).json({ error: body.error.message });
 }
```

```
 return;
 }
```

```
const [game] = await db
 .select()
 .from(gamesTable)
 .where(eq(gamesTable.id, params.data.gameId));
```

```
if (!game) {
 res.status(404).json({ error: "Game not found" });
 return;
}
```

```
const result = await generateGeminiQuestions({
 topic: body.data.topic,
 difficulty: body.data.difficulty,
 amount: body.data.amount,
 existingQuestions: body.data.existingQuestions,
});
```

```
if (!result.ok) {
 logger.warn({ error: result.error }, "Gemini generation failed");
 geminiErrorResponse(res, result.error);
 return;
}
```

```
if (result.questions.length === 0) {
 res.status(422).json({ error: "Gemini returned no valid questions. Try a different topic or
retry." });
 return;
}
```

```
// Drop image questions whose URLs don't actually resolve to an image
const questions = await filterValidImageQuestions(result.questions);
if (questions.length === 0) {
 res.status(422).json({ error: "Gemini returned no valid questions. Try a different topic or
retry." });
 return;
}
```

```
const existing = await db
 .select({ orderIndex: questionsTable.orderIndex })
 .from(questionsTable)
 .where(eq(questionsTable.gameId, game.id));
```

```
const maxOrder = existing.length > 0
 ? Math.max(...existing.map((r) => r.orderIndex))
 : -1;
```

```
const toInsert = questions.map((q, i) => ({
 gameId: game.id,
 questionText: q.questionText,
 questionType: q.questionType,
```



```
correctAnswer: q.correctAnswer,
options: q.options as Record<string, unknown> | null,
imageUrl: q.imageUrl,
points: q.points,
orderIndex: maxOrder + 1 + i,
source: q.source,
aiGenerated: q.aiGenerated,
verifiedByAdmin: q.verifiedByAdmin,
));
```

```
await db.insert(questionsTable).values(toInsert);
await syncQuestionCount(game.id);
```

```
const distinctTypes = new Set(questions.map((q) => q.questionType));
if (body.data.amount >= 5 && distinctTypes.size < 3) {
 req.log.warn(
 { gameId: game.id, types: [...distinctTypes], requested: body.data.amount },
 "Generated questions lack variety (fewer than 3 question types)",
);
}
```

```
res.json({ imported: questions.length, total: result.questions.length });
},
);
```

```
// — Regenerate single question (preview only, not saved) —————
```

```

router.post(
 "/games/:gameId/questions/:questionId/regenerate",
 requireAdmin,
 geminiOperationRateLimit,
 async (req, res): Promise<void> => {
 const params = RegenerateQuestionParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: "Invalid IDs" });
 return;
 }

 const body = RegenerateQuestionBody.safeParse(req.body);
 if (!body.success) {
 res.status(400).json({ error: body.error.message });
 return;
 }

 const [question] = await db
 .select()
 .from(questionsTable)
 .where(
 and(
 eq(questionsTable.id, params.data.questionId),
 eq(questionsTable.gameId, params.data.gameId),
),

```

```
);
```

```
if (!question) {
 res.status(404).json({ error: "Question not found" });
 return;
}
```

```
const [game] = await db
 .select()
 .from(gamesTable)
 .where(eq(gamesTable.id, params.data.gameId));
```

```
if (!game) {
 res.status(404).json({ error: "Game not found" });
 return;
}
```

```
const difficulty =
 body.data.difficulty ?? (game.difficulty as "easy" | "medium" | "hard") ?? "medium";
```

```
const questionType =
 (body.data.questionType as "multiple_choice" | "true_false" | "write_in" | undefined) ??
 (question.questionType as "multiple_choice" | "true_false" | "write_in");
```

```
const result = await regenerateSingleQuestion({
 topic: game.topic,
```

```
difficulty,
questionType,
avoidText: question.questionText,
points: question.points,
});
```

```
if (!result.ok) {
 logger.warn({ error: result.error }, "Gemini regenerate failed");
 geminiErrorResponse(res, result.error);
 return;
}
```

```
res.json({
 questionType: result.question.questionType,
 questionText: result.question.questionText,
 correctAnswer: result.question.correctAnswer,
 options: result.question.options,
 points: result.question.points,
 source: result.question.source,
});
},
);
```

```
// — Enhance question (suggestions only, not saved) —————
```

```
router.post(
```

```

"/games/:gameId/questions/:questionId/enhance",
requireAdmin,
geminiOperationRateLimit,
async (req, res): Promise<void> => {
 const params = EnhanceQuestionParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: "Invalid IDs" });
 return;
 }

 const [question] = await db
 .select()
 .from(questionsTable)
 .where(
 and(
 eq(questionsTable.id, params.data.questionId),
 eq(questionsTable.gameId, params.data.gameId),
),
);

 if (!question) {
 res.status(404).json({ error: "Question not found" });
 return;
 }

 const choices =

```

```
(question.options as { choices?: string[] } | null)?.choices ?? [];
```

```
const result = await enhanceQuestion({
 questionType: question.questionType,
 questionText: question.questionText,
 correctAnswer: question.correctAnswer,
 options: choices,
 source: question.source,
});
```

```
if (!result.ok) {
 logger.warn({ error: result.error }, "Gemini enhance failed");
 geminiErrorResponse(res, result.error);
 return;
}
```

```
res.json({
 improvedQuestionText: result.data.improvedQuestionText,
 improvedOptions: result.data.improvedOptions,
 factCheckResult: result.data.factCheckResult,
 factCheckNotes: result.data.factCheckNotes,
 suggestedSource: result.data.suggestedSource,
 suggestions: result.data.suggestions,
});
},
);
```

// — Fact-check single question

---

```
router.post(
 "/games/:gameId/questions/:questionId/fact-check",
 requireAdmin,
 geminiOperationRateLimit,
 async (req, res): Promise<void> => {
 const params = FactCheckQuestionParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: "Invalid IDs" });
 return;
 }

 const [question] = await db
 .select()
 .from(questionsTable)
 .where(
 and(
 eq(questionsTable.id, params.data.questionId),
 eq(questionsTable.gameId, params.data.gameId),
),
);

 if (!question) {
 res.status(404).json({ error: "Question not found" });
 }
 }
);
```

```
 return;
 }

 const result = await factCheckSingleQuestion({
 questionText: question.questionText,
 correctAnswer: question.correctAnswer,
 });

 if (!result.ok) {
 logger.warn({ error: result.error }, "Gemini fact-check failed");
 geminiErrorResponse(res, result.error);
 return;
 }

 res.json({
 verdict: result.data.verdict,
 confidence: result.data.confidence,
 explanation: result.data.explanation,
 correctAnswerIfWrong: result.data.correctAnswerIfWrong,
 });
},
);

export default router;

...

```



---

## `artifacts/api-server/src/routes/opentdb.ts`

```typescript

```
import { Router, type IRouter } from "express";

import { eq } from "drizzle-orm";

import { db, gamesTable, questionsTable } from "@workspace/db";

import {
  ImportOpenTdbQuestionsParams,
  ImportOpenTdbQuestionsBody,
  ImportOpenTdbQuestionsResponse,
} from "@workspace/api-zod";

import { fetchOpenTdbQuestions } from "../services/triviaApi";

import { requireAdmin } from "../middleware/requireAdmin";

import { opentdbRateLimit } from "../middleware/providerRateLimit";
```

```
const router: IRouter = Router();
```

```
async function syncQuestionCount(gameId: number): Promise<void> {
  const rows = await db
    .select({ id: questionsTable.id })
    .from(questionsTable)
    .where(eq(questionsTable.gameId, gameId));
  await db
```

```
.update(gamesTable)
.set({ questionCount: rows.length })
.where(eq(gamesTable.id, gameId));
}
```

```
router.post(
  "/games/:gameId/questions/import-opentdb",
  requireAdmin,
  opentdbRateLimit,
  async (req, res): Promise<void> => {
    const params = ImportOpenTdbQuestionsParams.safeParse(req.params);
    if (!params.success) {
      res.status(400).json({ error: params.error.message });
      return;
    }
  }
```

```
const body = ImportOpenTdbQuestionsBody.safeParse(req.body);
if (!body.success) {
  res.status(400).json({ error: body.error.message });
  return;
}
```

```
const [game] = await db
  .select()
  .from(gamesTable)
  .where(eq(gamesTable.id, params.data.gameId));
```

```
if (!game) {  
  res.status(404).json({ error: "Game not found" });  
  return;  
}
```

```
const result = await fetchOpenTdbQuestions({  
  amount: body.data.amount,  
  categoryId: body.data.categoryId,  
  difficulty: body.data.difficulty,  
});
```

```
if (!result.ok) {  
  if (result.error.code === "rate_limited") {  
    res.status(429).json({  
      error:  
        "Open Trivia Database rate limit reached. Please wait a few seconds and try again.",  
    });  
    return;  
  }
```

```
  if (result.error.code === "no_results") {  
    res.status(422).json({  
      error:  
        "No questions available for this category/difficulty combination. Try a different  
difficulty.",  
    });  
  }
```

```

    return;
}
res.status(502).json({
    error: "Could not reach Open Trivia Database. Please try again.",
});
return;
}

if (result.questions.length === 0) {
    res.json(ImportOpenTdbQuestionsResponse.parse({ imported: 0, total: 0 }));
    return;
}

const toInsert = result.questions.map((q) => ({
    gameId: game.id,
    questionText: q.questionText,
    questionType: q.questionType,
    correctAnswer: q.correctAnswer,
    options: q.options as Record<string, unknown> | null,
    imageUrl: null,
    points: q.points,
    orderIndex: q.orderIndex,
    source: q.source,
}));

await db.insert(questionsTable).values(toInsert);

```

```
await syncQuestionCount(game.id);
```

```
res.json(  
  ImportOpenTdbQuestionsResponse.parse({  
    imported: result.questions.length,  
    total: result.questions.length,  
  }),  
);  
},  
);
```

```
export default router;
```

```
````
```

```

```

```
`artifacts/api-server/src/routes/stats.ts`
```

```
```typescript
```

```
import { Router, type IRouter } from "express";
```

```
import { eq, count } from "drizzle-orm";
```

```
import {
```

```
  db,
```

```
  gamesTable,
```

```
  usersTable,
```

```

    answersTable,
  } from "@workspace/db";
import { GetStatsSummaryResponse } from "@workspace/api-zod";
import { requireAdmin } from "../middleware/requireAdmin";

const router: IRouter = Router();

router.get("/stats/summary", requireAdmin, async (_req, res): Promise<void> => {
  const [totalGames] = await db.select({ value: count() }).from(gamesTable);
  const [activeGames] = await db
    .select({ value: count() })
    .from(gamesTable)
    .where(eq(gamesTable.status, "active"));
  const [totalPlayers] = await db.select({ value: count() }).from(usersTable);
  const [totalAnswers] = await db
    .select({ value: count() })
    .from(answersTable);

  res.json(
    GetStatsSummaryResponse.parse({
      totalGames: totalGames?.value ?? 0,
      activeGames: activeGames?.value ?? 0,
      totalPlayers: totalPlayers?.value ?? 0,
      totalAnswers: totalAnswers?.value ?? 0,
    }),
  );

```

```
});
```

```
export default router;
```

```
` `` `
```

```
---
```

```
## `artifacts/api-server/src/routes/settings.ts`
```

```
` `` `typescript
```

```
import { Router, type IRouter } from "express";
```

```
import { eq } from "drizzle-orm";
```

```
import { db, adminSettingsTable } from "@workspace/db";
```

```
import { requireAdmin } from "../middleware/requireAdmin";
```

```
const router: IRouter = Router();
```

```
// GET /api/settings — return current access codes (masked) for admins
```

```
router.get("/settings", requireAdmin, async (req, res): Promise<void> => {
```

```
  const [row] = await db.select().from(adminSettingsTable).limit(1);
```

```
  if (!row) {
```

```
    res.status(404).json({ error: "Settings not found" });
```

```
    return;
```

```
  }
```

```
  res.json({
```

```
triviaAccessCode: row.triviaAccessCode,  
adminAccessCode: row.adminAccessCode,  
});  
});
```

```
// PATCH /api/settings — update access codes
```

```
router.patch("/settings", requireAdmin, async (req, res): Promise<void> => {  
  const [existing] = await db.select().from(adminSettingsTable).limit(1);  
  if (!existing) {  
    res.status(404).json({ error: "Settings not found" });  
    return;  
  }  
}
```

```
const triviaCode =  
  typeof req.body?.triviaAccessCode === "string"  
    ? req.body.triviaAccessCode.trim()  
    : null;  
const adminCode =  
  typeof req.body?.adminAccessCode === "string"  
    ? req.body.adminAccessCode.trim()  
    : null;
```

```
if (triviaCode !== null && triviaCode.length < 4) {  
  res  
    .status(400)  
    .json({ error: "Trivia access code must be at least 4 characters" });  
}
```



```

    return;
}

if (adminCode !== null && adminCode.length < 4) {
    res
        .status(400)
        .json({ error: "Admin access code must be at least 4 characters" });
    return;
}

if (triviaCode !== null && adminCode !== null && triviaCode === adminCode) {
    res
        .status(400)
        .json({ error: "Trivia and admin codes must be different" });
    return;
}

const [updated] = await db
    .update(adminSettingsTable)
    .set({
        ...(triviaCode !== null && { triviaAccessCode: triviaCode }),
        ...(adminCode !== null && { adminAccessCode: adminCode }),
    })
    .where(eq(adminSettingsTable.id, existing.id))
    .returning();

res.json({
    triviaAccessCode: updated!.triviaAccessCode,

```

```
    adminAccessCode: updated!.adminAccessCode,
  });
});

export default router;

` ``

---

## `artifacts/api-server/src/routes/health.ts`

` `` `typescript
import { Router, type IRouter } from "express";
import { HealthCheckResponse } from "@workspace/api-zod";

const router: IRouter = Router();

router.get("/healthz", (_req, res) => {
  const data = HealthCheckResponse.parse({ status: "ok" });
  res.json(data);
});

export default router;

` ``
```

`artifacts/api-server/src/routes/users.ts`

```typescript

```
import { Router, type IRouter } from "express";

import { eq } from "drizzle-orm";

import { db, usersTable } from "@workspace/db";

import {
 CreateUserBody,
 CreateUserResponse,
 GetUserParams,
 GetUserResponse,
} from "@workspace/api-zod";

import { toJsonSafe } from "../lib/serialize";

const router: IRouter = Router();

router.post("/users", async (req, res): Promise<void> => {
 const parsed = CreateUserBody.safeParse(req.body);
 if (!parsed.success) {
 res.status(400).json({ error: parsed.error.message });
 return;
 }
}
```

```
const [user] = await db
 .insert(usersTable)
 .values({ name: parsed.data.name.trim() })
 .returning();

res.status(201).json(CreateUserResponse.parse(toJsonSafe(user)));
});
```

```
router.get("/users/:userId", async (req, res): Promise<void> => {
 const params = GetUserParams.safeParse(req.params);
 if (!params.success) {
 res.status(400).json({ error: params.error.message });
 return;
 }
}
```

```
const [user] = await db
 .select()
 .from(usersTable)
 .where(eq(usersTable.id, params.data.userId));
```

```
if (!user) {
 res.status(404).json({ error: "User not found" });
 return;
}
```

```
res.json(GetUserResponse.parse(toJsonSafe(user)));
```

```
});
```

```
export default router;
```

```
```\n
```

```
---
```

```
## `artifacts/api-server/src/services/geminiApi.ts`
```

```
```typescript
```

```
import { logger } from "../lib/logger";
```

```
const GEMINI_MODELS = [
```

```
 "gemini-3.5-flash-lite", // lighter quota bucket — try first
```

```
 "gemini-3.5-flash", // fallback
```

```
];
```

```
function geminiUrl(model: string) {
```

```
 return
```

```
 `https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent`;
```

```
}
```

```
// — Shared types
```

---

```
export type GeminiErrorKind = "rate_limit_minute" | "rate_limit_daily" |
"model_unavailable";
```

```
export type GeminiGenerateError =
 | { code: "no_api_key" }
 | { code: "api_error"; message: string; kind?: GeminiErrorKind }
 | { code: "parse_error"; message: string }
 | { code: "fetch_failed"; message: string };
```

```
interface GeminiApiResponse {
 candidates?: Array<{
 content?: {
 parts?: Array<{ text?: string }>;
 };
 }>;
}
```

```
// — Shared helpers
```

---

```
function shuffleArray<T>(arr: T[]): T[] {
 const a = [...arr];
 for (let i = a.length - 1; i > 0; i--) {
 const j = Math.floor(Math.random() * (i + 1));
 [a[i]!, a[j]!] = [a[j]!, a[i]!];
 }
 return a;
}
```

```
}
```

```
type RateLimitKind = "per_minute" | "daily";
```

```
interface GeminiRawResult {
```

```
 ok: true;
```

```
 text: string;
```

```
}
```

```
interface GeminiRawError {
```

```
 ok: false;
```

```
 error: GeminiGenerateError;
```

```
 rateLimitKind?: RateLimitKind;
```

```
}
```

```
async function callGeminiRaw(
```

```
 apiKey: string,
```

```
 model: string,
```

```
 prompt: string,
```

```
 temperature = 0.4,
```

```
 maxTokens = 8192,
```

```
): Promise<GeminiRawResult | GeminiRawError> {
```

```
 let resp: Response;
```

```
 try {
```

```
 resp = await fetch(`${geminiUrl(model)}?key=${apiKey}`, {
```

```
 method: "POST",
```

```
 headers: { "Content-Type": "application/json" },
```

```

body: JSON.stringify({
 contents: [{ parts: [{ text: prompt }] }],
 generationConfig: { temperature, maxOutputTokens: maxTokens },
}),
});
} catch (err) {
 logger.warn({ err }, "Gemini API fetch failed");
 return { ok: false, error: { code: "fetch_failed", message: String(err) } };
}

if (resp.status === 429) {
 // Read body to distinguish per-minute vs daily quota exhaustion
 const body = await resp.text().catch(() => "");
 const hasZeroLimit = /limit:\s*0\b/.test(body);
 const isDaily =
 body.includes("per_day") ||
 body.includes("PerDay") ||
 body.includes("daily") ||
 body.includes("free_tier") ||
 body.includes("RESOURCE_EXHAUSTED");
 const rateLimitKind: RateLimitKind = isDaily || hasZeroLimit ? "daily" : "per_minute";
 logger.warn({ model, rateLimitKind, hasZeroLimit, body: body.slice(0, 400) }, "Gemini
429");
 return {
 ok: false,
 rateLimitKind,

```



```

error: {
 code: "api_error",
 kind: hasZeroLimit
 ? "model_unavailable"
 : isDaily
 ? "rate_limit_daily"
 : "rate_limit_minute",
 message: hasZeroLimit
 ? "This API key has no quota for this Gemini model (limit is 0) — the model may be
retired for new accounts, or the key's project lacks free-tier access."
 : isDaily
 ? "Gemini daily quota exhausted. Please try again after midnight Pacific time."
 : "Too many requests. Please wait a moment and try again.",
},
};
}

```

```

if (!resp.ok) {
 const body = await resp.text().catch(() => "");
 logger.warn({ status: resp.status, body: body.slice(0, 300) }, "Gemini API error");
 return { ok: false, error: { code: "api_error", message: `HTTP ${resp.status}` } };
}

```

```

const data = (await resp.json()) as GeminiApiResponse;
const text = data.candidates?.[0]?.content?.parts?.[0]?.text ?? "";

```

```
if (!text) {
 return { ok: false, error: { code: "parse_error", message: "Empty response from Gemini" } };
}
```

```
 return { ok: true, text };
}
```

```
function stripFences(text: string): string {
 return text
 .replace(/^` ``` (?::json)?\s*/i, "")
 .replace(/\s*^` ``` \s*$/i, "")
 .trim();
}
```

```
// — Bulk generation —
```

---

```
export type GeminiQuestionType =
 | "multiple_choice"
 | "true_false"
 | "write_in"
 | "matching"
 | "image_recognition";
```

```
export interface GeminiQuestion {
 questionText: string;
 questionType: GeminiQuestionType;
```

```
correctAnswer: string;
options:
 | { choices: string[] }
 | { pairs: { left: string; right: string }[] }
 | { alternateAnswers: string[] }
 | null;
imageUrl: string | null;
points: number;
orderIndex: number;
source: string;
aiGenerated: true;
verifiedByAdmin: false;
}
```

```
export type GeminiGenerateResult =
 | { ok: true; questions: GeminiQuestion[] }
 | { ok: false; error: GeminiGenerateError };
```

```
export interface GeminiGenerateOptions {
 topic: string;
 difficulty: "easy" | "medium" | "hard";
 amount: number;
 existingQuestions?: string[];
}
```

```
interface RawGeminiQuestion {
```

```
question_type?: unknown;
question_text?: unknown;
correct_answer?: unknown;
options?: unknown;
acceptable_answers?: unknown;
left_items?: unknown;
right_items?: unknown;
correct_pairs?: unknown;
image_url?: unknown;
points?: unknown;
source?: unknown;
}
```

```
const VALID_TYPES = new Set([
 "multiple_choice",
 "true_false",
 "write_in",
 "matching",
 "image_recognition",
]);
```

```
function buildBulkPrompt(opts: GeminiGenerateOptions): string {
 const avoid =
 opts.existingQuestions && opts.existingQuestions.length > 0
 ? ` \nAVOID duplicating these existing question
topics:\n${opts.existingQuestions.map((q, i) => `${i + 1}. ${q}`).join("\n")}`
```

```
: "";
```

```
// Compute exact counts so the model cannot ignore the mix requirement.
```

```
// Target distribution: 40% MC, 20% TF, 20% write-in, 10% matching, 10% image
recognition.
```

```
// Types are added in priority order so the counts always sum exactly to `total`.
```

```
const total = opts.amount;
```

```
let remaining = total;
```

```
const take = (target: number, enabled = true): number => {
```

```
 if (!enabled || remaining <= 0) return 0;
```

```
 const n = Math.min(remaining, Math.max(1, Math.round(total * target)));
```

```
 remaining -= n;
```

```
 return n;
```

```
};
```

```
const matchCount = take(0.1, total >= 6);
```

```
const imgCount = take(0.1, total >= 6);
```

```
const tfCount = take(0.2, total >= 2);
```

```
const wiCount = take(0.2, total >= 3);
```

```
const mcCount = remaining; // everything left; >= 1 for total >= 1 given the caps above
```

```
const mcPoints = opts.difficulty === "easy" ? 5 : opts.difficulty === "hard" ? 15 : 10;
```

```
const matchingSpec = matchCount > 0
```

```
 ? ` - ${matchCount} questions with "question_type": "matching" (4 related pairs to
connect; points: 20)\n`
```

```
 : "";
```

```
const imageSpec = imgCount > 0
```

? ` - \${imgCount} questions with "question\_type": "image\_recognition" (real Wikimedia Commons image URL; points: 15)\n`  
: "";

return `You are a trivia question writer creating a fun, varied quiz. Generate exactly \${total} trivia questions about "\${opts.topic}" at \${opts.difficulty} difficulty level.\${avoid}

#### CRITICAL RULES:

1. Every question and answer MUST be based on verifiable, real-world facts.
2. Include the factual source for each answer (e.g., "Wikipedia: Eiffel Tower" or "Nobel Prize official records, 1921").
3. Do not invent any facts, statistics, dates, names, or events.
4. If you are unsure about a fact, do not include that question.
5. Prefer well-documented historical facts, scientific facts, and publicly verifiable information.
6. Avoid recent events from the last 2 years that may have uncertain details.
7. All wrong answers for multiple choice must be plausible but definitively incorrect.
8. Mix the question types throughout the array — do not group all of one type together.

YOU MUST produce EXACTLY this breakdown — no deviations:

- \${mcCount} questions with "question\_type": "multiple\_choice" (4 options; shuffle so correct answer is NOT always first; points: \${mcPoints})

- \${tfCount} questions with "question\_type": "true\_false" (options must be ["true","false"]; correct\_answer must be "true" or "false"; points: 5)

- \${wiCount} questions with "question\_type": "write\_in" (short factual answer — name, date, or place; points: 15)

\${matchingSpec}\${imageSpec}

QUESTION TYPE FORMATS (use these exact JSON structures):

multiple\_choice:

```
{ "question_type": "multiple_choice", "question_text": "The question?", "correct_answer":
"Correct Answer", "options": ["Wrong 1", "Correct Answer", "Wrong 2", "Wrong 3"], "points":
${mcPoints}, "source": "Wikipedia: Article Name" }
```

true\_false:

```
{ "question_type": "true_false", "question_text": "Statement to evaluate.",
"correct_answer": "true", "options": ["true", "false"], "points": 5, "source": "Source name" }
```

write\_in:

```
{ "question_type": "write_in", "question_text": "Question requiring a short typed answer?",
"correct_answer": "Answer", "acceptable_answers": ["Answer", "Alternate spelling"],
"points": 15, "source": "Source name" }
```

`${matchCount > 0 ? ``

matching (exactly 4 pairs; right\_items are the left\_items' matches in scrambled order):

```
{ "question_type": "matching", "question_text": "Match each country to its capital:",
"left_items": ["France", "Japan", "Brazil", "Egypt"], "right_items": ["Tokyo", "Paris", "Cairo",
"Brasilia"], "correct_pairs": [{"left": "France", "right": "Paris"}, {"left": "Japan", "right":
"Tokyo"}, {"left": "Brazil", "right": "Brasilia"}, {"left": "Egypt", "right": "Cairo"}], "points": 20,
"source": "World Atlas" }
```

`` : ""} ${imgCount > 0 ? ``

image\_recognition (ONLY real, well-known Wikimedia Commons URLs of famous subjects — the URL must actually exist; if you cannot recall a real URL exactly, use a different famous subject you are sure about):

```
{ "question_type": "image_recognition", "question_text": "Name this famous landmark:",
"image_url":
"https://upload.wikimedia.org/wikipedia/commons/a/a8/Tour_Eiffel_Wikimedia_Commons
.jpg", "correct_answer": "Eiffel Tower", "acceptable_answers": ["Eiffel Tower", "The Eiffel
Tower", "Tour Eiffel"], "points": 15, "source": "Wikimedia Commons" }
```

`` : ""}`

Return ONLY a valid JSON array with no other text, no markdown, no code fences.

Topic: \${opts.topic}

Difficulty: \${opts.difficulty}

Total questions required: \${total}` ;

}

```
function parseQuestions(raw: unknown, opts: GeminiGenerateOptions): GeminiQuestion[]
{
```

```
 if (!Array.isArray(raw)) return [];
```

```
 const results: GeminiQuestion[] = [];
```

```
 for (let i = 0; i < raw.length; i++) {
```

```
 const item = raw[i] as RawGeminiQuestion;
```

```
 if (!item || typeof item !== "object") continue;
```

```
 const questionType = typeof item.question_type === "string" ? item.question_type.trim() :
 null;
```

```
 const questionText = typeof item.question_text === "string" ? item.question_text.trim() :
 null;
```

```
 const correctAnswer = typeof item.correct_answer === "string" ?
 item.correct_answer.trim() : null;
```

```
 const sourceCitation =
```

```
 typeof item.source === "string" && item.source.trim()
```

```
 ? item.source.trim()
```

```
 : `AI Generated: ${opts.topic}` ;
```



```
if (!questionType || !VALID_TYPES.has(questionType) || !questionText) continue;

// Matching questions carry their answer in correct_pairs, not correct_answer
if (questionType === "matching") {
 const pairs = parseMatchingPairs(item);
 if (!pairs || pairs.length < 2) continue;

 // correctAnswer format used by the grader: "left:right|..." sorted alphabetically by left
 const answerString = [...pairs]
 .sort((a, b) => a.left.localeCompare(b.left))
 .map((p) => `${p.left}:${p.right}`)
 .join("|");
 results.push({
 questionText,
 questionType: "matching",
 correctAnswer: answerString,
 options: { pairs },
 imageUrl: null,
 points: 20,
 orderIndex: i,
 source: sourceCitation,
 aiGenerated: true,
 verifiedByAdmin: false,
 });
 continue;
}
```

```
if (!correctAnswer) continue;
```

```
if (questionType === "multiple_choice") {
```

```
 const rawOpts = Array.isArray(item.options) ? (item.options as unknown[]) : null;
```

```
 if (!rawOpts || rawOpts.length < 2) continue;
```

```
 const choices = rawOpts
```

```
 .map((c) => String(c).trim())
```

```
 .filter(Boolean)
```

```
 .slice(0, 4);
```

```
 if (!choices.includes(correctAnswer)) {
```

```
 choices[0] = correctAnswer;
```

```
 }
```

```
 const points =
```

```
 typeof item.points === "number"
```

```
 ? item.points
```

```
 : opts.difficulty === "easy"
```

```
 ? 5
```

```
 : opts.difficulty === "hard"
```

```
 ? 15
```

```
 : 10;
```

```
 results.push({
```

```
 questionText,
 questionType: "multiple_choice",
 correctAnswer,
 options: { choices: shuffleArray(choices) },
 imageUrl: null,
 points,
 orderIndex: i,
 source: sourceCitation,
 aiGenerated: true,
 verifiedByAdmin: false,
 });
} else if (questionType === "true_false") {
 const answer = correctAnswer.toLowerCase() === "false" ? "false" : "true";
 results.push({
 questionText,
 questionType: "true_false",
 correctAnswer: answer,
 options: null,
 imageUrl: null,
 points: 5,
 orderIndex: i,
 source: sourceCitation,
 aiGenerated: true,
 verifiedByAdmin: false,
 });
} else if (questionType === "write_in") {
```

```
results.push({
 questionText,
 questionType: "write_in",
 correctAnswer,
 options: buildAlternates(item.acceptable_answers, correctAnswer),
 imageUrl: null,
 points: 15,
 orderIndex: i,
 source: sourceCitation,
 aiGenerated: true,
 verifiedByAdmin: false,
});
} else if (questionType === "image_recognition") {
 const imageUrl = normalizeWikimediaUrl(
 typeof item.image_url === "string" ? item.image_url.trim() : "",
);
 if (!imageUrl) continue;
 results.push({
 questionText,
 questionType: "image_recognition",
 correctAnswer,
 options: buildAlternates(item.acceptable_answers, correctAnswer),
 imageUrl,
 points: 15,
 orderIndex: i,
 source: sourceCitation,
```

```
 aiGenerated: true,
 verifiedByAdmin: false,
 });
}
}
```

```
return results;
}
```

```
function buildAlternates(
 raw: unknown,
 correctAnswer: string,
): { alternateAnswers: string[] } | null {
 if (!Array.isArray(raw)) return null;
 const alternates = raw
 .map((a) => String(a).trim())
 .filter((a) => a && a.toLowerCase() !== correctAnswer.toLowerCase());
 return alternates.length > 0 ? { alternateAnswers: alternates } : null;
}
```

```
function parseMatchingPairs(
 item: RawGeminiQuestion,
): { left: string; right: string }[] | null {
 if (!Array.isArray(item.correct_pairs)) return null;
 const pairs: { left: string; right: string }[] = [];
 for (const p of item.correct_pairs as unknown[]) {
```

```

 if (!p || typeof p !== "object") continue;

 const left = String((p as { left?: unknown }).left ?? "").trim();
 const right = String((p as { right?: unknown }).right ?? "").trim();

 if (left && right) pairs.push({ left, right });
 }

 return pairs.length > 0 ? pairs.slice(0, 5) : null;
}

/**
 * Only accept Wikimedia Commons URLs, and rewrite thumbnail URLs to the
 * original file URL (thumb paths frequently 400 for LLM-recalled sizes).
 * e.g. .../commons/thumb/a/a8/File.jpg/800px-File.jpg → .../commons/a/a8/File.jpg
 */
function normalizeWikimediaUrl(url: string): string | null {
 if (!/^https:\\\\upload\\.wikimedia\\.org\\/wikipedia\\/commons\\/.test(url)) return null;

 const thumbMatch = url.match(

/^https:\\\\upload\\.wikimedia\\.org\\/wikipedia\\/commons\\/thumb\\/([\\^]+V[\\^/]+V[\\^/]+)V[\\^/]+$/
+$/
);

 return thumbMatch ?
`https://upload.wikimedia.org/wikipedia/commons/${thumbMatch[1]}` : url;
}

/** Verify image URLs actually load; drop image questions with dead links. */
export async function filterValidImageQuestions(
 questions: GeminiQuestion[],

```

```

): Promise<GeminiQuestion[]> {
 const checks = await Promise.all(
 questions.map(async (q) => {
 if (q.questionType !== "image_recognition" || !q.imageUrl) return q;
 try {
 const resp = await fetch(q.imageUrl, {
 method: "GET",
 headers: {
 "User-Agent": "TriviaNightApp/1.0 (question image validation)",
 Range: "bytes=0-0",
 },
 signal: AbortSignal.timeout(5000),
 });
 const contentType = resp.headers.get("content-type") ?? "";
 if (resp.ok && contentType.startsWith("image/")) return q;
 logger.warn({ imageUrl: q.imageUrl, status: resp.status }, "Dropping image question: URL not a valid image");
 return null;
 } catch {
 logger.warn({ imageUrl: q.imageUrl }, "Dropping image question: URL unreachable");
 return null;
 }
 }),
);
 return checks.filter((q): q is GeminiQuestion => q !== null);
}

```

```
export async function generateGeminiQuestions(
 opts: GeminiGenerateOptions,
): Promise<GeminiGenerateResult> {
 const apiKey = process.env.GOOGLE_API_KEY;
 if (!apiKey) {
 return { ok: false, error: { code: "no_api_key" } };
 }

 const prompt = buildBulkPrompt(opts);
 let lastError: GeminiGenerateError = { code: "api_error", message: "Not attempted" };

 // Cascade through available models; within each model retry once on per-minute limits
 for (const model of GEMINI_MODELS) {
 let raw: GeminiRawResult | GeminiRawError = { ok: false, error: lastError };

 for (let attempt = 0; attempt < 2; attempt++) {
 raw = await callGeminiRaw(apiKey, model, prompt, 0.4, 8192);
 if (raw.ok) break;
 lastError = raw.error;
 const isPerMinute = !raw.ok && raw.rateLimitKind === "per_minute";
 if (!isPerMinute || attempt >= 1) break;
 const delay = 25000;
 logger.info({ model, attempt: attempt + 1, delay }, "Gemini per-minute limit, retrying");
 await new Promise((resolve) => setTimeout(resolve, delay));
 }
 }
}
```



```
if (raw.ok) {
 // success — fall through to parse
 let parsed: unknown;
 try {
 parsed = JSON.parse(stripFences(raw.text));
 } catch {
 logger.warn({ text: raw.text.slice(0, 500) }, "Failed to parse Gemini JSON response");
 return {
 ok: false,
 error: { code: "parse_error", message: "Invalid response format from Gemini. Please try
again." },
 };
 }
 const questions = parseQuestions(parsed, opts);
 if (questions.length === 0) {
 return {
 ok: false,
 error: { code: "parse_error", message: "No valid questions in Gemini response. Please
try again." },
 };
 }
 return { ok: true, questions };
}

// If this model hit a daily limit, no point trying it again — but try next model
const isDaily = !raw.ok && raw.rateLimitKind === "daily";
```

```
 logger.warn({ model, isDaily, error: raw.error }, "Gemini model failed, trying next");
 if (!isDaily) break; // non-rate-limit error — don't try further models
}
```

```
 return { ok: false, error: lastError };
}
```

```
// — Regenerate single question —————
```

```
export interface RegenerateOpts {
 topic: string;
 difficulty: "easy" | "medium" | "hard";
 questionType: "multiple_choice" | "true_false" | "write_in";
 avoidText: string;
 points: number;
}
```

```
export interface RegeneratePreview {
 questionType: "multiple_choice" | "true_false" | "write_in";
 questionText: string;
 correctAnswer: string;
 options: string[] | null;
 points: number;
 source: string;
}
```

```

export type RegenerateResult =
 | { ok: true; question: RegeneratePreview }
 | { ok: false; error: GeminiGenerateError };

export async function regenerateSingleQuestion(opts: RegenerateOpts):
Promise<RegenerateResult> {

 const apiKey = process.env.GOOGLE_API_KEY;

 if (!apiKey) return { ok: false, error: { code: "no_api_key" } };

 const prompt = `Generate exactly 1 trivia question about "${opts.topic}" at
${opts.difficulty} difficulty level.

```

The question type must be: \${opts.questionType}

#### CRITICAL RULES:

1. The question and answer MUST be based on verifiable, real-world facts
2. Include the factual source for the answer
3. Do not invent any facts, statistics, dates, names, or events
4. Make sure this question is DIFFERENT from: "\${opts.avoidText}"

Return ONLY valid JSON with no other text:

```

{
 "question_type": "${opts.questionType}",
 "question_text": "Your new question here?",
 "correct_answer": "Correct Answer",
 "options": ["Correct Answer", "Wrong 1", "Wrong 2", "Wrong 3"],
 "points": ${opts.points},

```

```
"source": "Source name"
}
```

For true\_false, options should be ["true", "false"].

For write\_in, options should be an empty array [].` ;

```
const raw = await callGeminiRaw(apiKey, GEMINI_MODELS[0]!, prompt, 0.5, 2048);
if (!raw.ok) return raw;
```

```
let parsed: unknown;

try {
 parsed = JSON.parse(stripFences(raw.text));
} catch {
 return { ok: false, error: { code: "parse_error", message: "Could not parse Gemini
response" } };
}
```

```
const q = parsed as RawGeminiQuestion;

const questionText = typeof q.question_text === "string" ? q.question_text.trim() : null;

const correctAnswer = typeof q.correct_answer === "string" ? q.correct_answer.trim() :
null;
```

```
const questionType = opts.questionType;

if (!questionText || !correctAnswer) {
 return { ok: false, error: { code: "parse_error", message: "Invalid question structure from
Gemini" } };
}
```

```

let options: string[] | null = null;

if (questionType === "multiple_choice" && Array.isArray(q.options)) {

 const choices = (q.options as unknown[])

 .map((c) => String(c).trim())

 .filter(Boolean)

 .slice(0, 4);

 if (!choices.includes(correctAnswer)) choices[0] = correctAnswer;

 options = shuffleArray(choices);

} else if (questionType === "true_false") {

 options = ["true", "false"];

}

const points = typeof q.points === "number" ? q.points : opts.points;

const source =

 typeof q.source === "string" && q.source.trim()

 ? q.source.trim()

 : `AI Generated: ${opts.topic}`;

return { ok: true, question: { questionType, questionText, correctAnswer, options, points,
source } };

}

```

// — Enhance question

---

```
export interface EnhanceOpts {
```

```
questionType: string;
questionText: string;
correctAnswer: string;
options: string[];
source?: string | null;
}
```

```
export interface EnhanceData {
 improvedQuestionText: string;
 improvedOptions: string[] | null;
 factCheckResult: "VERIFIED" | "UNCERTAIN" | "LIKELY_INCORRECT";
 factCheckNotes: string;
 suggestedSource: string;
 suggestions: string;
}
```

```
export type EnhanceResult =
 | { ok: true; data: EnhanceData }
 | { ok: false; error: GeminiGenerateError };
```

```
export async function enhanceQuestion(opts: EnhanceOpts): Promise<EnhanceResult> {
 const apiKey = process.env.GOOGLE_API_KEY;
 if (!apiKey) return { ok: false, error: { code: "no_api_key" } };
```

```
 const prompt = `Review this trivia question and suggest improvements:
```

Question Type: \${opts.questionType}  
Question: \${opts.questionText}  
Correct Answer: \${opts.correctAnswer}  
Options: \${opts.options.length > 0 ? opts.options.join(", ") : "N/A"}  
Current Source: \${opts.source || "None provided"}

Please analyze and return JSON with:

```
{
 "improved_question_text": "Clearer or better worded version of the question",
 "improved_options": ["Better distractor 1", "Better distractor 2", "Better distractor 3",
 "Correct Answer"],
 "fact_check_result": "VERIFIED" or "UNCERTAIN" or "LIKELY_INCORRECT",
 "fact_check_notes": "Explanation of verification, any concerns about accuracy",
 "suggested_source": "A reliable source that confirms this fact",
 "suggestions": "Any other suggestions to improve this question"
}
```

IMPORTANT:

- Verify the factual accuracy of the correct answer
- If the fact seems wrong or outdated, mark as LIKELY\_INCORRECT and explain why
- Suggest better wrong answers that are plausible but clearly incorrect
- Improve question clarity without changing the core fact being tested
- For true\_false or write\_in, improved\_options may be null`;

```
const raw = await callGeminiRaw(apiKey, GEMINI_MODELS[0]!, prompt, 0.3, 2048);
if (!raw.ok) return raw;
```

```

let parsed: Record<string, unknown>;

try {

 parsed = JSON.parse(stripFences(raw.text)) as Record<string, unknown>;

} catch {

 return { ok: false, error: { code: "parse_error", message: "Could not parse Gemini
response" } };

}

```

```

const fcRaw = String(parsed.fact_check_result ??
"UNCERTAIN").toUpperCase().replace(/s+/g, "_");

const factCheckResult: EnhanceData["factCheckResult"] =

 fcRaw === "VERIFIED" ? "VERIFIED" : fcRaw === "LIKELY_INCORRECT" ?
"LIKELY_INCORRECT" : "UNCERTAIN";

```

```

let improvedOptions: string[] | null = null;

if (Array.isArray(parsed.improved_options) && (parsed.improved_options as
unknown[]).length > 0) {

 improvedOptions = (parsed.improved_options as unknown[]).map((o) =>
String(o).trim()).filter(Boolean);

}

```

```

return {

 ok: true,

 data: {

 improvedQuestionText: String(parsed.improved_question_text ??
opts.questionText).trim(),

 improvedOptions,

```



```
 factCheckResult,
 factCheckNotes: String(parsed.fact_check_notes ?? "").trim(),
 suggestedSource: String(parsed.suggested_source ?? "").trim(),
 suggestions: String(parsed.suggestions ?? "").trim(),
 },
};
}
```

// — Fact-check single question

---

```
export interface FactCheckOpts {
 questionText: string;
 correctAnswer: string;
}
```

```
export interface FactCheckData {
 verdict: "CORRECT" | "INCORRECT" | "UNCERTAIN";
 confidence: "HIGH" | "MEDIUM" | "LOW";
 explanation: string;
 correctAnswerIfWrong: string | null;
}
```

```
export type FactCheckSingleResult =
 | { ok: true; data: FactCheckData }
 | { ok: false; error: GeminiGenerateError };
```

```

export async function factCheckSingleQuestion(opts: FactCheckOpts):
Promise<FactCheckSingleResult> {

 const apiKey = process.env.GOOGLE_API_KEY;

 if (!apiKey) return { ok: false, error: { code: "no_api_key" } };

 const prompt = `Verify this trivia fact:

Question: ${opts.questionText}

Stated Answer: ${opts.correctAnswer}

Is this factually correct? Return JSON only:

{
 "verdict": "CORRECT" or "INCORRECT" or "UNCERTAIN",
 "confidence": "HIGH" or "MEDIUM" or "LOW",
 "explanation": "Brief explanation of verification",
 "correct_answer_if_wrong": "The actual correct answer if the stated one is wrong, or null if
correct"
}`;

 const raw = await callGeminiRaw(apiKey, GEMINI_MODELS[0]!, prompt, 0.2, 1024);

 if (!raw.ok) return raw;

 let parsed: Record<string, unknown>;

 try {

 parsed = JSON.parse(stripFences(raw.text)) as Record<string, unknown>;

 } catch {

 return { ok: false, error: { code: "parse_error", message: "Could not parse Gemini
response" } };
 }

```

```
}
```

```
const verdictRaw = String(parsed.verdict ?? "UNCERTAIN").toUpperCase();
```

```
const verdict: FactCheckData["verdict"] =
```

```
 verdictRaw === "CORRECT" ? "CORRECT" : verdictRaw === "INCORRECT" ?
 "INCORRECT" : "UNCERTAIN";
```

```
const confidenceRaw = String(parsed.confidence ?? "LOW").toUpperCase();
```

```
const confidence: FactCheckData["confidence"] =
```

```
 confidenceRaw === "HIGH" ? "HIGH" : confidenceRaw === "MEDIUM" ? "MEDIUM" :
 "LOW";
```

```
return {
```

```
 ok: true,
```

```
 data: {
```

```
 verdict,
```

```
 confidence,
```

```
 explanation: String(parsed.explanation ?? "").trim(),
```

```
 correctAnswerIfWrong:
```

```
 parsed.correct_answer_if_wrong && parsed.correct_answer_if_wrong !== "null"
```

```
 ? String(parsed.correct_answer_if_wrong).trim()
```

```
 : null,
```

```
 },
```

```
};
```

```
}
```

```
...
```

---

## `artifacts/api-server/src/services/triviaApi.ts`

```typescript

import { logger } from "../lib/logger";

const OPENTDB_BASE = "https://opentdb.com";

// — HTML entity decoder —————

function decodeHtml(html: string): string {

 return html

 .replace(/&/g, "&")

 .replace(/</g, "<")

 .replace(/>/g, ">")

 .replace(/"/g, "")

 .replace(/'/g, "")

 .replace(/'/g, "")

 .replace(/“/g, "\u201C")

 .replace(/”/g, "\u201D")

 .replace(/‘/g, "\u2018")

 .replace(/’/g, "\u2019")

 .replace(/–/g, "\u2013")

 .replace(/—/g, "\u2014")

```

.replace(/&hellip;/g, "\u2026")
.replace(/&#(\d+)/g, (_, code: string) => String.fromCharCode(Number(code)))
.replace(/&#x([0-9a-fA-F]+)/g, (_, hex: string) =>
  String.fromCharCode(parseInt(hex, 16)),
);
}

```

```

// — Session token cache —————

```

```

let sessionToken: string | null = null;
let tokenFetchedAt: number | null = null;
const TOKEN_TTL_MS = 6 * 60 * 60 * 1000;

async function getSessionToken(): Promise<string | null> {
  if (
    sessionToken &&
    tokenFetchedAt &&
    Date.now() - tokenFetchedAt < TOKEN_TTL_MS
  ){
    return sessionToken;
  }
  try {
    const resp = await fetch(
      `${OPENTDB_BASE}/api_token.php?command=request`,
    );
    const data = (await resp.json()) as {

```

```

    response_code: number;

    token: string;
};

if (data.response_code === 0 && data.token) {
    sessionToken = data.token;
    tokenFetchedAt = Date.now();
    return sessionToken;
}
} catch (err) {
    logger.warn({ err }, "Failed to fetch OpenTDB session token");
}
return null;
}

```

```

async function resetSessionToken(): Promise<void> {
    if (!sessionToken) return;

    try {
        await fetch(
            `${OPENTDB_BASE}/api_token.php?command=reset&token=${sessionToken}`,
        );
        tokenFetchedAt = Date.now();
    } catch {
        sessionToken = null;
        tokenFetchedAt = null;
    }
}

```

// — Internal types —

```
interface RawOpenTdbResult {  
  category: string;  
  type: "multiple" | "boolean";  
  difficulty: "easy" | "medium" | "hard";  
  question: string;  
  correct_answer: string;  
  incorrect_answers: string[];  
}
```

```
interface RawOpenTdbResponse {  
  response_code: number;  
  results?: RawOpenTdbResult[];  
}
```

// — Public types —

```
export interface OpenTdbQuestion {  
  questionText: string;  
  questionType: "multiple_choice" | "true_false";  
  correctAnswer: string;  
  options: { choices: string[] } | null;  
  points: number;  
  orderIndex: number;
```

```
    source: string;
}
```

```
export type FetchQuestionsError =
  | { code: "rate_limited" }
  | { code: "no_results" }
  | { code: "invalid_params" }
  | { code: "fetch_failed"; message: string };
```

```
export type FetchQuestionsResult =
  | { ok: true; questions: OpenTdbQuestion[] }
  | { ok: false; error: FetchQuestionsError };
```

```
export interface FetchQuestionsOptions {
  amount: number;
  categoryId?: number;
  difficulty?: "easy" | "medium" | "hard";
}
```

```
// ——— Helpers
```

```
function shuffleArray<T>(arr: T[]): T[] {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
```



```
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}
```

```
const POINTS_MAP: Record<string, number> = { easy: 5, medium: 10, hard: 15 };
```

```
function mapResult(r: RawOpenTdbResult, index: number): OpenTdbQuestion {
  const questionText = decodeHtml(r.question);
  const correctAnswer = decodeHtml(r.correct_answer);
  const incorrectAnswers = r.incorrect_answers.map(decodeHtml);
  const points = POINTS_MAP[r.difficulty] ?? 10;

  if (r.type === "boolean") {
    return {
      questionText,
      questionType: "true_false",
      correctAnswer: correctAnswer.toLowerCase(),
      options: null,
      points,
      orderIndex: index,
      source: "opentdb",
    };
  }
}
```

```
const choices = shuffleArray([correctAnswer, ...incorrectAnswers]);
```

```
return {  
  questionText,  
  questionType: "multiple_choice",  
  correctAnswer,  
  options: { choices },  
  points,  
  orderIndex: index,  
  source: "opentdb",  
};  
}
```

```
// — Public API —
```

```
export async function fetchOpenTdbQuestions(  
  opts: FetchQuestionsOptions,  
  _retried = false,  
): Promise<FetchQuestionsResult> {  
  const token = await getSessionToken();  
  
  const params = new URLSearchParams({ amount: String(opts.amount) });  
  if (opts.categoryId) params.set("category", String(opts.categoryId));  
  if (opts.difficulty) params.set("difficulty", opts.difficulty);  
  if (token) params.set("token", token);  
  
  let resp: Response;  
  try {
```

```
    resp = await fetch(` ${OPENTDB_BASE}/api.php?${params.toString()} ` );
  } catch (err) {
    return {
      ok: false,
      error: { code: "fetch_failed", message: String(err) },
    };
  }
}
```

```
if (!resp.ok) {
  return {
    ok: false,
    error: { code: "fetch_failed", message: ` HTTP ${resp.status} ` },
  };
}
```

```
const data = (await resp.json()) as RawOpenTdbResponse;
```

```
switch (data.response_code) {
  case 0:
    return {
      ok: true,
      questions: (data.results ?? []).map(mapResult),
    };
  case 1:
    return { ok: false, error: { code: "no_results" } };
  case 2:
```

```
return { ok: false, error: { code: "invalid_params" } };
```

case 3:

```
// Token not found — clear and retry once without token
```

```
if (!_retried) {
```

```
    sessionToken = null;
```

```
    tokenFetchedAt = null;
```

```
    return fetchOpenTdbQuestions(opts, true);
```

```
}
```

```
return { ok: false, error: { code: "fetch_failed", message: "Token error" } };
```

case 4:

```
// Token exhausted — reset and retry once
```

```
if (!_retried) {
```

```
    await resetSessionToken();
```

```
    return fetchOpenTdbQuestions(opts, true);
```

```
}
```

```
return { ok: false, error: { code: "no_results" } };
```

case 5:

```
return { ok: false, error: { code: "rate_limited" } };
```

default:

```
return {
```

```
    ok: false,
```

```
    error: {
```

```
        code: "fetch_failed",
```

```
        message: `Unknown response code: ${data.response_code}`,
```

```
    },
```

```
};
```

```
}  
}
```

```
```\n
```

```

```

```
`lib/db/src/schema/index.ts`
```

```
```typescript
```

```
export * from "./users";  
export * from "./games";  
export * from "./questions";  
export * from "./answers";  
export * from "./gameParticipants";  
export * from "./adminSettings";
```

```
```\n
```

```

```

```
`lib/db/src/schema/users.ts`
```

```
```typescript
```

```
import { pgTable, text, serial, timestamp } from "drizzle-orm/pg-core";  
import { createInsertSchema } from "drizzle-zod";
```

```
import { z } from "zod/v4";
```

```
export const usersTable = pgTable("users", {  
  id: serial("id").primaryKey(),  
  name: text("name").notNull(),  
  createdAt: timestamp("created_at", { withTimezone: true })  
    .notNull()  
    .defaultNow(),  
});
```

```
export const insertUserSchema = createInsertSchema(usersTable).omit({  
  id: true,  
  createdAt: true,  
});  
  
export type InsertUser = z.infer<typeof insertUserSchema>;  
export type User = typeof usersTable.$inferSelect;
```

```
````
```

```

```

```
`lib/db/src/schema/games.ts`
```

```
```typescript
```

```
import {  
  pgTable,
```

```
text,
serial,
integer,
boolean,
timestamp,
} from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod/v4";

export const gamesTable = pgTable("games", {
  id: serial("id").primaryKey(),
  topic: text("topic").notNull(),
  difficulty: text("difficulty", {
    enum: ["easy", "medium", "hard"],
  }).notNull(),
  questionCount: integer("question_count").notNull().default(0),
  status: text("status", { enum: ["waiting", "active", "completed"] })
    .notNull()
    .default("waiting"),
  createdAt: timestamp("created_at", { withTimezone: true })
    .notNull()
    .defaultNow(),
  createdByAdmin: boolean("created_by_admin").notNull().default(true),
});

export const insertGameSchema = createInsertSchema(gamesTable).omit({
```

```
    id: true,  
    createdAt: true,  
  });  
  
export type InsertGame = z.infer<typeof insertGameSchema>;  
export type Game = typeof gamesTable.$inferSelect;
```

```
````
```

```

```

```
`lib/db/src/schema/questions.ts`
```

```
```typescript
```

```
import {  
  pgTable,  
  text,  
  serial,  
  integer,  
  jsonb,  
  boolean,  
} from "drizzle-orm/pg-core";  
  
import { createInsertSchema } from "drizzle-zod";  
  
import { z } from "zod/v4";  
  
import { gamesTable } from "../games";  
  
  
export const questionsTable = pgTable("questions", {
```



```

id: serial("id").primaryKey(),
gameId: integer("game_id")
    .notNull()
    .references(() => gamesTable.id, { onDelete: "cascade" }),
questionText: text("question_text").notNull(),
questionType: text("question_type", {
    enum: [
        "multiple_choice",
        "write_in",
        "matching",
        "image_recognition",
        "true_false",
    ],
}).notNull(),
correctAnswer: text("correct_answer").notNull(),
options: jsonb("options"),
imageUrl: text("image_url"),
points: integer("points").notNull().default(10),
orderIndex: integer("order_index").notNull().default(0),
source: text("source"),
factCheckUrl: text("fact_check_url"),
aiGenerated: boolean("ai_generated").notNull().default(false),
verifiedByAdmin: boolean("verified_by_admin").notNull().default(false),
});

export const insertQuestionSchema = createInsertSchema(questionsTable).omit({

```

```
    id: true,  
  });  
  
export type InsertQuestion = z.infer<typeof insertQuestionSchema>;  
export type Question = typeof questionsTable.$inferSelect;
```

```
```\n
```

```

```

```
`lib/db/src/schema/answers.ts`
```

```
```typescript
```

```
import {  
  pgTable,  
  text,  
  serial,  
  integer,  
  boolean,  
  timestamp,  
} from "drizzle-orm/pg-core";  
  
import { createInsertSchema } from "drizzle-zod";  
  
import { z } from "zod/v4";  
  
import { usersTable } from "../users";  
  
import { gamesTable } from "../games";  
  
import { questionsTable } from "../questions";
```

```

export const answersTable = pgTable("answers", {
  id: serial("id").primaryKey(),
  userId: integer("user_id")
    .notNull()
    .references(() => usersTable.id, { onDelete: "cascade" }),
  gameId: integer("game_id")
    .notNull()
    .references(() => gamesTable.id, { onDelete: "cascade" }),
  questionId: integer("question_id")
    .notNull()
    .references(() => questionsTable.id, { onDelete: "cascade" }),
  userAnswer: text("user_answer").notNull(),
  isCorrect: boolean("is_correct").notNull(),
  pointsEarned: integer("points_earned").notNull().default(0),
  answeredAt: timestamp("answered_at", { withTimezone: true })
    .notNull()
    .defaultNow(),
});

```

```

export const insertAnswerSchema = createInsertSchema(answersTable).omit({
  id: true,
  answeredAt: true,
});

export type InsertAnswer = z.infer<typeof insertAnswerSchema>;
export type Answer = typeof answersTable.$inferSelect;

```

```

---

## `lib/db/src/schema/gameParticipants.ts`

```typescript

```
import {  
  pgTable,  
  serial,  
  integer,  
  timestamp,  
} from "drizzle-orm/pg-core";  
  
import { createInsertSchema } from "drizzle-zod";  
import { z } from "zod/v4";  
  
import { usersTable } from "../users";  
import { gamesTable } from "../games";  
  
export const gameParticipantsTable = pgTable("game_participants", {  
  id: serial("id").primaryKey(),  
  gameId: integer("game_id")  
    .notNull()  
    .references(() => gamesTable.id, { onDelete: "cascade" }),  
  userId: integer("user_id")  
    .notNull()  
    .references(() => usersTable.id, { onDelete: "cascade" }),  
});
```

```
totalScore: integer("total_score").notNull().default(0),
joinedAt: timestamp("joined_at", { withTimezone: true })
  .notNull()
  .defaultNow(),
});
```

```
export const insertGameParticipantSchema = createInsertSchema(
  gameParticipantsTable,
).omit({ id: true, joinedAt: true });
export type InsertGameParticipant = z.infer<
  typeof insertGameParticipantSchema
>;
export type GameParticipant = typeof gameParticipantsTable.$inferSelect;
```

```
...
```

```
---
```

```
## `lib/db/src/schema/adminSettings.ts`
```

```
```typescript
```

```
import { pgTable, text, serial } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod/v4";
```

```
export const adminSettingsTable = pgTable("admin_settings", {
```

```
id: serial("id").primaryKey(),
triviaAccessCode: text("trivia_access_code").notNull(),
adminAccessCode: text("admin_access_code").notNull(),
});
```

```
export const insertAdminSettingsSchema = createInsertSchema(
 adminSettingsTable,
).omit({ id: true });
export type InsertAdminSettings = z.infer<typeof insertAdminSettingsSchema>;
export type AdminSettings = typeof adminSettingsTable.$inferSelect;
```

```
` ``
```

```

```

```
`lib/db/src/index.ts`
```

```
` `` typescript
```

```
import { drizzle } from "drizzle-orm/node-postgres";
```

```
import pg from "pg";
```

```
import * as schema from "../schema";
```

```
const { Pool } = pg;
```

```
if (!process.env.DATABASE_URL) {
```

```
 throw new Error(
```

```

 "DATABASE_URL must be set. Did you forget to provision a database?",
);
}

export const pool = new Pool({ connectionString: process.env.DATABASE_URL });
export const db = drizzle(pool, { schema });

export * from "./schema";

` ``

`lib/api-spec/openapi.yaml`

`` `yaml
openapi: 3.1.0
info:
 # Do not change the title, if the title changes, the import paths will be broken
 title: Api
 version: 0.1.0
 description: Trivia game API
servers:
 - url: /api
 description: Base API path
tags:

```

- name: health

description: Health operations

- name: auth

description: Access code verification

- name: users

description: Player management

- name: games

description: Game management

- name: questions

description: Question management

- name: play

description: Gameplay operations

- name: stats

description: Aggregate statistics

paths:

/healthz:

get:

operationId: healthCheck

tags: [health]

summary: Health check

description: Returns server health status

responses:

"200":

description: Healthy

content:

application/json:



schema:

\$ref: "#/components/schemas/HealthStatus"

/auth/verify:

post:

operationId: verifyAccessCode

tags: [auth]

summary: Verify a trivia or admin access code

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/AccessCodeInput"

responses:

"200":

description: Verification result

content:

application/json:

schema:

\$ref: "#/components/schemas/AccessCodeResult"

/users:

post:

operationId: createUser

tags: [users]

summary: Create a player by name

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/UserInput"

responses:

"201":

description: Created

content:

application/json:

schema:

\$ref: "#/components/schemas/User"

/users/{userId}:

get:

operationId: getUser

tags: [users]

summary: Get a player

parameters:

- name: userId

in: path

required: true

schema: { type: integer }

responses:

"200":

description: The user

content:

application/json:

schema:

\$ref: "#/components/schemas/User"

"404":

description: Not found

/games:

get:

operationId: listGames

tags: [games]

summary: List games, optionally by status

parameters:

- name: status

in: query

required: false

schema:

type: string

enum: [waiting, active, completed]

responses:

"200":

description: Games

content:

application/json:

schema:

type: array

items:

\$ref: "#/components/schemas/Game"

post:

operationId: createGame

tags: [games]

summary: Create a game (admin)

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/GameInput"

responses:

"201":

description: Created

content:

application/json:

schema:

\$ref: "#/components/schemas/Game"

/games/{gameId}:

get:

operationId: getGame

tags: [games]

summary: Get a game with participant and question counts

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

responses:

"200":

description: The game

content:

application/json:

schema:

\$ref: "#/components/schemas/GameDetail"

"404":

description: Not found

patch:

operationId: updateGame

tags: [games]

summary: Update a game (status, topic, difficulty)

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/GameUpdate"

responses:

"200":

description: Updated

content:

application/json:

schema:

\$ref: "#/components/schemas/Game"

"404":

description: Not found

delete:

operationId: deleteGame

tags: [games]

summary: Delete a game

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

responses:

"204":

description: Deleted

"404":

description: Not found

/games/{gameId}/questions/import-opentdb:

post:

operationId: importOpenTdbQuestions

tags: [questions]

summary: Import questions from Open Trivia Database (admin)

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/OpenTdbImportInput"

responses:

"200":

description: Questions imported successfully

content:

application/json:

schema:

\$ref: "#/components/schemas/OpenTdbImportResult"

"404":

description: Game not found

"429":

description: Rate limited by OpenTDB

/games/{gameId}/questions/generate-gemini:

post:

operationId: generateGeminiQuestions

tags: [questions]

summary: Generate questions via Google Gemini for custom topics (admin)

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/GeminiGenerateInput"

responses:

"200":

description: Questions generated successfully

content:

application/json:

schema:

\$ref: "#/components/schemas/OpenTdbImportResult"

"404":

description: Game not found

"503":

description: Gemini API unavailable or key missing

/games/{gameId}/questions/{questionId}/regenerate:

post:



operationId: regenerateQuestion

tags: [questions]

summary: Regenerate a single AI question preview using Gemini (not saved)

parameters:

- name: gameId

  - in: path

  - required: true

  - schema: { type: integer }

- name: questionId

  - in: path

  - required: true

  - schema: { type: integer }

requestBody:

required: true

content:

- application/json:

  - schema:

    - \$ref: "#/components/schemas/RegenerateQuestionInput"

responses:

"200":

description: New question preview (not yet saved to DB)

content:

- application/json:

  - schema:

    - \$ref: "#/components/schemas/RegenerateQuestionPreview"

"404":

description: Question or game not found

"503":

description: Gemini unavailable or key missing

/games/{gameId}/questions/{questionId}/enhance:

post:

operationId: enhanceQuestion

tags: [questions]

summary: Analyze a question and suggest improvements using Gemini

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

- name: questionId

in: path

required: true

schema: { type: integer }

responses:

"200":

description: Enhancement suggestions

content:

application/json:

schema:

\$ref: "#/components/schemas/EnhanceQuestionResult"

"404":

description: Not found

"503":

description: Gemini unavailable

/games/{gameId}/questions/{questionId}/fact-check:

post:

operationId: factCheckQuestion

tags: [questions]

summary: Fact-check a single question using Gemini

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

- name: questionId

in: path

required: true

schema: { type: integer }

responses:

"200":

description: Fact check result

content:

application/json:

schema:

\$ref: "#/components/schemas/FactCheckSingleResult"

"404":

description: Not found

"503":

description: Gemini unavailable

/games/{gameId}/questions:

get:

operationId: listGameQuestions

tags: [questions]

summary: List questions of a game ordered by order\_index

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

responses:

"200":

description: Questions

content:

application/json:

schema:

type: array

items:

\$ref: "#/components/schemas/Question"

post:

operationId: createQuestion

tags: [questions]

summary: Add a question to a game (admin)

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/QuestionInput"

responses:

"201":

description: Created

content:

application/json:

schema:

\$ref: "#/components/schemas/Question"

/questions/{questionId}:

patch:

operationId: updateQuestion

tags: [questions]

summary: Update a question (admin)

parameters:

- name: questionId

in: path

required: true

schema: { type: integer }

requestBody:  
 required: true  
 content:  
 application/json:  
 schema:  
 \$ref: "#/components/schemas/QuestionUpdate"

responses:  
 "200":  
 description: Updated  
 content:  
 application/json:  
 schema:  
 \$ref: "#/components/schemas/Question"

"404":  
 description: Not found

delete:  
 operationId: deleteQuestion  
 tags: [questions]  
 summary: Delete a question (admin)  
 parameters:  
 - name: questionId  
 in: path  
 required: true  
 schema: { type: integer }

responses:  
 "204":

description: Deleted

"404":

description: Not found

/games/{gameId}/join:

post:

operationId: joinGame

tags: [play]

summary: Join a game as a participant

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

requestBody:

required: true

content:

application/json:

schema:

\$ref: "#/components/schemas/JoinGameInput"

responses:

"201":

description: Joined (or already joined)

content:

application/json:

schema:

\$ref: "#/components/schemas/Participant"

"404":

description: Game not found

/games/{gameId}/participants:

get:

operationId: listGameParticipants

tags: [play]

summary: Leaderboard - participants with scores, sorted descending

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

responses:

"200":

description: Participants with names and scores

content:

application/json:

schema:

type: array

items:

\$ref: "#/components/schemas/ParticipantWithUser"

/games/{gameId}/answers:

post:

operationId: submitAnswer

tags: [play]

summary: Submit an answer to a question



parameters:

- name: gameId

- in: path

- required: true

- schema: { type: integer }

requestBody:

- required: true

- content:

- application/json:

- schema:

- \$ref: "#/components/schemas/AnswerInput"

responses:

- "201":

- description: Answer graded and recorded

- content:

- application/json:

- schema:

- \$ref: "#/components/schemas/AnswerResult"

- "404":

- description: Game or question not found

- "409":

- description: Already answered this question

/games/{gameId}/users/{userId}/answers:

get:

- operationId: listUserAnswers

- tags: [play]

summary: List a user's answers for a game

parameters:

- name: gameId

in: path

required: true

schema: { type: integer }

- name: userId

in: path

required: true

schema: { type: integer }

responses:

"200":

description: Answers

content:

application/json:

schema:

type: array

items:

\$ref: "#/components/schemas/Answer"

/stats/summary:

get:

operationId: getStatsSummary

tags: [stats]

summary: Aggregate stats for dashboard

responses:

"200":

description: Summary counts

content:

application/json:

schema:

\$ref: "#/components/schemas/StatsSummary"

components:

schemas:

HealthStatus:

type: object

properties:

status:

type: string

required:

- status

AccessCodeInput:

type: object

required: [code]

properties:

code: { type: string, minLength: 1 }

AccessCodeResult:

type: object

required: [valid, role]

properties:

valid: { type: boolean }

role:

type: string

enum: [player, admin, none]

UserInput:

type: object

required: [name]

properties:

name: { type: string, minLength: 1, maxLength: 80 }

User:

type: object

required: [id, name, createdAt]

properties:

id: { type: integer }

name: { type: string }

createdAt: { type: string }

GameInput:

type: object

required: [topic, difficulty]

properties:

topic: { type: string, minLength: 1 }

difficulty:

type: string

enum: [easy, medium, hard]

createdByAdmin: { type: boolean }

GameUpdate:

type: object

properties:

topic: { type: string, minLength: 1 }

difficulty:

type: string

enum: [easy, medium, hard]

status:

type: string

enum: [waiting, active, completed]

Game:

type: object

required: [id, topic, difficulty, questionCount, status, createdAt, createdByAdmin]

properties:

id: { type: integer }

topic: { type: string }

difficulty:

type: string

enum: [easy, medium, hard]

questionCount: { type: integer }

status:

type: string

enum: [waiting, active, completed]

createdAt: { type: string }

createdByAdmin: { type: boolean }

GameDetail:

type: object

required: [id, topic, difficulty, questionCount, status, createdAt, createdByAdmin, participantCount]

properties:

id: { type: integer }

topic: { type: string }

difficulty:

type: string

enum: [easy, medium, hard]

questionCount: { type: integer }

status:

type: string

enum: [waiting, active, completed]

createdAt: { type: string }

createdByAdmin: { type: boolean }

participantCount: { type: integer }

QuestionInput:

type: object

required: [questionText, questionType, correctAnswer, points, orderIndex]

properties:

questionText: { type: string, minLength: 1 }

questionType:

type: string

enum: [multiple\_choice, write\_in, matching, image\_recognition, true\_false]

correctAnswer: { type: string, minLength: 1 }

options:

type: ["object", "null"]

additionalProperties: true

imageUrl: { type: ["string", "null"] }

points: { type: integer, minimum: 1 }

orderIndex: { type: integer, minimum: 0 }

source: { type: ["string", "null"] }

factCheckUrl: { type: ["string", "null"] }

verifiedByAdmin: { type: boolean }

aiGenerated: { type: boolean }

QuestionUpdate:

type: object

properties:

questionText: { type: string, minLength: 1 }

questionType:

type: string

enum: [multiple\_choice, write\_in, matching, image\_recognition, true\_false]

correctAnswer: { type: string, minLength: 1 }

options:

type: ["object", "null"]

additionalProperties: true

imageUrl: { type: ["string", "null"] }

points: { type: integer, minimum: 1 }

orderIndex: { type: integer, minimum: 0 }

source: { type: ["string", "null"] }

factCheckUrl: { type: ["string", "null"] }

verifiedByAdmin: { type: boolean }

RegenerateQuestionInput:

type: object

properties:

difficulty:

type: string

enum: [easy, medium, hard]

questionType:

type: string

enum: [multiple\_choice, true\_false, write\_in]

RegenerateQuestionPreview:

type: object

required: [questionType, questionText, correctAnswer, points, source]

properties:

questionType: { type: string }

questionText: { type: string }

correctAnswer: { type: string }

options:

type: array

nullable: true

items: { type: string }

points: { type: integer }

source: { type: string }

EnhanceQuestionResult:

type: object

required: [improvedQuestionText, factCheckResult, factCheckNotes, suggestedSource, suggestions]

properties:

improvedQuestionText: { type: string }

improvedOptions:

type: array



nullable: true

items: { type: string }

factCheckResult:

type: string

enum: [VERIFIED, UNCERTAIN, LIKELY\_INCORRECT]

factCheckNotes: { type: string }

suggestedSource: { type: string }

suggestions: { type: string }

FactCheckSingleResult:

type: object

required: [verdict, confidence, explanation]

properties:

verdict:

type: string

enum: [CORRECT, INCORRECT, UNCERTAIN]

confidence:

type: string

enum: [HIGH, MEDIUM, LOW]

explanation: { type: string }

correctAnswerIfWrong:

type: string

nullable: true

GeminiGenerateInput:

type: object

required: [topic, difficulty, amount]

properties:

topic: { type: string, minLength: 1 }

difficulty:

type: string

enum: [easy, medium, hard]

amount:

type: integer

minimum: 1

maximum: 20

existingQuestions:

type: array

items: { type: string }

OpenTdbImportInput:

type: object

required: [categoryId, difficulty, amount]

properties:

categoryId: { type: integer }

difficulty:

type: string

enum: [easy, medium, hard]

amount:

type: integer

minimum: 1

maximum: 50

OpenTdbImportResult:

type: object

required: [imported, total]

properties:

imported: { type: integer }

total: { type: integer }

Question:

type: object

required: [id, gameId, questionText, questionType, options, imageUrl, points, orderIndex, verifiedByAdmin, aiGenerated]

properties:

id: { type: integer }

gameId: { type: integer }

questionText: { type: string }

questionType:

type: string

enum: [multiple\_choice, write\_in, matching, image\_recognition, true\_false]

correctAnswer: { type: string }

options:

type: ["object", "null"]

additionalProperties: true

imageUrl: { type: ["string", "null"] }

points: { type: integer }

orderIndex: { type: integer }

source: { type: ["string", "null"] }

factCheckUrl: { type: ["string", "null"] }

verifiedByAdmin: { type: boolean }

aiGenerated: { type: boolean }

JoinGameInput:

type: object

Participant:

type: object

required: [id, gameId, userId, totalScore, joinedAt]

properties:

id: { type: integer }

gameId: { type: integer }

userId: { type: integer }

totalScore: { type: integer }

joinedAt: { type: string }

ParticipantWithUser:

type: object

required: [id, gameId, userId, userName, totalScore, joinedAt]

properties:

id: { type: integer }

gameId: { type: integer }

userId: { type: integer }

userName: { type: string }

totalScore: { type: integer }

joinedAt: { type: string }

AnswerInput:

type: object

required: [questionId, userAnswer]

properties:

questionId: { type: integer }

userAnswer: { type: string }

Answer:

type: object

required: [id, userId, gameId, questionId, userAnswer, isCorrect, pointsEarned, answeredAt]

properties:

id: { type: integer }

userId: { type: integer }

gameId: { type: integer }

questionId: { type: integer }

userAnswer: { type: string }

isCorrect: { type: boolean }

pointsEarned: { type: integer }

answeredAt: { type: string }

correctAnswer: { type: string }

AnswerResult:

type: object

required: [id, userId, gameId, questionId, userAnswer, isCorrect, answeredAt, pointsEarned, totalScore]

properties:

id: { type: integer }

userId: { type: integer }

gameId: { type: integer }

questionId: { type: integer }

userAnswer: { type: string }

isCorrect: { type: boolean }

answeredAt: { type: string }

pointsEarned: { type: integer }

totalScore: { type: integer }

StatsSummary:

type: object

required: [totalGames, activeGames, totalPlayers, totalAnswers]

properties:

totalGames: { type: integer }

activeGames: { type: integer }

totalPlayers: { type: integer }

totalAnswers: { type: integer }

...